

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

30 个 Kernel · 9 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 7 月 8 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (~30 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler x4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit (4 bytes) = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory
49 //   → blocks/SM; block 大小 → blocks/SM
50 //
51 // ---- 常见优化手段速查清单 ----
52 //
53 // 1. Coalesced Memory Access (合并访问)
54 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
55 //   - 否则产生多次事务 (最坏 32 次)
56 //
57 // 2. Tiling (分块)
58 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
59 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
60 //
61 // 3. Vectorized Memory Access (向量化)
62 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数

```

```

63 // - float4 = 128-bit, 单条指令加载 16 bytes
64 //
65 // 4. Thread Tile (寄存器分块)
66 // - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
67 // - 减少线程总数, 降低同步开销
68 //
69 // 5. Bank Conflict Avoidance
70 // - Shared memory 有 32 banks × 4 bytes
71 // - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
72 // - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
73 //
74 // 6. Pipeline / Double Buffering (流水线)
75 // - cp.async 异步拷贝下一批数据时同时做当前批的计算
76 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
77 //
78 // 7. Tensor Core (MMA / WGMMa)
79 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
80 // - WGMMa m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
81 //
82 // 8. Warp Specialization (Hopper+)
83 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
84 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
85 //
86 // 9. TMA (Tensor Memory Accelerator, Hopper+)
87 // - 硬件 DMA 引擎, 支持 1D~5D 寻址, 低寄存器开销
88 // - 配合 cp.async.bulk 实现异步数据搬运
89 //
90 // ---- Roofline 分析公式 ----
91 //
92 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
93 //
94 // GEMM (M=N=K=4096):
95 // FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPs
96 // Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
97 // AI ≈ 137G / 200M ≈ 685 FLOPs/Byte → compute-bound (远超 H100 ridge point:
98 // FP16 TC ≈ 295:1, FP32 ≈ 20:1)
99 //
100 // GEMV (M=4096, K=4096):
101 // FLOPs = 2 × M × K = 2 × 40962 ≈ 33 MFLOPs
102 // Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
103 // AI ≈ 33M / 67M ≈ 0.5 FLOPs/Byte → severely memory-bound
104 //
105 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPs/Byte → memory-bound
106 //
107 // =====
108 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
109 // =====
110 // 面试要点:
111 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
112 //   memory
113 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
114 //   注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
115 // - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
116 //
117 #include <algorithm>
118 #include <cstring>
119 #include <cuda_fp16.h>
120 #include <cuda_runtime.h>
121 #include <float.h>
122 #include <stdio.h>
123 #include <stdlib.h>
124 #include <math.h>
125 #include <cublas_v2.h>

```

```

126 #include <cuda.h>
127 #include <cuda/barrier>
128
129 #define WARP_SIZE 32
130 #define INT4(value) (reinterpret_cast<int4*>(&(value)))[0])
131 #define FLOAT4(value) (reinterpret_cast<float4*>(&(value)))[0])
132 #define HALF2(value) (reinterpret_cast<half2*>(&(value)))[0])
133
134 // =====
135 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
136 // =====
137
138 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
139 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
140 // 复杂度  $O(\log N)$ ,  $N=32$  时仅需 5 步
141 // 模板参数: T=数据类型, kWarpSize=segment width (默认 32)
142 // kWarpSize 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
143 // 当 kWarpSize < 32 (如 FA 中 kWarpSize=4) 时, 只有同 segment 的 lane 参与通信
144 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
145 //
146 // 初始: 每个 lane 持有自己的值 v0..v7
147 // lane:  0    1    2    3    4    5    6    7
148 // val:  v0   v1   v2   v3   v4   v5   v6   v7
149 //
150 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
151 //
152 //      ┌──────────┐
153 // 对:   (0,4) (1,5) (2,6) (3,7)
154 //
155 // lane:  0    1    2    3    4    5    6    7
156 // val:  v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
157 //
158 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
159 //
160 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
161 // 对:   (0,2) (1,3) (4,6) (5,7)
162 //      ─── 前4个一组 ───      ─── 后4个一组 ───
163 //
164 // lane:  0    1    2    3    4    5    6    7 (每lane持有前一轮2个值的和再加本轮配对)
165 // val:   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$ 
166 //      = v0+v2+v4+v6 ... (逐步归约)
167 //
168 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
169 //
170 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
171 // 对:   (0,1) (2,3) (4,5) (6,7)
172 //
173 // lane:  0    1    2    3    4    5    6    7
174 // val:   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$  ← 所有 lane 拥有全归约结果!
175 //
176 // mask=0: 循环终止, 归约完成。
177 //
178 // 关键性质:
179 // - XOR 配对是对称的: lane i 的配对对象是 lane  $i^{mask}$ , 而  $(i^{mask})^{mask} = i$ 
180 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
181 // - 每轮信息传递距离减半:  $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$  (距离减半, 信息翻倍)
182 // -  $O(\log_2 N)$  步完成, 无需 shared memory, 纯寄存器操作
183 // - __shfl_xor_sync 第四个参数 kWarpSize 限制 segment width:
184 //   当 kWarpSize=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
185
186 template <const int kWarpSize = WARP_SIZE, typename T = float>
187 __device__ __forceinline__ T warp_reduce_sum(T val) {
188 #pragma unroll
189     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
190         val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
191     }
192     return val;
193 }

```

```

189 }
190
191 // Warp Reduce Max — generic
192 template <const int kWarpSize = WARP_SIZE, typename T = float>
193 __device__ __forceinline__ T warp_reduce_max(T val) {
194 #pragma unroll
195     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
196         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpSize));
197     }
198     return val;
199 }
200
201 // =====
202 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
203 // =====
204
205 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
206 // 两级归约流程:
207 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
208 // 2. warp leader (lane=0) 写入 shared memory
209 // 3. syncthreads 后, lane 0~NUM_WARPS-1 读取 shared memory
210 // 4. 所有 warp 内再做一次 warp_reduce<NUM_WARPS> → 得到最终结果
211 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<NUM_WARPS 知道结果)
212 template <const int NUM_THREADS = 256>
213 __device__ float block_reduce_sum(float val) {
214     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
215     int warp = threadIdx.x / WARP_SIZE;
216     int lane = threadIdx.x % WARP_SIZE;
217     __shared__ float shared[NUM_WARPS];
218
219     float value = warp_reduce_sum<WARP_SIZE>(val);
220     if (lane == 0)
221         shared[warp] = value;
222     __syncthreads();
223     value = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
224     value = warp_reduce_sum<NUM_WARPS>(value);
225     // 关键: broadcast 结果到所有线程, 后续用 result
226     // 做除法等操作时所有线程都能拿到
227     value = __shfl_sync(0xffffffff, value, 0, 32);
228     return value;
229 }
230
231 // Block Reduce Max — FP32 (增强版, 带 broadcast)
232 template <const int NUM_THREADS = 256>
233 __device__ float block_reduce_max(float val) {
234     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
235     int warp = threadIdx.x / WARP_SIZE;
236     int lane = threadIdx.x % WARP_SIZE;
237     __shared__ float shared[NUM_WARPS];
238
239     float value = warp_reduce_max<WARP_SIZE>(val);
240     if (lane == 0)
241         shared[warp] = value;
242     __syncthreads();
243     value = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
244     value = warp_reduce_max<NUM_WARPS>(value);
245     value = __shfl_sync(0xffffffff, value, 0, 32);
246     return value;
247 }
248
249 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
250 // 多 block 各自做 warp→smem→warp0 reduce, 然后 atomicAdd 到全局 y
251 // 跨 block 求和的常见模式, 适合 N 较大时使用;

```

```

252 // Grid: ((N + 255) / 256, 1, 1)
253 // Block: (256, 1, 1)
254 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
255 template <const int NUM_THREADS = 256>
256 __global__ void block_reduce_all(float *a, float *y, int N) {
257     int tid = threadIdx.x;
258     int idx = blockIdx.x * NUM_THREADS + tid;
259     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
260     __shared__ float shared[NUM_WARPS];
261
262     float val = (idx < N) ? a[idx] : 0.0f;
263     int warp = tid / WARP_SIZE;
264     int lane = tid % WARP_SIZE;
265
266     val = warp_reduce_sum<WARP_SIZE>(val);
267     if (lane == 0)
268         shared[warp] = val;
269     __syncthreads();
270
271     val = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
272     if (warp == 0)
273         val = warp_reduce_sum<NUM_WARPS>(val);
274     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果，避免重复累加
275         atomicAdd(y, val);
276 }
277
278 // ---- Dot Product: y = sum(a[i] * b[i]) ----
279 // 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
280 // Grid: ((N + 255) / 256, 1, 1)
281 // Block: (256, 1, 1)
282 // source: LeetCUDA/kernels/dot-product/dot_product.cu
283 template <const int NUM_THREADS = 256>
284 __global__ void dot(float *a, float *b, float *y, int N) {
285     int tid = threadIdx.x;
286     int idx = blockIdx.x * NUM_THREADS + tid;
287     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
288     __shared__ float shared[NUM_WARPS];
289
290     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
291     int warp = tid / WARP_SIZE;
292     int lane = tid % WARP_SIZE;
293
294     prod = warp_reduce_sum<WARP_SIZE>(prod);
295     if (lane == 0)
296         shared[warp] = prod;
297     __syncthreads();
298
299     prod = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
300     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
301         prod = warp_reduce_sum<NUM_WARPS>(prod);
302     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果，避免重复累加
303         atomicAdd(y, prod);
304 }
305
306 // Dot Product + float4
307 // Grid: ((N + 255) / 256, 1, 1), 每个block处理256元素，float4向量化后每个线程处理4元素
308 // Block: (64, 1, 1), 256/4=64
309 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
310 // source: LeetCUDA/kernels/dot-product/dot_product.cu
311 template <const int NUM_THREADS = 256 / 4>
312 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
313     int tid = threadIdx.x;
314     int idx = (blockIdx.x * NUM_THREADS + tid) * 4;

```

```

315 constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
316 __shared__ float shared[NUM_WARPS];
317
318 float4 reg_a = FLOAT4(a[idx]);
319 float4 reg_b = FLOAT4(b[idx]);
320 float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
321                          reg_a.z * reg_b.z + reg_a.w * reg_b.w)
322                  : 0.0f;
323 int warp = tid / WARP_SIZE;
324 int lane = tid % WARP_SIZE;
325
326 prod = warp_reduce_sum<WARP_SIZE>(prod);
327 if (lane == 0)
328     shared[warp] = prod;
329 __syncthreads();
330
331 prod = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
332 if (warp == 0)
333     prod = warp_reduce_sum<NUM_WARPS>(prod);
334 if (tid == 0)
335     atomicAdd(y, prod);
336 }
337
338
339 // =====
340 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
341 // =====
342 // 面试要点:
343 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
344 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
345 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
346
347 // ---- ReLU: y = max(0, x) ----
348 // Grid: ((N + 255) / 256, 1, 1)
349 // Block: (256, 1, 1)
350 // source: LeetCUDA/kernels/relu/relu.cu
351 __global__ void relu(float *x, float *y, int N) {
352     int idx = blockIdx.x * blockDim.x + threadIdx.x;
353     if (idx < N)
354         y[idx] = fmaxf(0.0f, x[idx]);
355 }
356
357 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
358 // block(64)*4(float4)=256 元素/block, 与基础版吞吐相同
359 // Grid: ((N + 255) / 256, 1, 1)
360 // Block: (64, 1, 1)
361 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
362 // source: LeetCUDA/kernels/relu/relu.cu
363 __global__ void relu_vec4(float *x, float *y, int N) {
364     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
365     if (idx < N) {
366         float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
367         float4 reg_y;
368         reg_y.x = fmaxf(0.0f, reg_x.x);
369         reg_y.y = fmaxf(0.0f, reg_x.y);
370         reg_y.z = fmaxf(0.0f, reg_x.z);
371         reg_y.w = fmaxf(0.0f, reg_x.w);
372         FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
373     }
374 }
375
376 // ---- Elementwise Add: c = a + b ----
377 // Grid: ((N + 255) / 256, 1, 1)

```

```

378 // Block: (256, 1, 1)
379 // source: LeetCUDA/kernels/elementwise/elementwise.cu
380 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
381     int idx = blockIdx.x * blockDim.x + threadIdx.x;
382     if (idx < N)
383         c[idx] = a[idx] + b[idx];
384 }
385
386 // Elementwise Add + float4 向量化
387 // Grid: ((N + 255) / 256, 1, 1)
388 // Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
389 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
390 // source: LeetCUDA/kernels/elementwise/elementwise.cu
391 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
392     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
393     if ((idx + 3) < N) {
394         float4 reg_a = FLOAT4(a[idx]);
395         float4 reg_b = FLOAT4(b[idx]);
396         float4 reg_c;
397         reg_c.x = reg_a.x + reg_b.x;
398         reg_c.y = reg_a.y + reg_b.y;
399         reg_c.z = reg_a.z + reg_b.z;
400         reg_c.w = reg_a.w + reg_b.w;
401         FLOAT4(c[idx]) = reg_c;
402     } else if (idx < N) {
403         for (int i = 0; (idx + i) < N; i++) {
404             c[idx + i] = a[idx + i] + b[idx + i];
405         }
406     }
407 }
408
409 // ---- Histogram: y[a[i]]++ ----
410 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
411 // Grid: ((N + 255) / 256, 1, 1)
412 // Block: (256, 1, 1)
413 // source: LeetCUDA/kernels/histogram/histogram.cu
414 __global__ void histogram(int *a, int *y, int N) {
415     int idx = blockIdx.x * blockDim.x + threadIdx.x;
416     if (idx < N)
417         atomicAdd(&(y[a[idx]]), 1);
418 }
419
420 // ---- Merge Attn States: 合并 Split-KV Attention 的部分结果 ----
421 // 实现 FlashInfer 论文 (arXiv 2501.01005) Section 2.2 的 attention 合并逻辑。
422 //
423 // 面试要点:
424 // - 应用场景: Split-KV attention 将序列沿 K 维分段计算 attention,
425 //   每段产出部分结果 (O_i, LSE_i), 本 kernel 将两段按指数权重加权合并。
426 // - 数学公式 (5 步推导):
427 //   Step 1 — max 归一化 (数值稳定, 与 safe softmax 同理):
428 //       L_max = max(LSE_1, LSE_2)
429 //   Step 2 — 还原指数权重 (un-normalized):
430 //       w_1 = exp(LSE_1 - L_max), w_2 = exp(LSE_2 - L_max)
431 //   Step 3 — 归一化为混合比例:
432 //       alpha = w_1 / (w_1 + w_2), beta = w_2 / (w_1 + w_2)
433 //       满足 alpha + beta = 1
434 //   Step 4 — 逐元素加权合并:
435 //       O = alpha * O_1 + beta * O_2
436 // - LSE 布局 [num_heads, num_tokens] (与 FlashAttention 惯例一致,
437 //   lse[head_idx][token_idx])
438 // - Output 布局 [num_tokens, num_heads, head_size] (token 维在最外,
439 //   展平为 [T0H0, T0H1, ..., T1H0, ...])
440 // - inf LSE → -inf: 空 attention 段 (causal mask 等导致全部 score

```



```

441 // 为 -inf) 的 LSE 可能为 +inf, 替换后  $\exp(-\text{inf} - L_{\text{max}}) = 0$ ,
442 // 该段权重退化为 0
443 // - 128-bit 向量化: uint4 = 4 × float, 每线程处理 4 个输出元素,
444 // pack load → FMA → pack store 一条流水线
445 // -  $\text{AI} \approx 3 \text{ FLOP} / 20 \text{ bytes} \approx 0.15 \text{ FLOPS/Byte} \rightarrow \text{severely memory-bound}$ 
446 //
447 // 线程到数据的 3D 映射 (以 num_tokens=2, num_heads=2, head_size=8 为例):
448 // pack_size = 16 / sizeof(float) = 4 (每线程 4 个 float = 128-bit)
449 // threads_per_head = head_size / 4 = 2
450 // total_threads = num_tokens * num_heads * threads_per_head = 8
451 //
452 // global_idx:      0      1      2      3      4      5      6      7
453 // token_head_idx:  0      0      1      1      2      2      3      3
454 // (第几个 (token,head) 对, 行优先)
455 // pack_idx:        0      1      0      1      0      1      0      1
456 // (该 (token,head) 对内第几个 pack)
457 // token_idx:       0      0      0      0      1      1      1      1
458 // (token_head_idx / num_heads, token 变化慢)
459 // head_idx:        0      0      1      1      0      0      1      1
460 // (token_head_idx % num_heads, head 变化快)
461 // pack_offset:     0      4      0      4      0      4      0      4
462 // (pack_idx * 4, 该 pack 在 head_size 中的起始元素偏移)
463 // head_offset:     0      0      8      8      16     16     24     24
464 // (token_idx * num_heads * head_size + head_idx * head_size,
465 // 该 (token,head) 对在 output 展平数组中的起始偏移)
466 //
467 // Grid: ((total_threads + 127) / 128, 1, 1)
468 // Block: (128, 1, 1)
469 // source: LeetCUDA/kernels/openai-triton/merge-attn-states/cuda_merge_attn_states.cu
470 __global__ void merge_attn_states(
471     float *output,           // [num_tokens, num_heads, head_size]
472     const float *prefix_output, // [num_tokens, num_heads, head_size]
473     const float *prefix_lse,   // [num_heads, num_tokens]
474     const float *suffix_output, // [num_tokens, num_heads, head_size]
475     const float *suffix_lse,   // [num_heads, num_tokens]
476     int num_tokens, int num_heads, int head_size) {
477
478     constexpr int NUM_THREADS = 128;
479     constexpr int PACK_SIZE = 16 / sizeof(float); // 4 floats = 128-bit
480     using pack_t = uint4;
481
482     // 每个 (token, head) 对需要 threads_per_head 个线程覆盖 head_size 个元素
483     const int threads_per_head = head_size / PACK_SIZE;
484     // 实际有效总线程数, 超过这个数的线程会被忽略, block是按照NUM_THREADS=128
485     // 来分配的, 那么最后一个block的线程数可能会超过实际需要的线程数
486     const int total_threads = num_tokens * num_heads * threads_per_head;
487
488     const int global_idx = blockIdx.x * NUM_THREADS + threadIdx.x;
489     if (global_idx >= total_threads)
490         return;
491
492     // global_idx → (token_head_idx, pack_idx) → (token_idx, head_idx, pack_offset)
493     // token_head_idx: 第几个 (token, head) 对, 行优先展平
494     const int token_head_idx = global_idx / threads_per_head;
495     // pack_idx: 该 (token, head) 对内第几个 pack, 0 ~ threads_per_head-1
496     const int pack_idx = global_idx % threads_per_head;
497
498     // token_head_idx 分解为 (token_idx, head_idx)
499     const int token_idx = token_head_idx / num_heads; // token 变化慢 (外维)
500     const int head_idx = token_head_idx % num_heads;  // head 变化快 (内维)
501
502     // pack_offset: 该 pack 覆盖的元素在 head_size 中的起始偏移 (0, 4, 8, ...)
503     const int pack_offset = pack_idx * PACK_SIZE;

```

```

504 // head_offset: 该 (token, head) 对在 output 展平一维数组中的起始偏移
505 const int head_offset = token_idx * num_heads * head_size + head_idx * head_size;
506
507 // 定位到当前 (token, head) 的输出段起始
508 const float *prefix_head = prefix_output + head_offset;
509 const float *suffix_head = suffix_output + head_offset;
510 float *output_head = output + head_offset;
511
512 // LSE 布局 [num_heads, num_tokens]: lse[head_idx][token_idx]
513 float p_lse = prefix_lse[head_idx * num_tokens + token_idx];
514 float s_lse = suffix_lse[head_idx * num_tokens + token_idx];
515
516 // inf → -inf: 空 attention 段 LSE 可能为 +inf, 替换后权重退化为 0
517 p_lse = isinf(p_lse) ? -INFINITY : p_lse;
518 s_lse = isinf(s_lse) ? -INFINITY : s_lse;
519
520 // Step 1: max 归一化 (与 safe softmax 同理, 防 exp 溢出)
521 const float max_lse = fmaxf(p_lse, s_lse);
522 p_lse -= max_lse;
523 s_lse -= max_lse;
524
525 // Step 2-3: 指数还原 → 混合比例 alpha, beta
526 const float p_se = expf(p_lse);
527 const float s_se = expf(s_lse);
528 const float out_se = p_se + s_se;
529 const float p_scale = p_se / out_se; // alpha = w_1 / (w_1 + w_2)
530 const float s_scale = s_se / out_se; // beta = w_2 / (w_1 + w_2)
531
532 // Step 4: 逐元素加权合并 0 = alpha * 0_1 + beta * 0_2
533 // 128-bit 向量化: uint4 load → per-element FMA → uint4 store
534 if (pack_offset < head_size) {
535     pack_t p_pack =
536         reinterpret_cast<const pack_t *>(prefix_head)[pack_idx];
537     pack_t s_pack =
538         reinterpret_cast<const pack_t *>(suffix_head)[pack_idx];
539     pack_t o_pack;
540
541 #pragma unroll
542     for (int i = 0; i < PACK_SIZE; ++i) {
543         const float p_v = reinterpret_cast<const float *>(&p_pack)[i];
544         const float s_v = reinterpret_cast<const float *>(&s_pack)[i];
545         const float o_v = p_v * p_scale + (s_v * s_scale); // FMA
546         reinterpret_cast<float *>(&o_pack)[i] = o_v;
547     }
548
549     reinterpret_cast<pack_t *>(output_head)[pack_idx] = o_pack;
550 }
551 }
552
553 // =====
554 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
555 // =====
556 // 面试要点:
557 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
558 //     online (1-pass, 增量更新)
559 //   - Online Softmax 是 FlashAttention 的数学基础
560 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
561 //     + variance)
562 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
563
564 // =====
565 // Phase 3a: Softmax — 三级递进 (面试核心考点)
566 // =====

```

```

567 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点?」
568 // 回答线索: naive(溢出) → safe → online
569
570 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
571 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
572 // 问题: x 值很大时 exp(x) 溢出为 inf
573 template <const int NUM_THREADS = 256>
574 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
575 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
576 // source: LeetCUDA/kernels/softmax/softmax.cu
577 __global__ void softmax_per_token(float *x, float *y, int N) {
578     const int tid = threadIdx.x;
579     const int idx = blockIdx.x * blockDim.x + tid;
580
581     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
582     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
583     if (idx < N)
584         y[idx] = exp_val / exp_sum;
585 }
586
587 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
588 // 面试重点: 为什么 Softmax 需要 Safe?
589 //   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
590 //   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
591 //   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
592 //   - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
593 // Grid: (S, 1, 1)
594 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
595 // source: LeetCUDA/kernels/softmax/softmax.cu
596 template <const int NUM_THREADS = 256>
597 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
598     const int tid = threadIdx.x;
599     const int idx = blockIdx.x * blockDim.x + tid;
600
601     // Pass 1: block reduce max — 找最大值
602     float val = (idx < N) ? x[idx] : -FLT_MAX;
603     float max_val = block_reduce_max<NUM_THREADS>(val);
604
605     // Pass 2: exp(x - max) → block reduce sum
606     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
607     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
608
609     if (idx < N)
610         y[idx] = exp_val / exp_sum;
611 }
612
613 // ---- Level 3: Online Safe Softmax (FlashAttention 的数学基础) ----
614 // 面试重点:
615 // 两种使用场景 (公式不同, 但结果等价):
616 // 1) 单元素增量更新 (online update, 处理新元素 x_i):
617 //     m_new = max(m_old, x_i)
618 //     d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
619 // 2) 二元合并 (binary merge, 合并两个部分累加器, warp/block reduce 使用):
620 //     m = max(m1, m2) safe softmax用的max值
621 //     d = d1*exp(m1-m) + d2*exp(m2-m) softmax用的分母值
622 //     当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
623 // 算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
624 // Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
625 //
626 // 不同于单元素增量更新公式 (m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new)),
627 // warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
628 // (m1,d1): max 和 Σexp(x_j - m1), 覆盖集合 S1
629 // (m2,d2): max 和 Σexp(x_k - m2), 覆盖集合 S2

```

```

630 //
631 // 合并公式 (对称, 不依赖"新/旧"概念):
632 //   m = max(m1, m2)
633 //   d = d1*exp(m1-m) + d2*exp(m2-m)   ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
634 //
635 // 该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
636 struct __align__(8) MD {
637     float m; // running max
638     float d; // running denominator (sum of exp(x - max))
639 };
640
641 template <const int kWarpSize = WARP_SIZE>
642 __device__ __forceinline__ MD warp_reduce_md(MD md1) {
643     #pragma unroll
644     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
645         MD md2;
646         md2.m = __shfl_xor_sync(0xffffffff, md1.m, mask, kWarpSize);
647         md2.d = __shfl_xor_sync(0xffffffff, md1.d, mask, kWarpSize);
648
649         MD b_m = (md1.m > md2.m) ? md1 : md2; // max
650         MD s_m = (md1.m > md2.m) ? md2 : md1;
651
652         // b_m.d 无需 rescale (其基准 max 已是全局 max),
653         // s_m.d 需 rescale: exp(s_m.m - b_m.m)
654         md1.d = b_m.d + s_m.d * __expf(s_m.m - b_m.m);
655         md1.m = b_m.m;
656     }
657     return md1;
658 }
659
660 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
661 // Grid: (S, 1, 1)
662 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
663 // source: LeetCUDA/kernels/softmax/softmax.cu
664 template <const int NUM_THREADS = 256>
665 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
666     int tid = threadIdx.x;
667     int idx = blockIdx.x * NUM_THREADS + threadIdx.x;
668     const int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
669     int warp = tid / WARP_SIZE;
670     int lane = tid % WARP_SIZE;
671
672     // 初始化: 每个线程持有一个 (max, denom) 对
673     MD md;
674     md.m = idx < N ? x[idx] : -FLT_MAX;
675     md.d = idx < N ? 1.0f : 0.0f;
676
677     // Block reduce: warp_reduce_md 在归约中自动更新 m 和 d
678     __shared__ MD shared[NUM_WARPS];
679     md = warp_reduce_md<WARP_SIZE>(md);
680
681     if (lane == 0)
682         shared[warp] = md;
683     __syncthreads();
684
685     // 第二级归约: warp0 收集各 warp 结果再做一次 warp_reduce_md (复用 block_reduce 模式)
686     // 只有 tid < 32 的线程参与; shared[0] 在第二次 __syncthreads 后才对整个 block 可见
687     if (tid < WARP_SIZE) {
688         md = tid < NUM_WARPS ? shared[tid] : MD{-FLT_MAX, 0.0f};
689         md = warp_reduce_md<NUM_WARPS>(md);
690         if (tid == 0) {
691             shared[0] = md;
692         }

```

```

693 }
694 __syncthreads();
695
696 // 用全局 max 和 denom 做最终 softmax
697 md = shared[0];
698 float d_inv = __fdividef(1.0f, md.d);
699 // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
700 if (idx < N) {
701     y[idx] = __expf(x[idx] - md.m) * d_inv;
702 }
703 }
704
705 // =====
706 // Phase 3b: RMS Normalization (1-pass reduce)
707 // =====
708 // 面试要点:
709 //   - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
710 //   - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
711 //   - Llama 系列使用 RMS Norm
712 //   - grid(N, K/K), block(K): 一行一个 block
713 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
714 // Block: (128, 1, 1), NUM_THREADS=K=128 (K>128 时调整模板参数)
715 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
716 template <const int NUM_THREADS = 128>
717 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
718     int tid = threadIdx.x;
719     int idx = blockIdx.x * blockDim.x + threadIdx.x;
720     const float epsilon = 1e-5f;
721
722     __shared__ float s_variance;
723     float value = (idx < N * K) ? x[idx] : 0.0f;
724     float variance = value * value;
725     variance = block_reduce_sum<NUM_THREADS>(variance);
726     if (tid == 0)
727         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
728     __syncthreads();
729     if (idx < N * K)
730         y[idx] = (value * s_variance) * g;
731 }
732
733 // RMS Norm + float4
734 // Grid: (N, 1, 1)
735 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
736 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
737 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
738 template <const int NUM_THREADS = 128 / 4>
739 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
740     int tid = threadIdx.x;
741     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
742     const float epsilon = 1e-5f;
743
744     __shared__ float s_variance;
745     float4 reg_x = FLOAT4(x[idx]);
746     float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
747                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w)
748                                     : 0.0f;
749     variance = block_reduce_sum<NUM_THREADS>(variance);
750     if (tid == 0)
751         s_variance = rsqrtf(variance / (float)K + epsilon);
752     __syncthreads();
753     float4 reg_y;
754     reg_y.x = reg_x.x * s_variance * g;
755     reg_y.y = reg_x.y * s_variance * g;

```

```

756 reg_y.z = reg_x.z * s_variance * g;
757 reg_y.w = reg_x.w * s_variance * g;
758 if (idx < N * K)
759     FLOAT4(y[idx]) = reg_y;
760 }
761
762 // =====
763 // Phase 3c: Layer Normalization (2-pass reduce)
764 // =====
765 // 面试要点:
766 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
767 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)2)/K)
768 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
769 // Grid: (N, 1, 1), 一行一个 block
770 // Block: (128, 1, 1), NUM_THREADS=K=128
771 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
772 template <const int NUM_THREADS = 128>
773 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
774     int tid = threadIdx.x;
775     int idx = blockIdx.x * blockDim.x + threadIdx.x;
776     const float epsilon = 1e-5f;
777
778     __shared__ float s_mean;
779     __shared__ float s_variance;
780     float value = (idx < N * K) ? x[idx] : 0.0f;
781
782     // Pass 1: compute mean
783     float sum = block_reduce_sum<NUM_THREADS>(value);
784     if (tid == 0)
785         s_mean = sum / (float)K;
786     __syncthreads(); // 必须等待 s_mean 对所有线程可见
787
788     // Pass 2: compute variance = (x - mean)2
789     float variance = (value - s_mean) * (value - s_mean);
790     variance = block_reduce_sum<NUM_THREADS>(variance);
791     if (tid == 0)
792         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
793     __syncthreads(); // 必须等待 s_variance 对所有线程可见
794
795     if (idx < N * K)
796         y[idx] = ((value - s_mean) * s_variance) * g + b;
797 }
798
799 // Layer Norm + float4
800 // Grid: (N, 1, 1)
801 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
802 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
803 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
804 template <const int NUM_THREADS = 128 / 4>
805 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N, int K) {
806     int tid = threadIdx.x;
807     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
808     const float epsilon = 1e-5f;
809
810     __shared__ float s_mean;
811     __shared__ float s_variance;
812     float4 reg_x = FLOAT4(x[idx]);
813     float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
814
815     float sum = block_reduce_sum<NUM_THREADS>(value);
816     if (tid == 0)
817         s_mean = sum / (float)K;
818     __syncthreads();

```

```

819
820 float4 reg_x_hat;
821 reg_x_hat.x = reg_x.x - s_mean;
822 reg_x_hat.y = reg_x.y - s_mean;
823 reg_x_hat.z = reg_x.z - s_mean;
824 reg_x_hat.w = reg_x.w - s_mean;
825 float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
826                 reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
827 variance = block_reduce_sum<NUM_THREADS>(variance);
828 if (tid == 0)
829     s_variance = rsqrtf(variance / (float)K + epsilon);
830 __syncthreads();
831
832 float4 reg_y;
833 reg_y.x = reg_x_hat.x * s_variance * g + b;
834 reg_y.y = reg_x_hat.y * s_variance * g + b;
835 reg_y.z = reg_x_hat.z * s_variance * g + b;
836 reg_y.w = reg_x_hat.w * s_variance * g + b;
837 if (idx < N * K)
838     FLOAT4(y[idx]) = reg_y;
839 }
840
841 // =====
842 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
843 // =====
844 // 面试要点:
845 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
846 //     [x1'] [cos(θ) -sin(θ)] [x1]
847 //     [x2'] = [sin(θ)  cos(θ)] [x2]
848 //   -  $\theta_i = 1 / (\text{theta}^{(2i/d)})$ , theta=10000.0f (Llama 风格)
849 //   - token_pos = idx / N: token 在序列中的位置
850 //   - token_idx = idx % N: token 内的维度对索引
851 //   - 输入 [seq_len, hidden_size], 输出同形状
852
853 // Grid: ((seq_len * N + 255) / 256, 1, 1)
854 // Block: (256, 1, 1)
855 // source: LeetCUDA/kernels/rope/rope.cu
856 __global__ void rope(float *x, float *out, int seq_len, int N) {
857     int idx = blockIdx.x * blockDim.x + threadIdx.x;
858     float x1 = x[idx * 2];
859     float x2 = x[idx * 2 + 1];
860     int token_pos = idx / N; // 序列位置
861     int token_idx = idx % N; // 维度对索引
862
863     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
864     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
865     float sin_v = sinf(token_pos * exp_v);
866     float cos_v = cosf(token_pos * exp_v);
867
868     // 2D 旋转
869     float out1 = x1 * cos_v - x2 * sin_v;
870     float out2 = x1 * sin_v + x2 * cos_v;
871     out[idx * 2] = out1;
872     out[idx * 2 + 1] = out2;
873 }
874
875 // =====
876 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
877 // =====
878 // 面试要点 (Bank Conflict 专题):
879 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
880 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
881 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次

```



```

882 // - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
883 //
884 // 转置的四步演进:
885 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
886 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
887 //   BCF:   smem 布局 [WARP_SIZE_S*4][WARP_SIZE_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict
888 //   merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
889 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
890
891 // 每个线程处理 1 个元素, block(16,16)
892 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
893 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
894 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
895 // Block: (16, 16, 1)
896 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
897 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
898     const int c = blockIdx.x * blockDim.x + threadIdx.x; // col in x
899     const int r = blockIdx.y * blockDim.y + threadIdx.y; // row in x
900     if (r < row && c < col) {
901         // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
902         y[c * row + r] = x[r * col + c];
903     }
904 }
905
906 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
907 // Block: (16, 16, 1)
908 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
909 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
910 __global__ void mat_transpose_padded(
911     float *x, float *y, const int row, const int col) {
912     const int tx = threadIdx.x, ty = threadIdx.y;
913
914     constexpr int TILE = 16;
915     constexpr int PAD = 1;
916     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
917     __shared__ float tile[TILE * 4][TILE + PAD];
918
919     // x 空间坐标; 每线程覆盖 4 行 (actual row = x_r * 4)
920     const int x_c = blockIdx.x * TILE + tx;
921     const int x_r = blockIdx.y * TILE + ty;
922
923     if (x_r * 4 < row && x_c < col) {
924         // Step 1: 4 rows × 1 col → smem (row-major);
925 #pragma unroll
926         for (int i = 0; i < 4; ++i) {
927             tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
928         }
929         __syncthreads();
930
931         // Step 2: Transposed read from smem
932         float4 reg_trans;
933         reg_trans.x = tile[tx * 4 + 0][ty];
934         reg_trans.y = tile[tx * 4 + 1][ty];
935         reg_trans.z = tile[tx * 4 + 2][ty];
936         reg_trans.w = tile[tx * 4 + 3][ty];
937
938         // y 空间坐标: y_r = x 的列, y_c = x 的行
939         const int y_r = blockIdx.x * TILE + ty; // = x col
940         const int y_c = (blockIdx.y * TILE + tx) * 4; // = x row
941
942         // Coalesced write: y[y_r][y_c .. y_c+3]
943         FLOAT4(y[y_r * row + y_c]) = reg_trans;
944     }

```



```

945 }
946
947 // =====
948 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
949 // =====
950 // 面试要点:
951 //   - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
952 //   - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
953 //   - 不同 K 值对应不同分块策略:
954 //       K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
955 //       K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
956 //       K=16 < 32: 一个 warp 用不满, 用 ROW_PER_WARP=2 让每个 warp 处理 2 行
957 //
958 // a: MxK, x: Kx1, y: Mx1, 计算:  $y = a * x$ ; N = 1
959
960 // ---- SGEMV K32: 基础 warp-per-row ----
961 // 设计: block(32, 4), blockDim.x=WARP_SIZE=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 NUM_WARPS 次)
962 // grid(M/4), 每个 warp 负责一行
963 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
964 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
965 // Block: (32, 4, 1), 每行 1 warp
966 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
967 // source: LeetCUDA/kernels/sgemv/sgemv.cu
968 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
969     int tx = threadIdx.x; // 0~31
970     int ty = threadIdx.y; // 0~3
971     int lane = tx % WARP_SIZE; // 0~31
972     int m = blockIdx.x * blockDim.y + ty; // 全局行号
973     if (m < M) {
974         float sum = 0.0f;
975         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
976         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
977         const int NUM_ITERS = (K + WARP_SIZE - 1) / WARP_SIZE;
978 #pragma unroll
979         for (int w = 0; w < NUM_ITERS; ++w) {
980             // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
981             int k = w * WARP_SIZE + lane;
982             sum += a[m * K + k] * x[k];
983         }
984         sum = warp_reduce_sum<WARP_SIZE>(sum);
985         // 每个 warp 处理一行, lane 0 写回结果
986         if (lane == 0)
987             y[m] = sum;
988     }
989 }
990
991 // ---- SGEMV K128: float4 向量化 ----
992 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
993 // Grid: ((M + 3) / 4, 1, 1)
994 // Block: (32, 4, 1)
995 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
996 // source: LeetCUDA/kernels/sgemv/sgemv.cu
997 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
998     int tx = threadIdx.x; // 0~31
999     int ty = threadIdx.y; // 0~3
1000     int lane = tx % WARP_SIZE; // 0~31
1001     int m = blockIdx.y * blockDim.x + ty;
1002
1003     if (m < M) {
1004         float sum = 0.0f;
1005         // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
1006         const int NUM_ITERS = (((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
1007 #pragma unroll

```

```

1008     for (int w = 0; w < NUM_ITERS; ++w) {
1009         int k = (w * WARP_SIZE + lane) * 4;
1010         float4 reg_x = FLOAT4(x[k]);
1011         float4 reg_a = FLOAT4(a[m * K + k]);
1012         sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
1013             reg_a.w * reg_x.w);
1014     }
1015     sum = warp_reduce_sum<WARP_SIZE>(sum);
1016     if (lane == 0)
1017         y[m] = sum;
1018 }
1019 }
1020
1021 // ---- SGEMV K16: K < WarpSize, ROW_PER_WARP=2 ----
1022 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
1023 // ROW_PER_WARP=2, K_WARP_SIZE=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
1024 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
1025 // Block: (32, 4, 1)
1026 // 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理 2 行
1027 // source: LeetCUDA/kernels/sgemv/sgemv.cu
1028 template <const int ROW_PER_WARP = 2>
1029 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
1030     constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) / ROW_PER_WARP; // 16
1031     int tx = threadIdx.x;
1032     int ty = threadIdx.y;
1033     int lane = tx % WARP_SIZE;
1034     int k = lane % K_WARP_SIZE; // 0~15
1035     int m = (blockDim.y * blockIdx.x + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
1036     if (m < M) {
1037         float sum = A[m * K + k] * x[k];
1038         // 按照K_WARP_SIZE=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
1039         sum = warp_reduce_sum<K_WARP_SIZE>(sum);
1040         // 注意: 判断条件是 k == 0, 不是 lane == 0!
1041         if (k == 0)
1042             y[m] = sum;
1043     }
1044 }
1045
1046 // =====
1047 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
1048 // =====
1049 // 面试要点 (GEMM 优化五层金字塔):
1050 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
1051 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
1052 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
1053 //   load/store 指令数 Level 4 — Tensor Core (MMA
1054 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
1055 //   TMA (WGMMMA m64n128k16): Hopper 异步执行
1056 //
1057 // 计算密度递进:
1058 //   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
1059 //   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
1060 //   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
1061
1062 // =====
1063 // Phase 7a: SGEMM (非 Tensor Core 路径)
1064 // =====
1065
1066 // ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
1067 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
1068 // C = A x B, C[M, N] = A[M, K] x B[K, N]
1069 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
1070 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)

```

```

1071 // Block: (32, 32, 1), 1024 线程
1072 // source: LeetCUDA/kernels/sgemm/sgemm.cu
1073 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
1074     constexpr int BM = 32; // vec 版: 32x4 = 128
1075     constexpr int BN = 32; // vec 版: 32x4 = 128
1076     constexpr int BK = 32;
1077     __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32x32x4=4KB smem, float = 4 bytes
1078
1079     int bx = blockIdx.x;
1080     int by = blockIdx.y;
1081     int tx = threadIdx.x;
1082     int tid = threadIdx.y * blockDim.x + tx;
1083
1084     // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
1085     int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载;
1086     int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载;
1087     int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载;
1088     int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载;
1089     int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row;
1090     int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col;
1091
1092     float sum = 0.f; // 遍历完整的K, slice K;
1093     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1094         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1095         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1096         s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
1097         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1098         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1099         s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
1100         __syncthreads(); // 确保整个 smem tile 加载完毕
1101
1102     #pragma unroll
1103     for (int k = 0; k < BK; ++k) {
1104         int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
1105         int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1106         sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
1107     }
1108     __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
1109 }
1110 int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
1111 int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1112 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1113 c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
1114 }
1115
1116 // ---- Level 1+: SGEMM Vec4 — Block Tile 128x128 + K Tile 32 + Thread Tile 4x4 ----
1117 // 在 Level 1 基础上引入两层优化:
1118 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
1119 // 2) Thread Tile 4x4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
1120 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
1121 // C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major
1122 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4x4=16 个 C 元素
1123 // 1024 x 16 = 16384 = 128 x 128 ✓
1124 //
1125 // 线程到 4x4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
1126 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
1127 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
1128 //
1129 // 加载映射 (每线程 4 个元素, float4):
1130 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8x4=32 列 ✓
1131 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32x4=128 列 ✓
1132 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
1133 //

```

```

1134 // △ Bank Conflict 提示（面试加分点）：
1135 //   s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1136 //   → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
1137 //   s_b[BK][BN+1] PAD 打散，这里保持最简布局便于讲解。
1138 //
1139 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
1140 // Block: (32, 32, 1), 1024 线程
1141 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
1142 // source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
1143 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1144     constexpr int BM = 128;
1145     constexpr int BN = 128;
1146     constexpr int BK = 32;
1147     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1148     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1149
1150     int bx = blockIdx.x;
1151     int by = blockIdx.y;
1152     int tx = threadIdx.x;
1153     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1154
1155     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1156     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1157     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1158     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1159     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1160     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
1161
1162     int load_gmem_a_m = by * BM + load_smem_a_m;
1163     int load_gmem_b_n = bx * BN + load_smem_b_n;
1164
1165     // 4x4 Thread Tile 基址 (独立于加载映射, 避免 /4 漏乘 bug)
1166     int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1167     int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1168
1169     float sum[4][4] = {0.f};
1170     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1171         int load_gmem_a_k = bk * BK + load_smem_a_k;
1172         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1173         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]);
1174         int load_gmem_b_k = bk * BK + load_smem_b_k;
1175         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1176         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);
1177         __syncthreads();
1178
1179 #pragma unroll
1180         for (int k = 0; k < BK; ++k) {
1181             // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4x4=16 次 FMA
1182             float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1183                                s_a[comp_smem_a_m_base + 1][k],
1184                                s_a[comp_smem_a_m_base + 2][k],
1185                                s_a[comp_smem_a_m_base + 3][k]};
1186             float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1187                                s_b[k][comp_smem_b_n_base + 1],
1188                                s_b[k][comp_smem_b_n_base + 2],
1189                                s_b[k][comp_smem_b_n_base + 3]};
1190
1191 #pragma unroll
1192             for (int i = 0; i < 4; ++i) {
1193 #pragma unroll
1194                 for (int j = 0; j < 4; ++j) {
1195                     sum[i][j] += a_vals[i] * b_vals[j];
1196                 }
1197             }
1198         }
1199     }

```

```

1197     }
1198     __syncthreads();
1199 }
1200
1201 // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1202 int store_gmem_c_m = by * BM + comp_smem_a_m_base;
1203 int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1204 #pragma unroll
1205 for (int i = 0; i < 4; ++i) {
1206     int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1207     float4 reg_c;
1208     reg_c.x = sum[i][0];
1209     reg_c.y = sum[i][1];
1210     reg_c.z = sum[i][2];
1211     reg_c.w = sum[i][3];
1212     FLOAT4(c[store_gmem_c_addr]) = reg_c;
1213 }
1214 }
1215
1216 // =====
1217 // Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMMA m64n128k16)
1218 // =====
1219 // 面试要点:
1220 //   - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1221 //   - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16x8] = [16x16]x[16x8]
1222 //   - ldmatrix: 从 shared memory 加载 16x16 的矩阵片段到寄存器 (4 条 32-bit
1223 //     寄存器)
1224 //   - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1225 //   - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1226 //
1227 // ★ TN 布局详解 (面试高频考点):
1228 //   TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1229 //   BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1230 //   N = Normal (列优先, BLAS 原生格式)
1231 //   T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1232 //   第一个字母 → A 的 op, 第二个字母 → B 的 op
1233 //   所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1234 //   BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
1235 //   视角下: TN = A row-major [MxK], B^T row-major [NxK] (等价于 B col-major [KxN]) 在 cuBLAS
1236 //   调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1237 //   T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
1238 //   N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1239 //
1240 //   记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1241 //   T = row-major (行优先, 对 BLAS 来说是 transposed)
1242 //   N = col-major (列优先, BLAS native = normal)
1243 //
1244 //   LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[MxN] = A[MxK] × B[KxN]
1245 //   - A: row-major [M, K], B: row-major [K, N]
1246 //   - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1247 //   - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1248 //
1249 //   TN 布局: C[MxN] = A[MxK] × B[KxN], B 以 B^T=[NxK] row-major 存储 (A 行优先, B 列优先)
1250 //   - A [MxK]: row-major → 全局索引 A[m*K + k], smem_s_a[BM][BK]
1251 //   - B^T [NxK]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (△ 内维连续的是 K)
1252 //   - 优势: B^T 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1253 //   MMA 指令: mma.sync.aligned.m16n8k16.row.col
1254 //   - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1255 //
1256 //   WGMMMA (Warp Group MMA, Hopper+):
1257 //   - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1258 //   - m64n128k16: 一次处理 64x128x16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1259 //   - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128

```

```

1260 // threads) 做计算
1261 // - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址
1262
1263 // =====
1264 // Phase 7b-1: MMA PTX 宏定义
1265 // =====
1266
1267 // ---- gmem → smem: cp.async ----
1268 // cp.async.commit_group / wait_group / wait_all 语义 (PTX ISA §9.7.9.25.3) :
1269 // - commit_group: 将此前所有未提交的 cp.async 归入一个新的 async-group (per-thread) 。
1270 // - wait_group N: 阻塞直到最多 N 个 async-group 尚未完成 (即 pending ≤ N) 。
1271 //   N=0 → 等所有 group 完成。与 wgmma.wait_group 语义一致。
1272 // - wait_all: 等价于 commit_group + wait_group 0。
1273 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1274 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1275 #define CP_ASYNC_WAIT_GROUP(n) \
1276     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1277
1278 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1279 #define CP_ASYNC_CG(dst, src, bytes) \
1280     asm volatile( \
1281         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1282         "l"(src), "n"(bytes))
1283
1284 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1285 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1286 // 每次加载 8x8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1287 // aligned: 要求 128-bit 对齐, trans: 转置加载
1288 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1289     asm volatile( \
1290         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
1291         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1292         : "r"(addr))
1293
1294 #define LDMATRIX_X2(R0, R1, addr) \
1295     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1296         : "=r"(R0), "=r"(R1) \
1297         : "r"(addr))
1298
1299 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1300 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1301 #define LDMATRIX_X2_T(R0, R1, addr) \
1302     asm volatile( \
1303         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1304         : "=r"(R0), "=r"(R1) \
1305         : "r"(addr))
1306
1307 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----
1308 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1309 // row.col: A 是 row-major, B 是 col-major
1310 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1311 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1312 // C 矩阵大小 16x8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1313 #define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1) \
1314     asm volatile( \
1315         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, " \
1316         "%4, %5}, {%6, %7}, {%8, %9};\n" \
1317         : "=r"(RD0), "=r"(RD1) \
1318         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1319         "r"(RC1))
1320
1321 // ---- MMA 辅助函数 ----
1322 // div_ceil: 整数除法向上取整

```



```

1323 #define HOST_DEVICE_INLINE __device__ __host__ inline
1324 HOST_DEVICE_INLINE int div_ceil(int a, int b) {
1325     return (a % b != 0) ? (a / b + 1) : (a / b);
1326 }
1327
1328 // =====
1329 // Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
1330 // =====
1331 // 面试重点 — Tile Hierarchy:
1332 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1333 //   MMA Tile (more warps): 2x4=8 个 MMA atom → [2x16, 8x4]=[32,32]
1334 //   VAL Tile (more values): 4x4=16 expand → [32x4, 32x4]=[128,128]
1335 //   实际: MMA_TILE_M=2, MMA_TILE_N=4, VAL_TILE_M=4, VAL_TILE_N=4
1336 //         → BM=16x2x4=128, BN=8x4x4=128, Warps=2x4=8, Threads=8x32=256
1337 //
1338 // ★ TN 布局 (T=A 行优先, N=B 列优先):
1339 //   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1340 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
1341 //   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
1342 //   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
1343 //   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1344 //
1345 // ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
1346 //   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % K_STAGE (零偏移)
1347 //   - 加载语义为“预取未来”: 迭代 k 加载 tile (k+K_STAGE-1) 供后续使用
1348 //   - cp.async 条件化: 仅当 k+K_STAGE-1 < NUM_K_TILES 时加载
1349 //   - WAIT_GROUP 自适应: 满载期用 K_STAGE-2, 尾部排空用 0
1350 //
1351 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1352 // Block: (256, 1, 1), 8 warps
1353 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1354 // =====
1355 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
1356           const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1357           const int VAL_TILE_M = 4, const int VAL_TILE_N = 4,
1358           const int K_STAGE = 3, const bool BLOCK_SWIZZLE = false>
1359 __global__ void __launch_bounds__(256)
1360     hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1361     // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1362     const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1363     const int by = blockIdx.y;
1364     constexpr int BM = MMA_M * MMA_TILE_M * VAL_TILE_M; // 16*2*4=128
1365     constexpr int BN = MMA_N * MMA_TILE_N * VAL_TILE_N; // 8*4*4=128
1366     constexpr int BK = MMA_K; // 16
1367
1368     // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B
1369     // TN 布局: s_a[BM][BK]=[128][16](A row-major), s_b[BN][BK]=[128][16](B^T
1370     // row-major, 即 B col-major [KxN] 在 smem 中按 B^T[NxK] 存储)
1371     // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
1372     // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
1373     extern __shared__ half smem[];
1374     half *s_a = smem;
1375     half *s_b = smem + K_STAGE * BM * BK; // A 和 B 连续存放
1376     constexpr int s_a_stage_offset = BM * BK; // 128*16
1377     constexpr int s_b_stage_offset = BN * BK; // 128*16 △ BN(128)xBK(16) = B^T row-major
1378     const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1379     const int warp_id = tid / WARP_SIZE; // 0~7
1380     const int lane_id = tid % WARP_SIZE; // 0~31
1381     const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1382     const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1383
1384     // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1385     // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)

```

```

1386 int load_smem_a_m = tid / 2; // 0~127
1387 int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1388 int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1389 int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1390 int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1391 int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1392 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1393     return;
1394
1395 // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16, 4*8]=[32, 32].
1396 // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1397 // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128, 128]的C block tile, 这就是Value Tile的作用。
1398 // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNK的概念。
1399 uint32_t RC[VAL_TILE_M][VAL_TILE_N][2] = {0}; // 初始化为 0
1400
1401 // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1402 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1403 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1404
1405 // 预加载前 (K_STAGE-1) 个 stage
1406 // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [K×N])
1407 #pragma unroll
1408 for (int k = 0; k < (K_STAGE - 1); ++k) {
1409     int load_gmem_a_k = k * BK + load_smem_a_k;
1410     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1411     uint32_t load_smem_a_ptr = (smem_a_base_ptr +
1412         (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
1413     );
1414     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1415
1416     int load_gmem_b_k = k * BK + load_smem_b_k;
1417     // B^T: [n][k] row-major (即 B[k][n] col-major) △
1418     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1419     uint32_t load_smem_b_ptr = (smem_b_base_ptr +
1420         (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
1421     );
1422     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1423
1424     CP_ASYNC_COMMIT_GROUP();
1425 }
1426
1427 CP_ASYNC_WAIT_GROUP(K_STAGE - 2); // 允许有 K_STAGE-2 个group未完成
1428 __syncthreads();
1429
1430 // 统一循环: k 从 0 开始, 每次迭代负责 tile k (加载 + 计算合并为单循环)
1431 const int NUM_K_TILES = div_ceil(K, BK);
1432 #pragma unroll
1433 for (int k = 0; k < NUM_K_TILES; ++k) {
1434     int smem_sel = k % K_STAGE; // 计算 tile k 的 stage
1435     int smem_sel_next = (k + K_STAGE - 1) % K_STAGE; // 预加载目标 stage
1436
1437     // 条件加载: 预加载 tile (k+K_STAGE-1) 供将来使用
1438     // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用 n*K+k (row-major, 内维连续的是 K)
1439     if (k + K_STAGE - 1 < NUM_K_TILES) {
1440         int load_gmem_a_k = (k + K_STAGE - 1) * BK + load_smem_a_k;
1441         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1442         int load_gmem_b_k = (k + K_STAGE - 1) * BK + load_smem_b_k;
1443         // B^T: row-major [n][k], 内维连续的是 K △
1444         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1445
1446         uint32_t load_smem_a_ptr =
1447             (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1448                 load_smem_a_m * BK + load_smem_a_k) *

```



```

1449         sizeof(half));
1450 CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1451
1452 uint32_t load_smem_b_ptr =
1453     (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1454         load_smem_b_n * BK + load_smem_b_k) *
1455         sizeof(half));
1456 CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1457 CP_ASYNC.COMMIT_GROUP();
1458 }
1459
1460 // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1461 // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major; B 用 x2 (非转置), smem 中
1462 // B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1463 uint32_t RA[VAL_TILE_M][4]; // [4][4], M方向4次repeat, A regs = 4 uint32_t per MMA
1464 uint32_t RB[VAL_TILE_N][2]; // [4][2], N方向4次repeat, B regs = 2 uint32_t per MMA
1465
1466 // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1467 #pragma unroll
1468 for (int i = 0; i < VAL_TILE_M; ++i) {
1469     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1470     int warp_smem_a_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1471     // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
1472     int lane_smem_a_m = warp_smem_a_m + lane_id % 16; // t{0...15}=0~15, t{16...31}=0~15
1473     int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8, t{0...15}=0, t{16...31}=8
1474     uint32_t lane_smem_a_ptr =
1475         (smem_a_base_ptr +
1476         (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1477         sizeof(half));
1478     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1479 }
1480
1481 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1482 // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1483 // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1484 #pragma unroll
1485 for (int j = 0; j < VAL_TILE_N; ++j) {
1486     // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1487     int warp_smem_b_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1488     // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
1489     int lane_smem_b_n = warp_smem_b_n + lane_id % 8; // t{0...7}=0~7, t{8...15}=0~7
1490     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // t{0...7}=0, t{8...15}=8
1491     uint32_t lane_smem_b_ptr =
1492         (smem_b_base_ptr +
1493         (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1494         sizeof(half));
1495     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1496 }
1497
1498 // MMA compute: 每个Warp(MMA) 在M方向重复VAL_TILE_M(4)次, 在N方向重复VAL_TILE_N(4)次
1499 #pragma unroll
1500 for (int i = 0; i < VAL_TILE_M; ++i) {
1501     #pragma unroll
1502     for (int j = 0; j < VAL_TILE_N; ++j) {
1503         HMMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1504             RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1505             RB[j][0], RB[j][1], // B fragment
1506             RC[i][j][0], RC[i][j][1]);
1507     }
1508 }
1509
1510 // 自适应等待: 流水线满载期用 K_STAGE-2, 尾部排空用 0
1511 if (k + K_STAGE - 1 < NUM_K_TILES) {

```

```

1512 CP_ASYNC_WAIT_GROUP(K_STAGE - 2);
1513 } else { // 对于尾部的 k, 等待所有剩余的 cp.async 完成
1514     CP_ASYNC_WAIT_GROUP(0);
1515 }
1516 __syncthreads();
1517 }
1518
1519 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1520 {
1521     for (int i = 0; i < VAL_TILE_M; ++i) {
1522         uint32_t RC0[VAL_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1523         uint32_t RC1[VAL_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1524         // =====
1525         // MMA m16n8k16 C fragment — a single 16x8 matrix (registers per warp):
1526         // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
1527         // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
1528         //
1529         //      cols: c0-1    c2-3    c4-5    c6-7
1530         //  r0:  T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  }
1531         //  r1:  T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  }
1532         //  r2:  T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} } RC[0]
1533         //  ...
1534         //  r7:  T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} }
1535         //  r8:  T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  }
1536         //  r9:  T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  }
1537         // r10:  T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} } RC[1]
1538         //  ...
1539         // r15:  T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} }
1540         //
1541         // Within a 4-lane group (e.g. T0-T3 for row 0):
1542         // lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
1543         // lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
1544         // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0, 4 次 shuffle 凑齐
1545         // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}, 再由 lane%4==0 128-bit store.
1546         // =====
1547 #pragma unroll
1548         for (int j = 0; j < VAL_TILE_N; ++j) {
1549             RC0[j][0] = RC[i][j][0];
1550             RC1[j][0] = RC[i][j][1];
1551             RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1552             RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1553             RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1554             RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1555             RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1556             RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1557         }
1558         // 每 4 个 lane 中只有 lane 0 做 128-bit store
1559         // lane_id / 4 → 行号映射 (每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行):
1560         //
1561         // | lane_id | lane_id/4 | RC[0] 的行 | RC[1] 的行 |
1562         // |-----|-----|-----|-----|
1563         // | 0~3   | 0       | row 0     | row 8     |
1564         // | 4~7   | 1       | row 1     | row 9     |
1565         // | 8~11  | 2       | row 2     | row 10    |
1566         // | 12~15 | 3       | row 3     | row 11    |
1567         // | 16~19 | 4       | row 4     | row 12    |
1568         // | 20~23 | 5       | row 5     | row 13    |
1569         // | 24~27 | 6       | row 6     | row 14    |
1570         // | 28~31 | 7       | row 7     | row 15    |
1571         //
1572         if (lane_id % 4 == 0) {
1573             // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1574             int store_warp_smem_c_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;

```

```

1575     int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1576 #pragma unroll
1577     for (int j = 0; j < VAL_TILE_N; ++j) {
1578         // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1579         int store_warp_smem_c_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1580         int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1581         int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1582         int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1583         // 128-bit store: 一次写入 8 个 half
1584         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1585             *reinterpret_cast<float4*>(&RC0[j][0]);
1586         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1587             *reinterpret_cast<float4*>(&RC1[j][0]);
1588     }
1589 }
1590 }
1591 }
1592 }
1593
1594 // =====
1595 // Phase 7b-3: HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局
1596 // + XOR swizzle 消除 smem bank conflict (统一循环版)
1597 // =====
1598 // 面试要点 (Swizzle 原理 — smem Bank Conflict 消除) :
1599 // - 问题: ldmatrix 以 8x8 片段 (m8n8) 从 smem 加载 16x16 的矩阵。同一 warp 中
1600 //   不同 lane 访问的地址在 bank 上产生冲突 → 串行化 (most common: 2/4-way) 。
1601 // - XOR swizzle 方案: 对列地址做 `((j>>3) ^ (i>>2)) % 2 << 3` 的 XOR 置换,
1602 //   将连续 4 行的 bank 映射打散。每 4 行为一组的 pattern 交替:
1603 //   rows 0~3: 列 0→地址偏移 0, 列 8→地址偏移 8
1604 //   rows 4~7: 列 0→地址偏移 8, 列 8→地址偏移 0 (XOR 翻转)
1605 //   rows 8~11: repeat rows 0~3 pattern
1606 //   rows 12~15: repeat rows 4~7 pattern
1607 // - 效果: 原来 n-way 的 bank conflict 降低到 1-way (fully conflict-free) 。
1608 // - 优势: 无需 smem PAD (不浪费空间), 原理上完全消除特定 pattern 的 bank conflict。
1609 // - 局限性: 要求 kColStride ≤ 16 (即 BK ≤ 16), kStep ∈ {4,8}。
1610 //   对 MMA m16n8k16 的 BK=16 而言刚好满足。
1611 //
1612 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1613 // 参考: https://zhuanlan.zhihu.com/p/4746910252
1614
1615 // ---- Swizzle 辅助函数 ----
1616
1617 // swizzle_permuted_j: 对 smem 列索引 j 做 XOR 置换, 消除 bank conflict。
1618 // i: row index; j: col index.
1619 // e.g kColStride = 16, kStep = 8 -> load 8 half as 128 bits memory issue.
1620 // 公式: ((j / kStep) ^ (i / 4)) % (kColStride / kStep) * kStep
1621 // 用位运算展开 (kStep=8) : (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3
1622 // 限制: kColStride ≤ 16 (BK ≤ 16), kStep ∈ {4, 8}, kColStride % kStep == 0
1623 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1624 template <const int kColStride = 16, const int kStep = 8>
1625 static __device__ __forceinline__ int swizzle_permuted_j(int i, int j) {
1626     // for col_stride > 16, we have to permute it using col major ZigZag order.
1627     // e.g, A smem logical layout [Br,d]=[Br,64] -> store layout [4][Br][16].
1628     static_assert(kColStride <= 16, "kColStride must <= 16");
1629     // swizzle: ((int(j / kStep) ^ int(i / 4)) % int(kColStride / kStep)) * kStep;
1630     static_assert(kStep == 4 || kStep == 8, "kStep must be 8 or 4.");
1631     static_assert(kColStride % kStep == 0,
1632         "kColStride must be multiple of kStep.");
1633     if constexpr (kStep == 8) {
1634         return (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3;
1635     } else {
1636         static_assert(kStep == 4);
1637         return (((j >> 2) ^ (i >> 2)) % (kColStride >> 2)) << 2;

```

```

1638     }
1639 }
1640
1641 // swizzle_permuted_A_j: A 矩阵专用封装 (kMmaAtomK=16, kStep=8) 。
1642 // 16 行 (一个 MMA atom 的 M 维) 内的 swizzle pattern:
1643 // -----
1644 // -col 0~16, step 8--
1645 // -----
1646 // | row 0 | (0, 8) |
1647 // | row 1 | (0, 8) |
1648 // | row 2 | (0, 8) |
1649 // | row 3 | (0, 8) |
1650 // -----
1651 // | row 4 | (8, 0) |
1652 // | row 5 | (8, 0) |
1653 // | row 6 | (8, 0) |
1654 // | row 7 | (8, 0) |
1655 // -----
1656 // | row 8 | (0, 8) |
1657 // | row 9 | (0, 8) |
1658 // | row 10 | (0, 8) |
1659 // | row 11 | (0, 8) |
1660 // -----
1661 // | row 12 | (8, 0) |
1662 // | row 13 | (8, 0) |
1663 // | row 14 | (8, 0) |
1664 // | row 15 | (8, 0) |
1665 // -----
1666 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1667 template <const int kMmaAtomK = 16>
1668 static __device__ __forceinline__ int swizzle_permuted_A_j(int i, int j) {
1669     return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
1670 }
1671
1672 // swizzle_permuted_B_j: B 矩阵专用封装 (与 A 相同的 pattern) 。
1673 // B^T smem 布局 [BN][BK]=[128][16] 在 BK 维做 swizzle,
1674 // pattern 与 A 完全相同 (kMmaAtomK=16, kStep=8) 。
1675 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1676 template <const int kMmaAtomK = 16>
1677 static __device__ __forceinline__ int swizzle_permuted_B_j(int i, int j) {
1678     return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
1679 }
1680
1681 // =====
1682 // Phase 7b-3: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
1683 // =====
1684 // 与 Phase 7b-2 (hgemm_mma_stages_tn) 的核心区别:
1685 // - 所有 smem 地址计算加 swizzle_permuted_A_j/swizzle_permuted_B_j, 消除 bank conflict
1686 // - VAL_TILE_M/N → VAL_TILE_M/N (与源 kernel 命名一致)
1687 // - 其余 tile hierarchy、pipeline、epilogue 与统一循环版完全一致
1688 //
1689 // Tile Hierarchy:
1690 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1691 //   MMA Tile (more warps): 2x4=8 个 MMA atom → [2x16, 8x4]=[32,32]
1692 //   WARP Tile (more values): 4x4=16 expand → [32x4, 32x4]=[128,128]
1693 //   实际: MMA_TILE_M=2, MMA_TILE_N=4, VAL_TILE_M=4, VAL_TILE_N=4
1694 //         → BM=16x2x4=128, BN=8x4x4=128, Warps=2x4=8, Threads=8x32=256
1695 //
1696 // TN 布局: C[MxN] = A[MxK] × B^T[NxK]
1697 // - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1698 // - B^T[N][K]: row-major → 全局索引 B[n*K+k]
1699 // - ldmatrix A: x4 (非转置), A row-major 原生匹配
1700 // - ldmatrix B: x2 (非转置), B^T row-major 逐行加载 = B 的列

```

```

1701 // - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1702 //
1703 // 统一循环设计 (同 hgemm_mma_stages_tn) :
1704 // - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % K_STAGE (零偏移)
1705 // - 加载语义为"预取未来": 迭代 k 加载 tile (k+K_STAGE-1) 供后续使用
1706 // - cp.async 条件化: 仅当 k+K_STAGE-1 < NUM_K_TILES 时加载
1707 // - WAIT_GROUP 自适应: 满载期用 K_STAGE-2, 尾部排空用 0
1708 //
1709 // Grid:  $((N+127)/128/S, (M+127)/128, S)$ ,  $S=(N+2047)/2048$ , 3D block swizzle
1710 // Block: (256, 1, 1), 8 warps
1711 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1712 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
1713         const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1714         const int VAL_TILE_M = 4, const int VAL_TILE_N = 4,
1715         const int K_STAGE = 3,
1716         const bool BLOCK_SWIZZLE = false>
1717 __global__ void __launch_bounds__(256)
1718     hgemm_mma_stages_tn_swizzle(half *A, half *B, half *C, int M, int N, int K) {
1719     // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1720     const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1721     const int by = blockIdx.y;
1722     constexpr int BM = MMA_M * MMA_TILE_M * VAL_TILE_M; // 16*2*4=128
1723     constexpr int BN = MMA_N * MMA_TILE_N * VAL_TILE_N; // 8*4*4=128
1724     constexpr int BK = MMA_K; // 16
1725
1726     // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B (与 hgemm_mma_stages_tn 相同)
1727     extern __shared__ half smem[];
1728     half *s_a = smem;
1729     half *s_b = smem + K_STAGE * BM * BK;
1730     constexpr int s_a_stage_offset = BM * BK; // 128*16
1731     constexpr int s_b_stage_offset = BN * BK; // 128*16 Δ B^T row-major
1732     const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1733     const int warp_id = tid / WARP_SIZE; // 0~7
1734     const int lane_id = tid % WARP_SIZE; // 0~31
1735     const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1736     const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1737
1738     // 线程到 global memory 的映射 (用于加载 A 和 B)
1739     int load_smem_a_m = tid / 2; // 0~127
1740     int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1741     int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1742     int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1743     int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号
1744     int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号
1745     if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1746         return;
1747
1748     // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32].
1749     // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1750     // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128,128]的C block tile, 这就是Value Tile的作用。
1751     // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNK的概念。
1752     uint32_t RC[VAL_TILE_M][VAL_TILE_N][2] = {0}; // 初始化为 0
1753
1754     // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1755     uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1756     uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1757
1758     // 预加载前 (K_STAGE-1) 个 stage
1759     // * swizzle 区别: cp.async 的目标 smem 地址加 swizzle_permuted_A_j/swizzle_permuted_B_j
1760 #pragma unroll
1761     for (int k = 0; k < (K_STAGE - 1); ++k) {
1762         int load_gmem_a_k = k * BK + load_smem_a_k;
1763         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;

```

```

1764     int load_gmem_b_k = k * BK + load_smem_b_k;
1765     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1766
1767     uint32_t load_smem_a_ptr =
1768         (smem_a_base_ptr +
1769          (k * s_a_stage_offset + load_smem_a_m * BK +
1770           swizzle_permuted_A_j<MMA_K>(load_smem_a_m, load_smem_a_k)) *
1771           sizeof(half));
1772     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1773
1774     uint32_t load_smem_b_ptr =
1775         (smem_b_base_ptr +
1776          (k * s_b_stage_offset + load_smem_b_n * BK +
1777           swizzle_permuted_B_j<MMA_K>(load_smem_b_n, load_smem_b_k)) *
1778           sizeof(half));
1779     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1780
1781     CP_ASYNC_COMMIT_GROUP();
1782 }
1783
1784 CP_ASYNC_WAIT_GROUP(K_STAGE - 2); // 允许有 K_STAGE-2 个group未完成
1785 __syncthreads();
1786
1787 // 统一循环: k 从 0 开始, 每次迭代负责 tile k (加载 + 计算合并为单循环)
1788 const int NUM_K_TILES = div_ceil(K, BK);
1789 #pragma unroll
1790 for (int k = 0; k < NUM_K_TILES; ++k) {
1791     int smem_sel = k % K_STAGE; // 计算 tile k 的 stage
1792     int smem_sel_next = (k + K_STAGE - 1) % K_STAGE; // 预加载目标 stage
1793
1794     // 条件加载: 预加载 tile (k+K_STAGE-1) 供将来使用
1795     // swizzle 区别: smem 地址加 swizzle_permuted_*_j
1796     if (k + K_STAGE - 1 < NUM_K_TILES) {
1797         int load_gmem_a_k = (k + K_STAGE - 1) * BK + load_smem_a_k;
1798         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1799         int load_gmem_b_k = (k + K_STAGE - 1) * BK + load_smem_b_k;
1800         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1801
1802         uint32_t load_smem_a_ptr =
1803             (smem_a_base_ptr +
1804              (smem_sel_next * s_a_stage_offset + load_smem_a_m * BK +
1805               swizzle_permuted_A_j<MMA_K>(load_smem_a_m, load_smem_a_k)) *
1806               sizeof(half));
1807         CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1808
1809         uint32_t load_smem_b_ptr =
1810             (smem_b_base_ptr +
1811              (smem_sel_next * s_b_stage_offset + load_smem_b_n * BK +
1812               swizzle_permuted_B_j<MMA_K>(load_smem_b_n, load_smem_b_k)) *
1813               sizeof(half));
1814         CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1815         CP_ASYNC_COMMIT_GROUP();
1816     }
1817
1818     // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1819     // * swizzle 区别: ldmatrix 的源 smem 地址也加 swizzle_permuted_*_j
1820     uint32_t RA[VAL_TILE_M][4];
1821     uint32_t RB[VAL_TILE_N][2];
1822
1823     // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1824 #pragma unroll
1825     for (int i = 0; i < VAL_TILE_M; ++i) {
1826         int warp_smem_a_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;

```

```

1827     int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1828     int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8
1829     uint32_t lane_smem_a_ptr =
1830         (smem_a_base_ptr +
1831          (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
1832           swizzle_permuted_A_j < MMA_K > (lane_smem_a_m, lane_smem_a_k)) *
1833           sizeof(half));
1834     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1835 }
1836
1837 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1838 #pragma unroll
1839 for (int j = 0; j < VAL_TILE_N; ++j) {
1840     int warp_smem_b_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1841     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1842     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // 0, 8
1843     uint32_t lane_smem_b_ptr =
1844         (smem_b_base_ptr +
1845          (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
1846           swizzle_permuted_B_j < MMA_K > (lane_smem_b_n, lane_smem_b_k)) *
1847           sizeof(half));
1848     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1849 }
1850
1851 // MMA compute: 每个Warp(MMA) 在M方向重复VAL_TILE_M(4)次, 在N方向重复VAL_TILE_N(4)次
1852 #pragma unroll
1853 for (int i = 0; i < VAL_TILE_M; ++i) {
1854     #pragma unroll
1855     for (int j = 0; j < VAL_TILE_N; ++j) {
1856         MMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1857                 RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1858                 RB[j][0], RB[j][1], // B fragment
1859                 RC[i][j][0], RC[i][j][1]);
1860     }
1861 }
1862
1863 // 自适应等待: 流水线满载期用 K_STAGE-2, 尾部排空用 0
1864 if (k + K_STAGE - 1 < NUM_K_TILES) {
1865     CP_ASYNC_WAIT_GROUP(K_STAGE - 2);
1866 } else {
1867     CP_ASYNC_WAIT_GROUP(0);
1868 }
1869 __syncthreads();
1870 }
1871
1872 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1873 // 与 hgemv_mma_stages_tn 完全一致的 epilogue 模式
1874 {
1875     for (int i = 0; i < VAL_TILE_M; ++i) {
1876         uint32_t RC0[VAL_TILE_N][4];
1877         uint32_t RC1[VAL_TILE_N][4];
1878     #pragma unroll
1879     for (int j = 0; j < VAL_TILE_N; ++j) {
1880         RC0[j][0] = RC[i][j][0];
1881         RC1[j][0] = RC[i][j][1];
1882         RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1883         RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1884         RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1885         RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1886         RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1887         RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1888     }
1889     if (lane_id % 4 == 0) {

```



```

1890     int store_warp_smem_c_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1891     int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1892 #pragma unroll
1893     for (int j = 0; j < VAL_TILE_N; ++j) {
1894         int store_warp_smem_c_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1895         int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1896         int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1897         int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1898         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1899             *reinterpret_cast<float4*>(&RC0[j][0]);
1900         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1901             *reinterpret_cast<float4*>(&RC1[j][0]);
1902     }
1903 }
1904 }
1905 }
1906 }
1907
1908 // =====
1909 // Phase 7c: HGEMM CuTe — CUTLASS CuTe DSL 实现 (SM80+, Tensor Core 全自动调度)
1910 // =====
1911 // 面试要点 (CuTe vs 手写 MMA PTX) :
1912 // - CuTe (CUTLASS Templates) 是 NVIDIA CUTLASS 3.x 的核心 DSL, 提供编译期
1913 //   Tensor 抽象, 自动推导 MMA、Copy、Swizzle 的线程-数据映射
1914 // - 与手写 MMA PTX (Phase 7b) 的对比:
1915 //   - 手写 MMA: 手动计算 smem 地址、手动 ldmatrix/mma PTX、手动 swizzle、手动 epilogue shuffle
1916 //   - CuTe: 声明 TiledMMA / TiledCopy / SmemLayout, 编译器自动推导线程-数据映射,
1917 //     cute::copy / cute::gemm 自动展开为高效指令序列
1918 // - 核心抽象 ("CuTe 五要素") :
1919 //   1. Tensor: 全局/共享/寄存器数据的逻辑视图 = (ptr, shape, stride)
1920 //   2. TiledMMA: 描述 MMA 指令 + warp/warpgroup 排布 + value tile 的 compile-time type
1921 //   3. TiledCopy: 描述数据搬运 (G→S, S→R, R→S, S→G) 的线程-元素映射
1922 //   4. Layout + Swizzle: 描述 smem 的数据排布和 bank conflict 消除
1923 //   5. Pipeline: cp.async + multistage, 由 cute::copy 自动管理
1924 //
1925 // CuTe kernel 的参数推导链 (launch wrapper 负责实例化, kernel 只接收实例化后的类型) :
1926 //   MMA Atom (SM80_16x8x16_F16F16F16F16_TN)
1927 //   → TiledMMA (EURepeat=MxNxK, ValTile=MxNxK)
1928 //   → TiledCopy (G2S/S2R/R2S/S2G, 每种有 ThrLayout + ValLayout)
1929 //   → SmemLayout (Atom + Swizzle + tile_to_shape + Stage)
1930 //
1931 // 调参口诀 (面试速记) :
1932 //   BM/BN/BK 越大 → smem 越大 → occupancy 越低, 但 K 循环更少 → 指令开销更低
1933 //   Swizzle<B,M,S>: B=bank 维度 bit, M=M维 bit, S=stride bit → Swizzle<3,3,3> = 512 values = 1024B
1934 //   kStage: 2=最低延迟, 3/4=更好隐藏延迟 → 权衡 smem 占用
1935 //   EURepeat: 增加 warp 数 → 更多并行但每个 warp 做更少 MMA → 寄存器压力降低
1936 //
1937 // ★ 与手写 MMA Swizzle (Phase 7b-3) 的本质区别:
1938 //   - 手写 swizzle: 在 smem 地址计算时手动 XOR 列索引
1939 //   - CuTe Swizzle: SmemLayout 声明式指定 swizzle pattern, cute::copy 自动应用
1940 //   - CuTe 优势: 类型安全, 编译器可做更激进的优化, 布局变更只需改类型定义
1941 //
1942 // ★ 本实现的 Tile 配置:
1943 //   BM=128, BN=256, BK=32, kStage=2
1944 //   MMA: SM80_16x8x16_F16F16F16F16_TN, EURepeat=2x2x1, ValTile=32x32x16
1945 //   Smem Swizzle<3,3,3>: 512-value XOR permutation, 消除 ldmatrix bank conflict
1946 //   Threads: size(MMA{}) = 128 (4 warps × 2x2 EU repeat)
1947 //   Smem: ~49KB (Stage=2, A[128×32] + B[256×32] × half)
1948 //
1949 // 参考: LeetCode/kernels/hgemm/cutlass/hgemm_mma_stage_tn_cute.cu
1950 // 参考: https://zhuanlan.zhihu.com/p/671419093 (CuTe Swizzle 详解)
1951 // =====

```

```

1953 #if defined(NOTES_V2_ENABLE_CUTE)
1954
1955 #include <cute/tensor.hpp>
1956
1957 // =====
1958 // Phase 7d-1: CuTe HGEMM Kernel — TN 布局 + Smem Swizzle + Multistage Pipeline
1959 // =====
1960 // TN 布局:  $C[M \times N] = A[M \times K] \times B^T[N \times K]$ 
1961 //   -  $A[M][K]$  row-major,  $B^T[N][K]$  row-major (等价 B col-major  $[K \times N]$ )
1962 //   - CuTe 中通过 make_tensor(make_gmem_ptr(ptr), shape, stride) 描述
1963 //
1964 // Pipeline 流程 (每个 stage 对应一次完整的 G→S→R→MMA→R→S→G 链条):
1965 //   1. PREFETCH: 预加载 kStage-1 个 tile (G→S via cp.async)
1966 //   2. 主循环 over K tiles:
1967 //       a. S→R: cute::copy(s2r_tiled_copy, sA, rA) ← 自动展开为 ldmatrix
1968 //       b. MMA: cute::gemm(tiled_mma, rD, rA, rB, rD) ← 自动展开为 mma.sync
1969 //       c. G→S (next tile): cute::copy(g2s_tiled_copy, gA, sA) ← cp.async
1970 //       d. Pipeline wait + smem stage advance
1971 //   3. Epilogue: R→S (via UniversalCopy) → S→G (128-bit store)
1972 //
1973 // 面试对比 — 手写 MMA vs CuTe 代码量:
1974 //   手写: ldmatrix PTX 地址计算 + swizzle XOR + mma PTX + epilogue shuffle (~200行)
1975 //   CuTe: partition + copy + gemm (~80行), 其余由 launch wrapper 的类型推导完成
1976 template <typename T, int BM, int BN, int BK, int kStage, typename TiledMMA,
1977           typename G2SCopyA, typename G2SCopyB, typename SmemLayoutA,
1978           typename SmemLayoutB, typename SmemLayoutC, typename S2RCopyAtomA,
1979           typename S2RCopyAtomB, typename R2SCopyAtomC, typename S2GCopyAtomC,
1980           typename S2GCopyC>
1981 __global__ void hgemm_mma_stages_tn_cute(T *Aptr, T *Bptr, T *Dptr, int m,
1982                                           int n, int k) {
1983     using namespace cute;
1984
1985     // 动态 shared memory: A tile + B tile (C epilogue 复用 A tile 的空间)
1986     extern __shared__ T shm_data[];
1987     T *Ashm = shm_data;
1988     T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
1989
1990     int idx = threadIdx.x;
1991     int ix = blockIdx.x; // N 方向 block 索引 (本实现默认不开启 BlockSwizzle)
1992     int iy = blockIdx.y; // M 方向 block 索引
1993     if (iy * BM >= m || ix * BN >= n) return;
1994
1995     // CuTe Tensor 抽象: (ptr, shape, stride) 三元组描述数据布局
1996     // make_stride(leading_dim, Int<1>{}) → row-major: 同行相邻元素步长=1, 跨行步长=leading_dim
1997     Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
1998                             make_stride(k, Int<1>{}));
1999     Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k),
2000                             make_stride(k, Int<1>{}));
2001     Tensor D = make_tensor(make_gmem_ptr(Dptr), make_shape(m, n),
2002                             make_stride(n, Int<1>{}));
2003
2004     // local_tile: 从全局 Tensor 切出当前 CTA 负责的 tile
2005     // make_coord(iy, _): M 维固定为 iy, K 维自动按 BK 分 tile (_ = 通配符)
2006     Tensor gA = local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), make_coord(iy, _));
2007     Tensor gB = local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), make_coord(ix, _));
2008     Tensor gD = local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), make_coord(iy, ix));
2009
2010     // Shared memory Tensor: 按 SmemLayout 解读 smem 区域
2011     auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2012     auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2013
2014     // TiledMMA partition: 将 MMA 的 A/B/C tile 映射到各线程的寄存器 fragment
2015     TiledMMA tiled_mma;

```

```

2016 auto thr_mma = tiled_mma.get_slice(threadIdx.x);
2017 auto tCrA = thr_mma.partition_fragment_A(gA(_, _, 0)); // (MMA, MMA_M, MMA_K)
2018 auto tCrB = thr_mma.partition_fragment_B(gB(_, _, 0)); // (MMA, MMA_N, MMA_K)
2019 auto tCrD = thr_mma.partition_fragment_C(gD); // (MMA, MMA_M, MMA_N)
2020 clear(tCrD); // 累加器清零
2021
2022 // G2S TiledCopy: 描述 global → shared memory 的数据搬运 (使用 cp.async 128-bit)
2023 G2SCopyA g2s_tiled_copy_a;
2024 auto g2s_thr_copy_a = g2s_tiled_copy_a.get_slice(idx);
2025 auto tAgA_copy = g2s_thr_copy_a.partition_S(gA); // (CPY, CPY_M, CPY_K, k_tile)
2026 auto tAsA_copy = g2s_thr_copy_a.partition_D(sA); // (CPY, CPY_M, CPY_K, kStage)
2027
2028 G2SCopyB g2s_tiled_copy_b;
2029 auto g2s_thr_copy_b = g2s_tiled_copy_b.get_slice(idx);
2030 auto tBgB_copy = g2s_thr_copy_b.partition_S(gB);
2031 auto tBsB_copy = g2s_thr_copy_b.partition_D(sB);
2032
2033 // S2R TiledCopy: 描述 shared → register 的数据搬运 (使用 ldmatrix)
2034 // make_tiled_copy_A/B: 根据 TiledMMA 自动推导 S2R copy 的线程-数据映射
2035 auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2036 auto s2r_thr_copy_a = s2r_tiled_copy_a.get_slice(idx);
2037 auto tAsA = s2r_thr_copy_a.partition_S(sA); // (CPY, CPY_M, CPY_K, kStage)
2038 auto tCrA_view = s2r_thr_copy_a.retile_D(tCrA); // (CPY, CPY_M, CPY_K) — 与 rA 寄存器布局对齐
2039
2040 auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2041 auto s2r_thr_copy_b = s2r_tiled_copy_b.get_slice(idx);
2042 auto tBsB = s2r_thr_copy_b.partition_S(sB);
2043 auto tCrB_view = s2r_thr_copy_b.retile_D(tCrB);
2044
2045 // PREFETCH: 预加载前 (kStage - 1) 个 K tile, 填满流水线
2046 int itile_to_read = 0, ismem_read = 0, ismem_write = 0;
2047 #pragma unroll
2048 for (int istage = 0; istage < kStage - 1; ++istage) {
2049     cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, istage),
2050               tAsA_copy(_, _, _, istage));
2051     cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, istage),
2052               tBsB_copy(_, _, _, istage));
2053     cp_async_fence();
2054     ++itile_to_read;
2055     ++ismem_write;
2056 }
2057 cp_async_wait<kStage - 2>();
2058 __syncthreads();
2059
2060 // 主循环: over K tiles, 每个 K tile 内再分 MMA_K 步迭代
2061 int ntile = k / BK; // K 方向总 tile 数
2062 int ik = 0;
2063 // 首轮 S→R 加载 (在进入循环前先加载第一个 ik slice)
2064 cute::copy(s2r_tiled_copy_a, tAsA(_, _, ik, ismem_read), tCrA_view(_, _, ik));
2065 cute::copy(s2r_tiled_copy_b, tBsB(_, _, ik, ismem_read), tCrB_view(_, _, ik));
2066
2067 #pragma unroll 1
2068 for (int itile = 0; itile < ntile; ++itile) {
2069     int nk = size<2>(tCrA); // 每个 K tile 内的 MMA_K 步数
2070
2071     #pragma unroll
2072     for (int ik = 0; ik < nk; ++ik) {
2073         int ik_next = (ik + 1) % nk;
2074
2075         if (ik == nk - 1) {
2076             // 等待下一批 smem 数据就绪 (pipeline 同步)
2077             cp_async_wait<kStage - 2>();
2078             __syncthreads();

```

```

2079     ismem_read = (ismem_read + 1) % kStage;
2080 }
2081
2082 // S→R: 加载下一组 A/B fragment 到寄存器 (用 ik_next 预取)
2083 cute::copy(s2r_tiled_copy_a, tAsA(_, _, ik_next, ismem_read),
2084            tCrA_view(_, _, ik_next));
2085 cute::copy(s2r_tiled_copy_b, tBsB(_, _, ik_next, ismem_read),
2086            tCrB_view(_, _, ik_next));
2087
2088 if (ik == 0) {
2089     // G→S: 提交下一个 K tile 的异步拷贝 (流水线加载)
2090     if (itile_to_read < ntile) {
2091         cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, itile_to_read),
2092                  tAsA_copy(_, _, _, ismem_write));
2093         cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, itile_to_read),
2094                  tBsB_copy(_, _, _, ismem_write));
2095         ++itile_to_read;
2096         ismem_write = (ismem_write + 1) % kStage;
2097     }
2098     cp_async_fence();
2099 }
2100
2101 // MMA: 发射 Tensor Core 指令 (自动展开为 mma.sync.aligned.m16n8k16)
2102 cute::gemm(tiled_mma, tCrD, tCrA(_, _, ik), tCrB(_, _, ik), tCrD);
2103 }
2104 }
2105
2106 // Epilogue: D寄存器 → shared memory → global memory
2107 // 使用 A tile 的最后一个 stage 作为 C 的 scratchpad (节省 smem)
2108 auto sC = make_tensor(sA(_, _, ismem_read).data(), SmemLayoutC{});
2109
2110 // R2S TiledCopy: 寄存器 → shared memory (使用 UniversalCopy 做类型转换)
2111 auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2112 auto r2s_thr_copy_c = r2s_tiled_copy_c.get_slice(idx);
2113 auto tCrC_r2s = r2s_thr_copy_c.retile_S(tCrD); // (CPY, CPY_M, CPY_N)
2114 auto tCsC_r2s = r2s_thr_copy_c.partition_D(sC); // (CPY, _1, _1, pipe)
2115
2116 // S2G TiledCopy: shared memory → global memory (128-bit store)
2117 S2GCopyC s2g_tiled_copy_c;
2118 auto s2g_thr_copy_c = s2g_tiled_copy_c.get_thread_slice(idx);
2119 auto tCsC_s2g = s2g_thr_copy_c.partition_S(sC); // (CPY, _1, _1, pipe)
2120 auto tCgC_s2g = s2g_thr_copy_c.partition_D(gD); // (CPY, CPY_M, CPY_N)
2121
2122 // group_modes: 将多维 Tensor 的某些 mode 合并, 简化遍历
2123 auto tCgC_s2gx = group_modes<1, 3>(tCgC_s2g);
2124 auto tCrC_r2sx = group_modes<1, 3>(tCrC_r2s);
2125
2126 int step = size<3>(tCsC_r2s); // pipeline depth of C smem
2127 #pragma unroll
2128 for (int i = 0; i < size<1>(tCrC_r2sx); i += step) {
2129     // R→S: 寄存器写回 shared memory
2130     #pragma unroll
2131     for (int j = 0; j < step; ++j) {
2132         auto t = make_tensor_like<T>(tCrC_r2sx(_, i + j));
2133         cute::copy(tCrC_r2sx(_, i + j), t);
2134         cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2135     }
2136     __syncthreads();
2137
2138     // S→G: shared memory 写回 global memory
2139     #pragma unroll
2140     for (int j = 0; j < step; ++j) {
2141         cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i + j));

```

```

2142     }
2143     __syncthreads();
2144 }
2145 }
2146
2147 // =====
2148 // Phase 7c-2: CuTe HGEMM Launch Wrapper — 类型实例化 + Grid/Block 配置
2149 // =====
2150 // CuTe 的核心设计理念: "类型即配置" (Type-level configuration)
2151 // 所有的 MMA atom、TiledCopy、SmemLayout、Swizzle 都是 **编译期类型**,
2152 // kernel 接收这些类型的实例 (通常为 struct), 在编译期完成全部映射推导。
2153 //
2154 // 面试常问: 「CuTe 为什么比手写 PTX 简洁?」
2155 // 答: 手写需要:
2156 //   1) 手动计算每线程的 smem 地址偏移
2157 //   2) 手动计算 ldmatrix 的 lane 映射
2158 //   3) 手动插入 swizzle XOR 到每个地址计算点
2159 //   4) 手动做 epilogue 的 warp shuffle 编排
2160 // CuTe 的 partition + copy + gemm 通过 TiledCopy/TiledMMA 的类型信息
2161 // 在编译期自动完成上述全部推导, 生成与手写等价的 PTX 指令序列。
2162 //
2163 // 模板参数推导链 (面试速记):
2164 //   SM80_16x8x16_F16F16F16F16_TN ← MMA 指令 atom
2165 //   → MMA_Atom<MMA_Traits<mma_op>>
2166 //   → make_tiled_mma(mma_atom, EURepeat{2,2,1}, ValTile{32,32,16})
2167 //   → TiledMMA (128 threads, 4 warps × 2×2 slices)
2168 //
2169 //   SM80_CP_ASYNC_CACHEGLOBAL<uint128_t> ← G→S copy atom
2170 //   → Copy_Atom<Copy_Traits<g2s_copy_op>, T>
2171 //   → make_tiled_copy(copy_atom, ThrLayout{32,4}, ValLayout{1,8})
2172 //   → G2SCopy: 32×4=128 threads, each copies 1×8=8 halves = 128 bits
2173 //
2174 //   Swizzle<3,3,3> + Atom{8,BK} → composition → tile_to_shape{BM,BK,kStage}
2175 //   → SmemLayoutA/B: 带 XOR swizzle 的 smem 排布
2176 //
2177 //   SM75_U32x4_LDSM_N ← S→R copy atom (ldmatrix 的 CuTe 封装)
2178 //   → Copy_Atom<Copy_Traits<s2r_copy_op>, T>
2179 //
2180 //   UniversalCopy<uint128_t> ← S→G copy atom (128-bit store)
2181 //   → make_tiled_copy(copy_atom, ThrLayout{32,4}, ValLayout{1,8})
2182 //
2183 // ★ Tile 尺寸速查:
2184 //   BM=128: M 方向 block tile (16 × 2 EU × 4 Val = 128)
2185 //   BN=256: N 方向 block tile (8 × 2 EU × 16 Val... etc. 实际 8 × 2 × 16 = 256)
2186 //   BK=32: K 方向 block tile (= kMmaPK = 1 × 1 × 16 = 16? 不对, BK=32 由 SmemLayout 控制)
2187 //   kStage=2: 双缓冲 pipeline
2188 // =====
2189 template <typename T, const int Stages = 2>
2190 void launch_hgemm_mma_stages_tn_cute(T *a, T *b, T *c, int M, int N, int K) {
2191     using namespace cute;
2192
2193     auto BM = Int<128>{};
2194     auto BN = Int<256>{};
2195     auto BK = Int<32>{};
2196     auto KStage = Int<Stages>{};
2197     auto kSmemLayoutCBatch = Int<4>{}; // C smem pipeline depth
2198
2199     // SmemLayout: Swizzle<3,3,3> 做 512-value (1024-byte) XOR 置换消除 bank conflict
2200     // composition: 将 swizzle pattern 应用到 8×BK 的 base layout
2201     // tile_to_shape: 将 atom layout 扩展到完整的 BM×BK×kStage
2202     using SmemLayoutAtom = decltype(composition(
2203         Swizzle<3, 3, 3>{},
2204         make_layout(make_shape(Int<8>{}, Int<BK>{}),

```

```

2205         make_stride(Int<BK>{}, Int<1>{}))));
2206 using SmemLayoutA = decltype(tile_to_shape(
2207     SmemLayoutAtom{}, make_shape(Int<BM>{}, Int<BK>{}, Int<KStage>{})));
2208 using SmemLayoutB = decltype(tile_to_shape(
2209     SmemLayoutAtom{}, make_shape(Int<BN>{}, Int<BK>{}, Int<KStage>{})));
2210
2211 // MMA: SM80_16x8x16_F16F16F16F16_TN
2212 // TN = A row-major, B col-major (与 Phase 7b 手写 MMA 完全相同的布局约定)
2213 using mma_op = SM80_16x8x16_F16F16F16F16_TN;
2214 using mma_traits = MMA_Traits<mma_op>;
2215 using mma_atom = MMA_Atom<mma_traits>;
2216 using mma_atom_shape = mma_traits::Shape_MNK; // (Int<16>, Int<8>, Int<16>)
2217
2218 static constexpr int kMmaEURepeatM = 2;
2219 static constexpr int kMmaEURepeatN = 2;
2220 static constexpr int kMmaEURepeatK = 1;
2221 static constexpr int kMmaPM = 1 * kMmaEURepeatM * get<0>(mma_atom_shape{}); // 32
2222 static constexpr int kMmaPN = 2 * kMmaEURepeatN * get<1>(mma_atom_shape{}); // 32
2223 static constexpr int kMmaPK = 1 * kMmaEURepeatK * get<2>(mma_atom_shape{}); // 16
2224
2225 using MMA_EU_RepeatT = decltype(make_layout(make_shape(
2226     Int<kMmaEURepeatM>{}, Int<kMmaEURepeatN>{}, Int<kMmaEURepeatK>{})));
2227 using MMA_P_T = Tile<Int<kMmaPM>, Int<kMmaPN>, Int<kMmaPK>>;
2228 using MMA = decltype(make_tiled_mma(mma_atom{}, MMA_EU_RepeatT{}, MMA_P_T{}));
2229
2230 // G2S Copy: cp.async 128-bit, 32x4 threads each loading 1x8 halves
2231 using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
2232 using g2s_copy_traits = Copy_Traits<g2s_copy_op>;
2233 using g2s_copy_atom = Copy_Atom<g2s_copy_traits, T>;
2234 using G2SCopyA = decltype(make_tiled_copy(
2235     g2s_copy_atom{},
2236     make_layout(make_shape(Int<32>{}, Int<4>{}),
2237         make_stride(Int<4>{}, Int<1>{})),
2238     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2239 using G2SCopyB = G2SCopyA;
2240
2241 // S2R Copy: ldmatrix (SM75_U32x4_LDSM_N), 由 make_tiled_copy_A/B 自动与 TiledMMA 对齐
2242 using s2r_copy_op = SM75_U32x4_LDSM_N;
2243 using s2r_copy_traits = Copy_Traits<s2r_copy_op>;
2244 using s2r_copy_atom = Copy_Atom<s2r_copy_traits, T>;
2245 using S2RCopyAtomA = s2r_copy_atom;
2246 using S2RCopyAtomB = s2r_copy_atom;
2247
2248 // Epilogue smem layout for C: Swizzle<3,3,3> on 32x32 atom, 4 pipeline stages
2249 using SmemLayoutAtomC = decltype(composition(
2250     Swizzle<3, 3, 3>{},
2251     make_layout(make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}),
2252         make_stride(Int<kMmaPN>{}, Int<1>{}))));
2253 using SmemLayoutC = decltype(tile_to_shape(
2254     SmemLayoutAtomC{},
2255     make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}, Int<kSmemLayoutCBatch>{})));
2256
2257 // R2S Copy: 寄存器 → shared memory (epilogue 阶段, UniversalCopy<int> 做逐 int 拷贝)
2258 using R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>;
2259
2260 // S2G Copy: shared memory → global (128-bit store)
2261 using S2GCopyAtomC = Copy_Atom<UniversalCopy<cute::uint128_t>, T>;
2262 using S2GCopyC = decltype(make_tiled_copy(
2263     S2GCopyAtomC{},
2264     make_layout(make_shape(Int<32>{}, Int<4>{}),
2265         make_stride(Int<4>{}, Int<1>{})),
2266     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2267

```



```

2268 // Grid/Block 配置
2269 int BX = (N + BN - 1) / BN;
2270 int BY = (M + BM - 1) / BM;
2271 dim3 block(size(MMA{})); // 128 threads
2272 dim3 grid(BX, BY);
2273
2274 // Smem 大小: A tile + B tile (C epilogue 复用 A tile 空间)
2275 static constexpr int shm_size_AB =
2276     cute::cosize(SmemLayoutA{}) + cute::cosize(SmemLayoutB{});
2277 static constexpr int shm_size_C = cute::cosize(SmemLayoutC{});
2278 // C smem ≤ A 的一个 pipe, 可以安全复用
2279 static_assert(size<0>(SmemLayoutA{}) * size<1>(SmemLayoutA{}) >= size(SmemLayoutC{}),
2280     "C shared memory must fit within one A pipe");
2281 static constexpr int kShmSize =
2282     cute::max(shm_size_AB, shm_size_C) * sizeof(T);
2283
2284 cudaFuncSetAttribute(
2285     hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2286         SmemLayoutA, SmemLayoutB, SmemLayoutC,
2287         S2RCopyAtomA, S2RCopyAtomB, R2SCopyAtomC,
2288         S2GCopyAtomC, S2GCopyC>,
2289     cudaFuncAttributeMaxDynamicSharedMemorySize, kShmSize);
2290
2291 hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2292     SmemLayoutA, SmemLayoutB, SmemLayoutC, S2RCopyAtomA,
2293     S2RCopyAtomB, R2SCopyAtomC, S2GCopyAtomC, S2GCopyC>
2294     <<<grid, block, kShmSize>>>(a, b, c, M, N, K);
2295 }
2296
2297 #endif /* NOTES_V2_ENABLE_CUTE */
2298
2299 // =====
2300 // Phase 7d: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
2301 // =====
2302 // 面试要点 (WGMMMA vs MMA 对比):
2303 // - MMA: warp 级 (32 threads), 同步执行
2304 // - WGMMMA: warpgroup 级 (128 threads = 4 warps), 异步执行 (fire-and-forget)
2305 // - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA m16n8k16的 4×16=64倍)
2306 // - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
2307 // - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过 barrier 同步
2308 // - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
2309
2310 // ---- WGMMMA 辅助函数 ----
2311 // wgmma.commit_group / wait_group 语义 (PTX ISA §9.7.15.7):
2312 // - commit_group: 将此前所有未提交的 wgmma.mma_async 归入一个新的 wgmma-group
2313 //   (per-warpgroup, 故需 .sync.aligned 确保所有线程同步执行)。
2314 // - wait_group N: 阻塞直到最多 N 个 wgmma-group 尚未完成 (pending ≤ N)。
2315 //   N=0 → 等所有 group 完成。★ 与 cp.async.wait_group 语义一致 (PTX ISA §9.7.9.25.3.3),
2316 //   都是 "wait until only N or fewer groups are pending"。
2317 #define WGMMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
2318 #define WGMMMA_COMMIT_GROUP() \
2319     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
2320 #define WGMMMA_WAIT_GROUP(n) \
2321     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(n) : "memory")
2322
2323 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMMA descriptor 位域中的 14-bit 值。
2324 // 编码规则 (PTX ISA §9.7.15.5.1.2.2):
2325 // encoded = (x & 0x3FFFF) >> 4
2326 // 其中 0x3FFFF = 2^18 - 1 = 262143, 只保留低 18 bit。
2327 // >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (2^4), 低 4 bit 恒为 0,
2328 // 可省略以扩大可表示范围。
2329 // 解码: decoded = encoded << 4 (回到原始 byte 值)。
2330 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4

```



```

2331
2332 // make_smem_desc: 构造 WGMMMA (warpgroup MMA) 的 64-bit shared memory 矩阵描述符。
2333 // 硬件 WGMMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,
2334 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
2335 //
2336 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor) :
2337 // -----
2338 // | 位域          | 范围      | 大小 | 含义
2339 // |-----|-----|-----|-----
2340 // | start_address  | [0, 14)   | 14   | smem 基址编码
2341 // | (unused)       | [14, 16)  | 2    | -
2342 // | leading_byte_offset | [16, 30) | 14   | 主要维度 (K) 的字节步长; swizzle 下硬件未使用 (assumed=1)
2343 // | (unused)       | [30, 32)  | 2    | -
2344 // | stride_byte_offset | [32, 46) | 14   | 跨越维度 (M/N) 的字节步长: K-Major 下为 8-row stripe 间距
2345 // | (unused)       | [46, 49)  | 3    | -
2346 // | base_offset    | [49, 52)  | 3    | swizzle pattern 偏移
2347 // | (unused)       | [52, 62)  | 10   | -
2348 // | layout_type    | [62, 64)  | 2    | swizzle 模式
2349 // -----
2350 // layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
2351 //
2352 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2) :
2353 // - 128B swizzle atom 在 128-bit 单位下为 8x8
2354 // - 归一化到 half(2B) 后: atom 覆盖 8x(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
2355 // - 即每个 atom = 64(K) × 8(M) 个 half = 64x8x2 = 1024 bytes
2356 // - 这是 stride_byte_offset=1024 的来源
2357 //
2358 // - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1) ,
2359 // 此处填 16 仅为占位, 编码后为 1。
2360 //
2361 // - base_offset=0 (bits [49,52) 未显式置位) , 表示 smem ptr 已 1024B 对齐。
2362 // 若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
2363 //
2364 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
2365 // 注: descA/descB 共用此函数; 对 B 而言物理存储为 [BN, BK], 详见 WgmmaSMem 注释。
2366 __device__ inline uint64_t make_smem_desc(half *ptr) {
2367 // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
2368 // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址)。
2369 uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
2370
2371 // 从全零开始: 所有位域初始化为 0。
2372 uint64_t desc = 0x0000000000000000;
2373
2374 // — bits [0, 14): start_address —
2375 // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit) ,
2376 // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
2377 // 可寻址范围: 2^(14+4) = 2^18 = 256K 个 half = 512 KB 共享内存。
2378 desc |= SMEM_DESC_ENCODE(addr);
2379
2380 // — bits [16, 30): leading_byte_offset —
2381 // 原始值 16 (字节), 编码后为 (16 & 0x3FFFF) >> 4 = 1。
2382 // << 16 将编码值放到 bits [16, 30)。
2383 // K-Major + 128B swizzle 下该字段硬件未使用 (assumed to be 1, 参见 PTX ISA §9.7.15.5.1.2.1.1) ,
2384 // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
2385 // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
2386 // 对 INTERLEAVE: 同一 8x2 brick 内第一列到第二列的字节偏移。
2387 // 对 MN-Major swizzle: 偏移从首 (swizzle-byte-size/16) 行到下一组行。
2388 desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
2389
2390 // — bits [32, 46): stride_byte_offset —
2391 // 原始值 1024 (字节), 编码后为 (1024 & 0x3FFFF) >> 4 = 64。
2392 // << 32 将编码值放到 bits [32, 46)。
2393 // K-Major 下定义为"从首 8 行到次 8 行的字节偏移" (PTX ISA §9.7.15.5.1.2.1.2) 。

```

```

2394 // 1024 的来源 (K-Major + 128B swizzle + half) :
2395 // 一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
2396 // 从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
2397 // 若为 MN-Major: SB0 是从首列到下一列的偏移 (8 列一组)。
2398 desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
2399
2400 // — bits [62, 64): layout_type (swizzle mode) —
2401 // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
2402 // 1 = 128B swizzle mode。
2403 // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
2404 // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射,
2405 // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
2406 desc |= 1llu << 62;
2407
2408 return desc;
2409 }
2410
2411 // ---- WGMMMA PTX 指令宏 ----
2412 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
2413 // m64n128k16: M=64, N=128, K=16。一次 WGMMMA 计算 D[64×128] += A[64×16] × B[16×128]。
2414 // f16.f16.f16: A=f16, B=f16, D=f16 (累加器也是 f16 精度)。
2415 // 每线程 32 个 uint32 输出: D=64×128=8192 half, warpgroup 128 线程分担,
2416 // 每线程 64 half = 32 uint32。
2417 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成, 64-bit 编码)。
2418 // ScaleD: 0=清零累加器再写入, 1=与累加器中的旧值累加 (K 维迭代必须用 1)。
2419 // ScaleA/ScaleB: 1=正常符号, -1=翻转符号 (此处始终用 1)。
2420 // TransA/TransB: 0=K-major (K 维连续), 1=MN-major (M/N 维连续)。本实现始终用 0。
2421 // 参考: PTX ISA §9.7.15.4 (wgmma.mma_async)
2422 #define WGMMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA, \
2423                                     TransB) \
2424 { \
2425     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
2426     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
2427     asm volatile( \
2428         "{\n" \
2429         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
2430         "{%0, %1, %2, %3, %4, %5, %6, %7, " \
2431         "%8, %9, %10, %11, %12, %13, %14, %15, " \
2432         "%16, %17, %18, %19, %20, %21, %22, %23, " \
2433         "%24, %25, %26, %27, %28, %29, %30, %31}, " \
2434         "%32, " \
2435         "%33, " \
2436         "%34, %35, %36, %37, %38;\n" \
2437         "}\n" \
2438         : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
2439         "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
2440         "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \
2441         "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
2442         "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
2443         "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
2444         "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
2445         "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
2446         : "l"(desc_a), "l"(desc_b), "n"(int32_t(ScaleD)), \
2447         "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
2448         "n"(int32_t(TransB))); \
2449 }
2450
2451 // ---- TMA Shared Memory Layout ----
2452 // Multi-stage pipeline: K_STAGE 个 stage, 每个 stage 存储 A[BM×BK] + B[BK×BN]
2453 //
2454 // 每个 stage 包含两块 smem:
2455 //   A tile: [BM, BK] row-major → BK×BM 个 half, 地址连续
2456 //   B tile: TMA 按 [BN, BK] 物理写入 (详见下方 ★), BN×BK 个 half

```

```

2457 //
2458 // ★ 关键区别 — WGMMMA 的 B 物理存储 vs 逻辑视图:
2459 //
2460 // 上层数据: 源矩阵  $B^T [N, K]$  row-major (TN 布局, B 的列以行优先形式存储)。
2461 // 物理存储: TMA 将  $B^T$  的 tile 写入 smem, 物理布局为  $[BN, BK]$  row-major
2462 // (BN=128 行, BK=64 列, 每行 64 个连续的 half)。
2463 // TMA 的 smem_box_shape = (BK=64, BN=128) 定义了每个 tile 的 shape,
2464 // TMA 按行写入, 行内 BK 个 half 连续 → 物理上就是  $[BN][BK]$ 。
2465 // 逻辑视图: WGMMMA 指令 (wgmma.mma_async.m64n128k16, PTX ISA §9.7.15.4)
2466 // 通过 imm-trans-b=0 指定 B 为 K-major, 即逻辑上按  $[BK, BN]$  寻址。
2467 // 实际  $[BN, BK]$  → 逻辑  $[BK, BN]$  的"转置"是通过 descB 中的 128B
2468 // swizzle (layout_type=1) 在硬件地址重映射层完成的, 无需软件转置。
2469 //
2470 // stride_byte_offset=1024 的来源 (K-major + 128B swizzle + f16, PTX ISA §9.7.15.5.1.2.1.2) :
2471 // 128B swizzle atom =  $8(N) \times 8(K) \times 128\text{-bit} = 8 \text{ rows} \times 64 \text{ 个 half}$ 。
2472 // stride_byte_offset = 从当前 8-row stripe 到下一个 8-row stripe 的字节偏移
2473 // =  $8 \text{ rows} \times (BK=64) \text{ halves} \times 2 \text{ bytes} = 1024$ 。
2474 // 这个值恰好等于物理  $[BN, BK]$  布局中连续 8 行的跨度 ← 进一步证实物理存储是  $[BN, BK]$ 。
2475 //
2476 // 与 MMA(TN) 的对比:
2477 // MMA (TN): smem 存  $B^T [N, K]$  row-major, ldmatrix 按列解出 col-major B fragment。
2478 // WGMMMA: smem 物理存  $[BN, BK]$  (TMA 直写), WGMMMA 通过 swizzle 硬件以 K-major  $[BK, BN]$  逻辑视图读取。
2479 // 简言之: swizzle 桥接了 "物理 row-major  $[BN, BK]$ " 和 "逻辑 K-major  $[BK, BN]$ " 的差异。
2480 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
2481     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
2482     // B tile 笔记:
2483     // - 物理存储: TMA 从  $B^T[N, K]$  row-major 搬入, 物理上是  $[BN, BK]$  row-major (BN 行, BK 列)
2484     // - 逻辑视图: WGMMMA 通过 descB 的 128B swizzle (PTX ISA §9.7.15.5.1.2) 将地址重映射,
2485     // 以 K-major  $[BK, BN]$  逻辑视图供 wgmma.mma_async (imm-trans-b=0, PTX ISA §9.7.15.4) 读取
2486     // -  $B[BN * BK * QSIZE]$  与  $B[BK * BN * QSIZE]$  数值等价 ( $BN \times BK = BK \times BN$ ),
2487     // 但写  $BN \times BK$  更能体现物理存储形态
2488     alignas(128) half B[BN * BK * QSIZE];
2489 };
2490
2491 // ---- WGMMMA Kernel: Warp Specialization + TMA ----
2492 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化) :
2493 //
2494 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
2495 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMMA 支持将工作拆分为两个
2496 // warpgroup, 让数据搬运和矩阵乘完全异步执行:
2497 //
2498 // WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
2499 // 将 A/B tile 从 HBM 搬到 shared memory。
2500 // WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMMA 矩阵乘。
2501 //
2502 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
2503 // - full[qidx]: Producer 发信号表示 stage qidx 的数据已就绪, 可被使用
2504 // - empty[qidx]: Consumer 发信号表示 stage qidx 的使用完毕, 可以被覆盖
2505 //
2506 // 本节重点理解:
2507 // 1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
2508 // 即可搬运整个 2D tile, 无需所有线程参与。
2509 // 2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
2510 // 3) 多 stage pipeline 使 Consumer 计算 stage qidx 的同时,
2511 // Producer 可以搬运 stage (qidx+1), 隐藏 HBM→SMEM 延迟。
2512 //
2513 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比) :
2514 // WGMMMA Atom: m64n128k16 (一次处理  $64 \times 128 \times 16$ , 是 MMA 的 64 倍)
2515 // K Tile: BK=64, 每个 K tile 包含  $BK/WGMMMA\_K=4$  个 WGMMMA atom
2516 // M Tile: BM=128, 每个 M tile 包含  $BM/WGMMMA\_M=2$  个 WGMMMA atom
2517 // N Tile: BN=128, 单个 WGMMMA_N=128 即可覆盖, 无需在 N 方向分块
2518 // Block Tile:  $C[128, 128] = A[128, 64] \times B[64, 128]$ 
2519 // Threads:  $256 = 2 \text{ warpgroups} \times 128 \text{ threads/warpgroup}$ 

```

```

2520 //      Producer(WG0): 128 threads (仅 thread 0 做 TMA 提交)
2521 //      Consumer(WG1): 128 threads = 4 warps (全部参与 WGMMMA 和写回)
2522 //
2523 // K_STAGE=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步, 确保 HBM-SMEM
2524 // 的延迟被计算完全掩盖。
2525 //
2526 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
2527 // - grid.z = S 个 swizzle 分区, 将连续 block 打散到不同 N 区域改善 L2 命中
2528 // Block: (256, 1, 1), 2 warpgroups
2529 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
2530 template <const int WGMMMA_M = 64, const int WGMMMA_N = 128,
2531          const int WGMMMA_K = 16, const int BM = 128, const int BN = 128,
2532          const int BK = 64, const int NUM_THREADS = 256, const int K_STAGE = 3,
2533          const bool BLOCK_SWIZZLE = false>
2534 __global__ void __launch_bounds__(NUM_THREADS)
2535   hgemm_wgmma_stages_tn(
2536       int M, int N, int K, half *C,
2537       const CUtensorMap *__restrict__ tensorMapA,
2538       const CUtensorMap *__restrict__ tensorMapB) {
2539     // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
2540     // 对 row-major [M, K] 矩阵, TMA shape 参数写的是 (K, M) 而不是 (M, K),
2541     // 也就是TMA descriptor中把连续的维度写在最内层, 非连续的维度写在最外层。
2542     // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
2543     // 不展开宿主侧 create_tensor_map 细节。
2544
2545     // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
2546     // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
2547     const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
2548     const int by = blockIdx.y;
2549     // num_consumers = (NUM_THREADS / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
2550     // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer
2551     constexpr int num_consumers = (NUM_THREADS / 128) - 1; // 1 consumer WG
2552     // B_WG_M = BM / num_consumers: 每个 consumer warpgroup 负责的 M 行数
2553     // 当只有一个 consumer 时, 它负责全部 BM=128 行
2554     constexpr int B_WG_M = BM / num_consumers; // 128
2555
2556     // 边界检查: 确保当前 block 不超出 M/N 范围
2557     if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
2558         return;
2559
2560     // ---- Shared Memory 分配 ----
2561     // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
2562     // __align__(128) 满足 TMA 和 WGMMMA 的 16B 对齐 + 128B swizzle 对齐要求
2563     extern __shared__ __align__(128) uint8_t smem_AB[];
2564     WgmmaSMem<BM, BN, BK, K_STAGE> &s =
2565         *reinterpret_cast<WgmmaSMem<BM, BN, BK, K_STAGE> *>(smem_AB);
2566     half *s_a = s.A;
2567     half *s_b = s.B;
2568
2569     // ---- cuda::barrier 初始化 ----
2570     //
2571     // ★ Barrier 机制速查 (arrive / wait 核心语义):
2572     //
2573     // cuda::barrier 是一个 CTA 级同步原语, 基于 **phase (奇偶交替)** 工作:
2574     //
2575     //   init(bar, N): 设置 arrive_count = N。
2576     //       含义: 每 phase 需要 N 个线程各调用一次 arrive(), barrier 才翻转 phase。
2577     //
2578     //   arrive(): 线程声明"我已到达此 barrier 点"。
2579     //       - 非阻塞, 立即返回一个 token (包含当前 phase 值)。
2580     //       - 不关心"谁"到达, 只关心"到达次数"是否达到 arrive_count。
2581     //
2582     //   wait(token): 线程阻塞, 直到 barrier 的当前 phase ≠ token 中的 phase。

```

```

2583 // - 即: 等待 phase 翻转。
2584 // - Phase 翻转条件: (1) arrive 次数达到 arrive_count
2585 // (2) 若使用了 barrier_arrive_tx, 还需 TMA 字节全部写完
2586 //
2587 // 典型用法: b.wait(b.arrive())
2588 // - arrive() 注册自己到达, 拿到 token (当前 phase = P)
2589 // - wait(token) 阻塞直到 phase 翻转 (P → P+1)
2590 // - 如果自己是第 N 个到达者 → phase 立即翻转 → wait 立即返回 (不阻塞)
2591 // - 如果自己不是最后一个 → wait 阻塞, 直到第 N 个到达者触发翻转
2592 //
2593 // ★ 本 kernel 的 Pipeline 同步协议 (K_STAGE=3, arrive_count=129) :
2594 //
2595 // Producer (1 thread) Consumer (128 threads)
2596 // -----
2597 // C0: arrive(empty[*]) ×128 [init: 标记所有 stage 为空]
2598 // ┌ for each k_tile: ─┐ ┌ for each k_tile: ─┐
2599 // | P1: arrive+wait(empty[q]) | C1: arrive+wait(full[q])
2600 // |   ↓ 等 stage q 被消费完 |   ↓ 等 TMA 数据就绪
2601 // | P2: TMA(A[q]) + TMA(B[q]) | C2: WGMMMA 计算
2602 // |   ↓ 异步拷贝 | C3: wait WGMMMA 完成
2603 // | P3: arrive_tx(full[q]) | C4: arrive(empty[q])
2604 // |   ↑ 通知: stage q 数据就绪 |   ↑ 通知: stage q 已消费完
2605 // └──────────────────┘ └──────────────────┘
2606 //
2607 // 关键不变式 (invariant) :
2608 // - Producer 不会覆盖 Consumer 正在读的 stage
2609 // - Consumer 不会读 Producer 还没写完的 stage
2610 // - 通过 full/empty 两个 barrier 的 phase 交替来保证
2611 //
2612 // 每个 stage 有两个 barrier:
2613 // full[qidx]: TMA 数据就绪信号。Producer 发 (arrive_tx), Consumer 等 (wait)。
2614 // empty[qidx]: Stage 空闲信号。Consumer 发 (arrive), Producer 等 (wait)。
2615 //
2616 // arrive_count = 128 (consumer) + 1 (producer) = 129:
2617 // 每 phase, 128 个 consumer 线程 + 1 个 producer 线程都要 arrive 一次。
2618 #pragma nv_diag_suppress static_var_with_dynamic_init
2619 __shared__ cuda::barrier<cuda::thread_scope_block> full[K_STAGE];
2620 __shared__ cuda::barrier<cuda::thread_scope_block> empty[K_STAGE];
2621 #pragma nv_diag_default static_var_with_dynamic_init
2622
2623 // K 方向总 tile 数。要求 K 能被 BK 整除, 否则尾 tile 被丢弃。
2624 const int num_blocks_k = K / BK;
2625 const int wg_idx = threadIdx.x / 128; // 0=Producer, 1=Consumer
2626 const int tid = threadIdx.x % 128; // 0-127 within warpgroup
2627
2628 // 初始化 barriers (仅 thread 0 执行)
2629 if (threadIdx.x == 0) {
2630     for (int i = 0; i < K_STAGE; ++i) {
2631         // arrive_count = 129: 128 consumer + 1 producer
2632         // init 是 CUDA barrier 的 hidden friend 函数 (定义在 cuda::barrier 类体内),
2633         // 只能通过 ADL (Argument-Dependent Lookup) 调用。因为第一个参数 &full[i] 的类型是
2634         // cuda::barrier<cuda::thread_scope_block>*, 编译器通过 ADL 在 cuda 命名空间中自动找到它。
2635         init(&full[i], num_consumers * 128 + 1); // 128 consumer + 1 producer
2636         init(&empty[i], num_consumers * 128 + 1); // same
2637     }
2638     // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
2639     // 对 async proxy (TMA/WGMMMA) 可见。
2640     cuda::device::experimental::fence_proxy_async_shared_cta();
2641 }
2642 __syncthreads();
2643
2644 // =====
2645 // ★ Warp Specialization 并发模型 — 为什么只有 if/else 没有 while?

```



```

2646 // =====
2647 //
2648 // 256 个线程在同一个 SM 上**并发执行**。if/else 只是分工，不是先后顺序：
2649 //   - WG0 (threadIdx.x 0~127): 全部走 if 分支，做 Producer。
2650 //   - WG1 (threadIdx.x 128~255): 全部走 else 分支，做 Consumer。
2651 //
2652 // SM 的 warp scheduler 会交替调度来自 WG0 和 WG1 的 warp，两者**同时**推进：
2653 //   Producer: for (k_iter = 0 .. num_blocks_k) { P1→P2→P3 } ← 自己的循环
2654 //   Consumer: for (k_iter = 0 .. num_blocks_k) { C1→C2→C3→C4 } ← 自己的循环
2655 //
2656 // 两个 warpgroups 各自独立迭代 K-tiles，通过 full[] / empty[] barrier 同步：
2657 //   - Producer 领先 Consumer 太多 → wait(empty[q]) 阻塞，等 Consumer 消费完
2658 //   - Consumer 领先 Producer 太多 → wait(full[q]) 阻塞，等 TMA 搬运完
2659 //
2660 // 这是生产者-消费者模型的 GPU 实现：不需要共享循环变量，barrier 的 phase
2661 // 交替天然保证了流水线串行化 (at most K_STAGE-1 steps ahead)。
2662 // =====
2663 // Producer Warpgroup (WG0, threadIdx.x 0~127)
2664 // 职责：提交 TMA 2D 拷贝，将 A/B tile 从 HBM 异步搬运到 SMEM。
2665 // 只有 tid==0 执行实际拷贝提交，其余 127 个线程空闲。
2666 // =====
2667 if (wg_idx == 0) {
2668     if (tid == 0) {
2669         // qidx: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
2670         int qidx = 0;
2671         for (int block_k_iter = 0; block_k_iter < num_blocks_k; ++block_k_iter, ++qidx) {
2672             if (qidx == K_STAGE)
2673                 qidx = 0;
2674
2675             // Step P1: 等待 stage qidx 变为"空" (可被覆盖写入)
2676             //
2677             // 模式 empty[qidx].wait(empty[qidx].arrive()):
2678             //   - arrive(): Producer (1 个线程) 在 empty barrier 上注册到达，
2679             //     获得当前 phase 的 token。如果 Consumer 已经在 C4 步骤积累了
2680             //     128 次 arrive (来自上一轮迭代或 C0 初始化)，则加上这次共 129 次
2681             //     → phase 立即翻转。
2682             //   - wait(token): 阻塞直到 barrier phase ≠ token 中的 phase。
2683             //     由于 arrive() 是第 129 次到达，phase 在 arrive 时已翻转，
2684             //     wait 看到 phase 已变 → 立即返回 (首轮不阻塞)。
2685             //
2686             // 语义：Producer 说"我准备好写 stage qidx 了，Consumer 用完没有？"
2687             //     如果 Consumer 还没用完 → phase 未翻转 → wait 阻塞等待。
2688             //     如果 Consumer 已用完 → phase 已翻转 → wait 立即返回。
2689             empty[qidx].wait(empty[qidx].arrive());
2690
2691             // Step P2: 提交 TMA 2D 拷贝指令
2692             // cp_async_bulk_tensor_2d_global_to_shared:
2693             //   参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
2694             //   参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
2695             //   参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
2696             //   参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
2697             //
2698             // TMA 是硬件 DMA 引擎：一次指令提交即可搬运整个 2D tile，
2699             // 无需线程逐元素搬运，零寄存器开销。
2700             // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
2701             cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
2702                 &s_a[qidx * BK * BM], tensorMapA, block_k_iter * BK, by * BM,
2703                 full[qidx]);
2704
2705             // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
2706             cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
2707                 &s_b[qidx * BK * BN], tensorMapB, block_k_iter * BK, bx * BN,
2708                 full[qidx]);

```

```

2709 // Step P3: 通知 Consumer: stage qidx 的 TMA 数据已就绪
2710 //
2711 // barrier_arrive_tx(bar, arrive_count_update, byte_count):
2712 // - 在 bar 上注册 1 次到达 (arrive_count_update=1)
2713 // - 同时声明预期有 byte_count 字节将通过 async copy (TMA) 写入 smem
2714 // - Phase 翻转条件:
2715 // (a) 总 arrive 次数达到 129 (128 consumer + 1 producer)
2716 // (b) 所有声明的 async 字节已写入 smem
2717 // 两个条件都满足后 phase 才翻转, Consumer 的 wait() 才返回。
2718 // - 即: Consumer 不会在 TMA 数据完整到达前就开始读 smem。
2719 [[maybe_unused]] auto token = cuda::device::barrier_arrive_tx(
2720     full[qidx], 1, (BK * BN + BK * BM) * sizeof(half));
2721 }
2722 }
2723 }
2724 }
2725 // =====
2726 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
2727 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C
2728 // 所有 128 个线程 (4 warps) 全部参与。
2729 // =====
2730 else {
2731     // Step C0: Consumer 初始化 — 标记所有 stage 为"空" (可被 Producer 写入)
2732     //
2733     // 所有 128 个 Consumer 线程对每个 stage 的 empty barrier 调用 arrive()。
2734     // 这是 Pipeline 的"预热"步骤——没有它, Producer 的 empty[qidx].wait()
2735     // 在第一轮会永远阻塞 (因为 Producer 的 1 次 arrive 不足以凑够 129)。
2736     //
2737     // 注意: 此时每个 empty[i] 只有 128 次 arrive, 未达 129, phase 不翻转。
2738     // Producer 后续的 empty[qidx].arrive() 作为第 129 次, 触发 phase 翻转。
2739     for (int i = 0; i < K_STAGE; ++i) {
2740         [[maybe_unused]] auto token = empty[i].arrive();
2741     }
2742
2743     // 累加器寄存器声明
2744     // d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4]:
2745     // - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
2746     // - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
2747     // - d[*][g][*]: N 方向第 g 组 16 列
2748     // - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16x16 子块)
2749     // 每个线程总共 2 * 8 * 4 = 2 * 32 = 64 uint32 = 128 half (每次WGMMMA Atom 32 uint32 输出)
2750     // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
2751     uint32_t d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4] = {};
2752
2753     int qidx = 0;
2754     // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
2755     for (int block_k_iter = 0; block_k_iter < num_blocks_k; ++block_k_iter, ++qidx) {
2756         if (qidx == K_STAGE)
2757             qidx = 0;
2758
2759         // Step C1: 等待 TMA 数据就绪 (full 信号)
2760         //
2761         // 模式 full[qidx].wait(full[qidx].arrive()):
2762         // - 128 个 Consumer 线程各调用 arrive(), 共 128 次到达。
2763         // - 当前 phase 累积: 128/129, phase 尚未翻转。
2764         // - wait(token) 阻塞, 直到 Producer 的 barrier_arrive_tx (Step P3)
2765         //   贡献第 129 次到达 + TMA 字节全部写完 -> phase 翻转 -> wait 返回。
2766         //
2767         // 语义: Consumer 说"我准备好读 stage qidx 了, 数据到了没有? "
2768         full[qidx].wait(full[qidx].arrive());
2769
2770         // Step C2: 发射 WGMMMA 指令序列
2771         //

```



```

2772 // WGMMA 是异步指令 (fire-and-forget)，发射后立即返回，不等待计算完成。
2773 // 标准流程: FENCE → 发射 WGMMA → COMMIT → WAIT。
2774 //
2775 // WGMMA_FENCE (wgmma.fence.sync.aligned) :
2776 //   - 确保 TMA 写入 smem 的数据对 async proxy (WGMMA) 可见
2777 //   - 确保累加器寄存器 d[] 已准备好接收 WGMMA 输出
2778 //   - 本质是 proxy 间的内存序 (memory ordering)，不是传统 __syncthreads
2779 //
2780 // 每个 WGMMA_M64N128K16_F16F16F16 指令:
2781 //   D[64×128] += A[64×16] × B[16×128]
2782 //   A/B 均为 K-major (imm-trans=0)，由 descA/descB 描述 smem 中的布局。
2783 //   Scaled=1: 累加。K 维有 BK/WGMMA_K=4 次迭代，每次都用 Scaled=1。
2784 WGMMA_FENCE();
2785
2786 // M 维迭代: BM/WGMMA_M = 128/64 = 2 个 WGMMA atom
2787 // 每个 atom 处理 64 行 M 方向数据。
2788 #pragma unroll
2789 for (int m_it = 0; m_it < B_WG_M / WGMMA_M; ++m_it) {
2790     // wgmma_sA 指向当前 stage 中 A tile 的第 m_it 个 64*BK 子块
2791     // s_a 布局: [K_STAGE][BM][BK] -> qidx * BK*BM + BK * (m_it*64)
2792     half *wgmma_sA = s_a + qidx * BK * BM + BK * m_it * WGMMA_M;
2793
2794     // K 维迭代: BK/WGMMA_K = 64/16 = 4 次 WGMMA (累加)
2795     // 每次处理 K=16 维的矩阵乘，4 次累加后覆盖完整的 BK=64。
2796 #pragma unroll
2797 for (int k_it = 0; k_it < BK / WGMMA_K; ++k_it) {
2798     // 第 k_it 次 WGMMA:
2799     //   A: wgmma_sA + k_it * WGMMA_K (A 的 K 维起始位置)
2800     //   B: s_b + qidx * BK * BN + k_it * WGMMA_K (B 的 K 维起始位置)
2801     //   注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
2802     //   第 k_it*16 行开始取 16 行，每行 BN=128 列。
2803     //   Scaled=1: 累加 (K 维迭代需要累积结果)
2804     WGMMA_M64N128K16_F16F16F16(d[m_it], wgmma_sA + k_it * WGMMA_K,
2805                                   s_b + qidx * BK * BN + k_it * WGMMA_K,
2806                                   1, // Scaled=1: accumulate (不清零)
2807                                   1, 1, 0, 0);
2808 }
2809 }
2810
2811 // Step C3: 提交并等待 WGMMA 完成
2812 //
2813 // WGMMA_COMMIT_GROUP (wgmma.commit_group.sync.aligned):
2814 //   将自上一次 COMMIT 以来发射的所有 WGMMA 归为一组，提交到 async proxy 执行。
2815 //   类比 cp.async.commit_group，但 scope 是 warpgroup 而非 per-thread。
2816 //
2817 // WGMMA_WAIT_GROUP(0) (wgmma.wait_group.sync.aligned 0):
2818 //   阻塞直到最多 N 个 group 尚未完成 (pending ≤ N)。N=0 即等待所有 group。
2819 //   ★ 语义与 cp.async.wait_group 完全一致 (PTX ISA §9.7.15.7.3 vs §9.7.9.25.3.3) :
2820 //   两者都是 "wait until only N or fewer of the most recent groups are pending"。
2821 //   此处用 0 是因为 Consumer 在本次迭代中只 commit 了 1 个 group，
2822 //   必须等它全部完成才能进入下一步 (读下一 stage 的数据)。
2823 WGMMA_COMMIT_GROUP();
2824 WGMMA_WAIT_GROUP(0);
2825
2826 // Step C4: 释放 stage qidx — 通知 Producer 可以覆盖写入
2827 //
2828 // 128 个 Consumer 线程在 empty[qidx] 上 arrive(), 为 **下一 phase** 积累到达次数。
2829 // 这 128 次 arrive 不会立即翻转 phase (还需 Producer 在下一轮的 1 次 arrive)。
2830 //
2831 // 语义: Consumer 说 "stage qidx 我已经读完了，你可以放心覆盖了。"
2832 [[maybe_unused]] auto token = empty[qidx].arrive();
2833 }
2834

```

```

2835 // =====
2836 // Epilogue: 将寄存器中的累加结果写回 global memory C
2837 // =====
2838 //
2839 // WGMMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
2840 //
2841 // 对 warpgroup 内每个线程, 输出 64 个 half = 32 个 uint32。
2842 // 这些 half 分布在 d[2][8][4] 数组中:
2843 //   d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
2844 //   d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
2845 //   d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
2846 //   d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
2847 //
2848 // 其中:
2849 //   row = warp * 16 + lane / 4    (0~63, 4 warps * 16 rows)
2850 //   col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
2851 //
2852 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
2853 //   +-----+-----+
2854 //   | reg[0] | reg[2] | <- rows [row, row+8)
2855 //   |(col,+2)|(col+8)|
2856 //   +-----+-----+
2857 //   | reg[1] | reg[3] | <- rows [row+8, row+16)
2858 //   |(col,+2)|(col+8)|
2859 //   +-----+-----+
2860 // 每个 reg 包含 2 个连续 half (col 和 col+2), 用 uint32 存储。
2861 //
2862 // 三维遍历:
2863 //   m_it=0: rows 0~63, m_it=1: rows 64~127
2864 //   g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
2865 // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
2866 // 128 线程 * 128 half = 16384 half = 128 * 128 ok
2867
2868 const int lane = tid % 32;
2869 const int warp = tid / 32;
2870 // row: 当前线程在当前 WGMMMA atom 中负责的起始行。
2871 // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]。
2872 // lane/4: 每 4 个连续 lane 负责同一行 (因为 WGMMMA fragment 中每行有 4 个 uint32,
2873 //         分别覆盖列对 {c0,c1}、{c2,c3}、{c8,c9}、{c10,c11}, 由 lane%4 区分)。
2874 //         因此 lane/4 给出该行在 16-row block 内的行号 (0~7 对应 rows 0~7,
2875 //         同一 warp 中 lane/4==0 的有 lane 0,1,2,3, 都负责 row 0)。
2876 const int row = warp * 16 + lane / 4;
2877 // block_C: 当前 C tile 在 global memory 中的起始地址
2878 // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
2879 half *block_C = C + by * BM * N + bx * BN;
2880
2881 // 2 * (8 * 4) = 2 * 32 = 64 uint32 = 128 half
2882 #pragma unroll
2883 for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
2884     int yo = m_it * WGMMMA_M; // M 方向行偏移 (0 或 64)
2885     #pragma unroll
2886     for (int g = 0; g < WGMMMA_N / 16; ++g) {
2887         int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
2888         // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
2889         // 左上象限: (row, col)
2890         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) =
2891             d[m_it][g][0];
2892         // 左下象限: (row+8, col)
2893         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) =
2894             d[m_it][g][1];
2895         // 右上象限: (row, col+8)
2896         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) =
2897             d[m_it][g][2];

```

```

2898     // 右下象限: (row+8, col+8)
2899     *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) =
2900         d[m_it][g][3];
2901 }
2902 }
2903 }
2904 }
2905
2906 #if (defined(NOTES_V2_ENABLE_WGMMMA))
2907 // ---- Host-side TMA Tensor Map helpers (WGMMMA test) ----
2908 // 面试要点 (TMA descriptor 创建 — cuTensorMapEncodeTiled):
2909 //   - TMA descriptor 描述 global memory 中矩形的 shape/stride/dtype,
2910 //     以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
2911 //   - 关键易错点: TMA shape 参数写的是 (W,H) 而不是 (H,W)!
2912 //     对 row-major [M,K] 矩阵, TMA shape = (K,M), minor=K, major=M
2913 //   - cuTensorMapEncodeTiled 是 CUDA Driver API, 需 #include <cuda.h> 并链接 -lcuda
2914 //   - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
2915 //   - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMMA 的 128B swizzle
2916 //
2917 // ★ gmem_prob_stride + 1 的含义:
2918 // 语法层面 — 数组名退化 + 指针算术:
2919 //   gmem_prob_stride 类型为 uint64_t[5], 在表达式中退化为 uint64_t*
2920 //   (指向首元素 gmem_prob_stride[0])。+ 1 偏移 1 个 uint64_t, 指向
2921 //   gmem_prob_stride[1], 等价于 &gmem_prob_stride[1]。
2922 // 语义层面 — 跳过隐式的最内维 stride:
2923 //   cuTensorMapEncodeTiled 的 globalStrides 参数只接收 tensorRank-1 个
2924 //   stride 值。最内维 (fastest-changing dimension) 的 stride 是隐式的,
2925 //   固定等于 sizeof(element_type), 不需要显式传入。
2926 //   对于 tensorRank=2 的 FP16 矩阵:
2927 //       dim 0 (内维, K) : stride = sizeof(half)    ← 隐式, API 不需要
2928 //       dim 1 (外维, M) : stride = sizeof(half)*K  ← 需要传给 API
2929 //   gmem_prob_stride 数组的布局是内维在前:
2930 //       [0] = sizeof(half)                        ← 内维, 不传
2931 //       [1] = sizeof(half) * BlockMinorSize * blocks_width ← 外维, 传给 API
2932 //   所以 + 1 的作用是跳过 [0], 让 API 从 [1] 开始读取, 正好得到外维 stride。
2933 //
2934 // ★ smem_box_stride 为什么不需要 +1?
2935 //   与 globalStrides 不同, smemBoxStrides 参数接收完整的 tensorRank 个 stride,
2936 //   最内维 stride 也必须显式传入 (不隐式)。
2937 //   smem_box_stride[5] = {1, 1, 1, 1, 1} 表示 smem tile 中所有维度元素
2938 //   都是连续存放的, 维度间没有 padding/stride。
2939 //   对于 tensorRank=2: strides[0]=1 (内维 stride), strides[1]=1 (外维 stride),
2940 //   即 smem 中第 (i,j) 元素的偏移 = i*1 + j*1 = i+j (元素连续排列)。
2941 //   所以这里直接传 smem_box_stride (即 &smem_box_stride[0]), API 会读到全部
2942 //   两个 stride 值, 不需要跳过任何一个。
2943 //
2944 // 参考: CUDA Programming Guide §TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor
2945
2946 template <int BlockMajorSize, int BlockMinorSize>
2947 __host__ static inline void create_tensor_map(CUtensorMap *tma_map,
2948         half *gmem_ptr,
2949         int blocks_height,
2950         int blocks_width) {
2951     void *gmem_address = (void *)gmem_ptr;
2952     uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
2953         (uint64_t)BlockMajorSize * blocks_height,
2954         1, 1, 1};
2955     uint64_t gmem_prob_stride[5] = {
2956         sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
2957     uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
2958         uint32_t(BlockMajorSize), 1, 1, 1};
2959     uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
2960     CUresult result = cuTensorMapEncodeTiled(

```

```

2961     tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
2962     gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
2963     CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
2964     CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
2965     if (result != CUDA_SUCCESS)
2966         printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
2967 }
2968
2969 __host__ static inline CUTensorMap *allocate_and_create_tensor_map(
2970     half *src, int blocks_height, int blocks_width) {
2971     CUTensorMap *tma_map_d;
2972     cudaMalloc(&tma_map_d, sizeof(CUTensorMap));
2973     CUTensorMap tma_map_host;
2974     create_tensor_map<128, 64>(&tma_map_host, src, blocks_height, blocks_width);
2975     cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUTensorMap),
2976         cudaMemcpyHostToDevice);
2977     return tma_map_d;
2978 }
2979 #endif /* NOTES_V2_ENABLE_WGMMMA */
2980
2981 // =====
2982 // Phase 8: FlashAttention-2 (Split-Q + MMA m16n8k16)
2983 // =====
2984 // 面试要点 (FlashAttention 算法) :
2985 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
2986 //    但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
2987 // 2. FA 三板斧:
2988 //    a) Tiling: Q 分块 [Br,d], K/V 沿 seqlen 分块 [Bc,d]
2989 //    b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
2990 //    c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
2991 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
2992 //    - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
2993 //    - 优点: 减少 warp 间通信和 shuffle
2994 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691) :
2995 //    for each K,V tile:
2996 //        S_cur = Q @ K^T // 未缩放, 存入 R_S
2997 //        m_new = max(m_old, row_max(S_cur * scale))
2998 //        P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
2999 //        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
3000 //        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
3001 //        O_final = O_new / l_final
3002 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
3003 //    - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
3004 //    - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
3005 //      后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
3006 //    - 这是此实现的寄存器布局复用技巧, 不要背成“所有 MMA A/C fragment
3007 //      都天然同构”的通用结论
3008 //
3009 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
3010 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
3011 // Grid: ((QKV_seqlen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
3012 // Block: (128, 1, 1), kNumThreads=WARP_SIZE×kMmaTileSeqLenQ×kMmaTileSeqLenK=128
3013 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
3014
3015 // ---- 寄存器填充辅助函数 ----
3016 template <typename T, int M, const int N, const int K = 2>
3017 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
3018     #pragma unroll
3019     for (int i = 0; i < M; ++i)
3020     #pragma unroll
3021         for (int j = 0; j < N; ++j)
3022     #pragma unroll
3023         for (int k = 0; k < K; ++k)

```

```

3024     R[i][j][k] = val;
3025 }
3026
3027 template <typename T, int M, const int N = 2>
3028 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
3029     #pragma unroll
3030     for (int i = 0; i < M; ++i)
3031     #pragma unroll
3032         for (int j = 0; j < N; ++j)
3033             R[i][j] = val;
3034 }
3035
3036 // =====
3037 // FlashAttention-2 Split-Q Kernel (完整实现)
3038 // =====
3039 // Q,K,V,O: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
3040 //
3041 // Tile 设计 (以 kHeadDim=64 为例) :
3042 //   Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*4*1 = 64
3043 //   Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8 = 64
3044 //   Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
3045 //
3046 // 执行流程:
3047 //   1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
3048 //   2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
3049 //   3)   3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
3050 //   4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMA16816
3051 //   5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
3052 //       → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
3053 //   6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
3054 //       → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
3055 //   7)   3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
3056 //   8) 最终 rescale: O_final = (1/l_final) * O_final
3057 //   9) Epilogue: warp shuffle + 128-bit collective store
3058
3059 template <
3060     const int kHeadDim,           // head dim: 32, 64, 128
3061     const int kMmaAtomM,          // 16 (MMA instruction M dimension)
3062     const int kMmaAtomN,          // 8 (MMA instruction N dimension)
3063     const int kMmaAtomK,          // 16 (MMA instruction K dimension)
3064     const int kMmaTileSeqLenQ,    // MMA tiles along Q's M dim, 4 → Br=16*4=64
3065     const int kMmaTileSeqLenK,    // MMA tiles along K's N dim, 1 → Bc basis=8
3066     const int kMmaTileSeqLenP,    // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
3067     const int kMmaTileHeadDimV,   // MMA tiles for P@V N dim (head dim direction)
3068     const int kValTileSeqLenQ,    // value tiles along Q's M, 1 → Br per warp=16
3069     const int kValTileSeqLenK,    // value tiles along K's N, 8 → Bc_warp=8*8=64
3070     const int kValTileSeqLenP,    // value tiles for P@V M dim, 1
3071     const int kValTileHeadDimV,   // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
3072     const int kStage,             // pipeline stages for K: 1 or 2
3073     const int kPad>              // padding for bank conflict avoidance
3074 __global__ void __launch_bounds__(WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK)
3075     flash_attn_mma_stages_split_q(half *Q, half *K, half *V, half *O,
3076                                     int QKV_seqlen, int QKV_head) {
3077     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
3078     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
3079     constexpr int kNumThreads = WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
3080     const int Tc = (QKV_seqlen + Bc - 1) / Bc;
3081     // 原始实现默认 seqlen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
3082     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
3083     const float scale = 1.0f / sqrtf((float)kHeadDim);
3084
3085     // Block indexing
3086     const int QKV_batch_id = blockIdx.y / QKV_head;

```

```

3087 const int QKV_head_id = blockIdx.y % QKV_head;
3088 const int Q_tile_id = blockIdx.x;
3089 const int tid = threadIdx.x;
3090 const int warp_id = tid / WARP_SIZE;
3091 const int lane_id = tid % WARP_SIZE;
3092 const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
3093 const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
3094
3095 // Global memory base offsets for this (batch, head)
3096 // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
3097 const int Q_gmem_offset =
3098     (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
3099     kHeadDim;
3100 const int K_gmem_offset = Q_gmem_offset;
3101 const int V_gmem_offset = Q_gmem_offset;
3102 const int O_gmem_offset = Q_gmem_offset;
3103
3104 // Thread-to-smem mapping for cooperative load
3105 int load_smem_Q_Br = tid / (kNumThreads / Br);
3106 int load_smem_Q_d =
3107     (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
3108 int load_smem_K_Bc = tid / (kNumThreads / Bc);
3109 int load_smem_K_d =
3110     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3111 int load_smem_V_Bc = tid / (kNumThreads / Bc);
3112 int load_smem_V_d =
3113     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3114
3115 int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
3116 if (load_gmem_Q_Br >= QKV_seqlen)
3117     return;
3118
3119 // ---- Shared memory layout ----
3120 extern __shared__ half smem[];
3121 constexpr int Q_tile_size = Br * (kHeadDim + kPad); // Q tile: [Br, d+kPad]
3122 constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
3123 half *Q_tile_smem = smem;
3124 half *K_tile_smem = Q_tile_smem + Q_tile_size;
3125 half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage copies of K
3126 // 原始 kernel 还留了一个优化点: 若 kStage=1, K 和 V 在时序上并不重叠,
3127 // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。
3128
3129 uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
3130 uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
3131 uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
3132
3133 // ---- Online Softmax persistent state ----
3134 // lane_block_row_max_old[i][r]: running max for row r of warp tile i
3135 // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
3136 float lane_block_row_max_old[kValTileSeqLenQ][2];
3137 float lane_block_row_sum_old[kValTileSeqLenQ][2];
3138 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
3139 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
3140
3141 // ---- Register allocation ----
3142 uint32_t R_Q[kValTileSeqLenQ][4]; // Q regs
3143 uint32_t R_K[kValTileSeqLenK][2]; // K regs
3144 uint32_t R_V[kValTileHeadDimV][2]; // V regs
3145 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
3146 // 对单个 m16n8k16 tile 而言:
3147 // - reg[0] 持有该 tile 前 8 行里的两个 half 值
3148 // - reg[1] 持有该 tile 后 8 行里的两个 half 值
3149 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。

```



```

3150 uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
3151 uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][2]; // 0 for current tile
3152 uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV]
3153         [2]; // 0 accumulator (final output)
3154
3155 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
3156 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_D, 0);
3157
3158 // =====
3159 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
3160 // =====
3161 {
3162     int load_gmem_Q_addr =
3163         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
3164     uint32_t load_smem_Q_ptr =
3165         smem_Q_base_ptr +
3166         (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(half);
3167 #pragma unroll
3168     for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
3169         CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
3170     }
3171     CP_ASYNC_COMMIT_GROUP();
3172 }
3173
3174 // =====
3175 // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
3176 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqlen 遍历始终从 tile 0 开始,
3177 // 后续在外循环里不断递增到 tile 1/2/3/.../Tc-1。
3178 // =====
3179 if constexpr (kStage > 1) {
3180 #pragma unroll
3181     for (int stage = 0; stage < (kStage - 1); ++stage) {
3182         int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
3183         int load_gmem_K_addr =
3184             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
3185         uint32_t load_smem_K_ptr =
3186             smem_K_base_ptr +
3187             (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
3188              load_smem_K_d) *
3189             sizeof(half);
3190 #pragma unroll
3191         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3192             CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
3193         }
3194         CP_ASYNC_COMMIT_GROUP();
3195     }
3196     CP_ASYNC_WAIT_GROUP(kStage - 2);
3197     __syncthreads();
3198 }
3199
3200 // =====
3201 // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
3202 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
3203 // =====
3204 #pragma unroll 1
3205 for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
3206     int smem_sel = tile_K_seqlen % kStage;
3207     int smem_sel_next = (tile_K_seqlen + (kStage - 1)) % kStage;
3208
3209     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
3210     if constexpr (kStage > 1) {
3211         // 只有 kStage>1 才能真正做 K 的 pipeline:
3212         // smem_sel 负责 “当前正在计算” 的 K tile, smem_sel_next 负责 “下一轮预取” 的 K tile。

```



```

3213 // 若 kStage=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖同一块 smem。
3214 // Load current V tile (no pipeline for V — one stage is enough)
3215 {
3216     int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
3217     int load_gmem_V_addr =
3218         V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
3219     uint32_t load_smem_V_ptr =
3220         smem_V_base_ptr +
3221         (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof(half);
3222 #pragma unroll
3223     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3224         CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
3225     }
3226     CP_ASYNC_COMMIT_GROUP();
3227 }
3228
3229 // Prefetch next K tile (pipelined)
3230 if ((tile_K_seqlen + 1) < Tc) {
3231     int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
3232     int load_gmem_K_addr =
3233         K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
3234     uint32_t load_smem_K_ptr =
3235         smem_K_base_ptr +
3236         (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
3237          load_smem_K_d) *
3238         sizeof(half);
3239 #pragma unroll
3240     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3241         CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
3242     }
3243     CP_ASYNC_COMMIT_GROUP();
3244 }
3245 }
3246
3247 // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
3248 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
3249 #pragma unroll
3250 for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
3251     // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
3252     #pragma unroll
3253     for (int i = 0; i < kValTileSeqLenQ; ++i) {
3254         int warp_smem_Q_Br =
3255             warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
3256         int lane_smem_Q_Br =
3257             warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
3258         int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
3259         uint32_t lane_smem_Q_ptr =
3260             smem_Q_base_ptr +
3261             (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
3262         LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
3263                    lane_smem_Q_ptr);
3264     }
3265
3266     // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
3267     // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
3268     #pragma unroll
3269     for (int j = 0; j < kValTileSeqLenK; ++j) {
3270         int warp_smem_K_Bc =
3271             warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
3272         int lane_smem_K_Bc =
3273             warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
3274         int lane_smem_K_d =
3275             tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8

```

```

3276     uint32_t lane_smem_K_ptr =
3277         smem_K_base_ptr +
3278         (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +
3279         lane_smem_K_d) *
3280         sizeof(half);
3281     LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
3282 }
3283
3284 // MMA: S[tile] += Q[tile] @ K^T[tile]
3285 #pragma unroll
3286 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3287     #pragma unroll
3288     for (int j = 0; j < kValTileSeqLenK; ++j) {
3289         HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
3290                 R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
3291                 R_S[i][j][1]);
3292     }
3293 }
3294 } // end loop over d
3295 __syncthreads();
3296
3297 // =====
3298 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
3299 // =====
3300 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
3301 //   - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
3302 //   - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
3303 //   - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
3304 //   - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
3305 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
3306 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
3307 // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
3308 // kValTileSeqLenK tiles.
3309
3310 float lane_row_max_new[kValTileSeqLenQ][2];
3311 float lane_row_sum_new[kValTileSeqLenQ][2];
3312 fill_2D_regs<float>, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
3313 fill_2D_regs<float>, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
3314
3315 // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
3316 #pragma unroll
3317 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3318     #pragma unroll
3319     for (int j = 0; j < kValTileSeqLenK; ++j) {
3320         // Extract half values from R_S registers (C matrix fragment layout)
3321         float2 t_reg_S_0 =
3322             __half22float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}
3323         float2 t_reg_S_1 =
3324             __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
3325         // S = (Q@K^T) * scale
3326         float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
3327         float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
3328         lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
3329         lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
3330     }
3331     // Warp-level reduce max (warp_size = 4 for Q@K^T — only lanes
3332     // {0,4,8,...,28} hold valid data)
3333     lane_row_max_new[i][0] =
3334         warp_reduce_max<4, float>(lane_row_max_new[i][0]);
3335     lane_row_max_new[i][1] =
3336         warp_reduce_max<4, float>(lane_row_max_new[i][1]);
3337 }
3338

```

```

3339 // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
3340 // 面试关键点: 这里将 P 写回 R_S 寄存器!
3341 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
3342 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
3343 #pragma unroll
3344 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3345     // m_new = max(m_old, m_cur)
3346     float block_row_max_new_0 =
3347         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
3348     float block_row_max_new_1 =
3349         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
3350
3351 #pragma unroll
3352 for (int j = 0; j < kValTileSeqLenK; ++j) {
3353     float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
3354     float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
3355
3356     // P = exp(S * scale - m_new), 用 fma 保证精度
3357     t_reg_S_0.x =
3358         __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
3359     t_reg_S_0.y =
3360         __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
3361     t_reg_S_1.x =
3362         __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
3363     t_reg_S_1.y =
3364         __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
3365
3366     // Accumulate row sums
3367     lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
3368     lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
3369
3370     // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
3371     HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
3372     HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
3373 }
3374
3375 // Warp-level reduce sum (warp_size = 4, same as max)
3376 lane_row_sum_new[i][0] =
3377     warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
3378 lane_row_sum_new[i][1] =
3379     warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
3380 }
3381 __syncthreads();
3382
3383 // =====
3384 // 3d: P@V — P[Br,Bc] @ V[Bc,d] = O[Br,d]
3385 // =====
3386 // Wait for V to be ready before computing P@V
3387 if constexpr (kStage > 1) {
3388     if ((tile_K_seqLen + 1) < Tc) {
3389         CP_ASYNC_WAIT_GROUP(1);
3390     } else {
3391         CP_ASYNC_WAIT_GROUP(0);
3392     }
3393 } else {
3394     CP_ASYNC_WAIT_GROUP(0);
3395 }
3396 __syncthreads();
3397
3398 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_0, 0);
3399
3400 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
3401 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations

```

```

3402 #pragma unroll
3403 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
3404     // ldmatrix.x2.trans: load V[Bc,d] with transposition
3405     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
3406     // transposed ldmatrix
3407 #pragma unroll
3408 for (int j = 0; j < kValTileHeadDimV; ++j) {
3409     int warp_smem_V_d =
3410         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
3411     int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
3412     int lane_smem_V_d = warp_smem_V_d;
3413     uint32_t lane_smem_V_ptr =
3414         smem_V_base_ptr +
3415         (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(half);
3416     LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
3417 }
3418
3419 // P matrix layout in R_S[i][j][2]:
3420 // MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
3421 // | 64x64 | warp_KV 0 |
3422 // | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
3423 // | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
3424 // | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
3425 // | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
3426 // tile_V_Bc selects which 16-column slice of P to use:
3427 // tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
3428 // tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
3429 // tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
3430 // tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
3431 // 对应的 MMA A fragment 布局可以这样记:
3432 // - rows 0~7: lane 0~3 → {a0,a1} 与 {a4,a5}, lane 4~7 → 下一行, 依次类推
3433 // - rows 8~15: lane 0~3 → {a2,a3} 与 {a6,a7}, lane 4~7 → 下一行, 依次类推
3434 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
3435 // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
3436 // fragment 都无条件成立的通用结论。layout转换逻辑:
3437 // C layout: 8 x (16 x 8) layout → A layout: 4 x (16 x 16) layout
3438 int w = tile_V_Bc * 2;
3439 #pragma unroll
3440 for (int i = 0; i < kValTileSeqLenP; ++i) {
3441 #pragma unroll
3442 for (int j = 0; j < kValTileHeadDimV; ++j) {
3443     HMMA16816(R_O[i][j][0], R_O[i][j][1], // C fragment output
3444         R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1], // A fragment = P
3445         R_V[j][0], R_V[j][1], // B fragment = V
3446         R_O[i][j][0], R_O[i][j][1]); // C fragment output
3447     }
3448 }
3449 } // end for tile_V_Bc
3450 __syncthreads();
3451
3452 // =====
3453 // 3e: Online rescaling — 0_new = exp(m_old - m_new) * 0_old + P@V
3454 // =====
3455 // 公式来源: FA2 paper Eq.(7-8), 使用 exp(m_old - m_new) 做 0 与 l 的 rescale
3456 #pragma unroll
3457 for (int i = 0; i < kValTileSeqLenP; ++i) {
3458     float block_row_max_new_0 = lane_row_max_new[i][0];
3459     float block_row_max_new_1 = lane_row_max_new[i][1];
3460     float block_row_sum_new_0 = lane_row_sum_new[i][0];
3461     float block_row_sum_new_1 = lane_row_sum_new[i][1];
3462
3463     float block_row_max_old_0 = lane_block_row_max_old[i][0];
3464     float block_row_max_old_1 = lane_block_row_max_old[i][1];

```

```

3465
3466     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
3467     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
3468
3469     // Handle first iteration: m_old = -inf, need to use m_new directly
3470     block_row_max_old_0 =
3471         (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
3472     block_row_max_old_1 =
3473         (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);
3474
3475     float rescale_o_factor_0 =
3476         __expf(block_row_max_old_0 - block_row_max_new_0);
3477     float rescale_o_factor_1 =
3478         __expf(block_row_max_old_1 - block_row_max_new_1);
3479
3480     // Rescale O_old + Add P@V in one fused step
3481     #pragma unroll
3482     for (int j = 0; j < kValTileHeadDimV; ++j) {
3483         // R_0 / R_D 与前面的 R_S 一样, 都按 MMA C fragment 布局解释:
3484         //   reg[0] -> rows 0~7 的 {c0,c1}
3485         //   reg[1] -> rows 8~15 的 {c2,c3}
3486         float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
3487         float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
3488         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
3489         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
3490
3491         // O_new = exp(m_old - m_new) * O_old + P@V (fused multiply-add)
3492         t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
3493         t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
3494         t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
3495         t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
3496
3497         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
3498         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
3499     }
3500
3501     // Update l: l_new = exp(m_old - m_new) * l_old + row_sum(P)
3502     float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
3503     float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
3504     lane_block_row_sum_old[i][0] = __fmaf_rn(
3505         rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
3506     lane_block_row_sum_old[i][1] = __fmaf_rn(
3507         rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
3508
3509     // Update m
3510     lane_block_row_max_old[i][0] = block_row_max_new_0;
3511     lane_block_row_max_old[i][1] = block_row_max_new_1;
3512 }
3513
3514 // Wait for next K tile to be ready in smem before next iteration
3515 if constexpr (kStage > 1) {
3516     if ((tile_K_seqlen + 1) < Tc) {
3517         CP_ASYNC_WAIT_GROUP(0);
3518     }
3519     __syncthreads();
3520 }
3521 } // end outer loop over K seqlen
3522 __syncthreads();
3523
3524 // =====
3525 // Step 4: 最终 rescale — O_final = (1/l_final) * O_final
3526 // =====
3527 #pragma unroll

```

```

3528     for (int i = 0; i < kValTileSeqLenP; ++i) {
3529         float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
3530         float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
3531     #pragma unroll
3532     for (int j = 0; j < kValTileHeadDimV; ++j) {
3533         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
3534         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
3535         t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
3536         t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
3537         t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
3538         t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
3539         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
3540         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
3541     }
3542 }
3543
3544 // =====
3545 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
3546 // =====
3547 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
3548 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
3549 #pragma unroll
3550 for (int i = 0; i < kValTileSeqLenP; ++i) {
3551     #pragma unroll
3552     for (int j = 0; j < kValTileHeadDimV; ++j) {
3553         uint32_t R_Z[2][4];
3554         R_Z[0][0] = R_D[i][j][0];
3555         R_Z[1][0] = R_D[i][j][1];
3556         R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
3557         R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
3558         R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
3559         R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
3560         R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
3561         R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
3562
3563         // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
3564         if (lane_id % 4 == 0) {
3565             int store_warp_regs_0_Br =
3566                 warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
3567             int store_lane_gmem_0_Br =
3568                 Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
3569             int store_warp_regs_0_d =
3570                 warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
3571             int store_lane_gmem_0_d = store_warp_regs_0_d;
3572             int store_gmem_0_addr_0 = 0_gmem_offset +
3573                 (store_lane_gmem_0_Br + 0) * kHeadDim +
3574                 store_lane_gmem_0_d;
3575             int store_gmem_0_addr_1 = 0_gmem_offset +
3576                 (store_lane_gmem_0_Br + 8) * kHeadDim +
3577                 store_lane_gmem_0_d;
3578             // LDST128BITS = reinterpret_cast<float4*>
3579             *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
3580                 *reinterpret_cast<float4*>(&R_Z[0][0]);
3581             *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
3582                 *reinterpret_cast<float4*>(&R_Z[1][0]);
3583         }
3584     }
3585 }
3586 }
3587
3588 // =====
3589 // 以下是测试代码, 验证 Phase 1 - Phase 8 的kernel的正确性, 不评估性能。
3590 // =====

```



```

3591
3592 static inline void check(cudaError_t err, const char *msg) {
3593     if (err != cudaSuccess) {
3594         fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
3595         exit(EXIT_FAILURE);
3596     }
3597 }
3598
3599
3600 static void test_block_reduce(int N) {
3601
3602     srand(42);
3603     float *h_a = (float *)malloc((size_t)N * sizeof(float));
3604     for (int i = 0; i < N; i++)
3605         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3606
3607     // CPU reference: sum of all elements
3608     double ref = 0.0;
3609     for (int i = 0; i < N; i++) ref += (double)h_a[i];
3610
3611     float *d_a, *d_y;
3612     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
3613     check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
3614
3615     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
3616     check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
3617
3618     dim3 block(128);
3619     dim3 grid((N + 127) / 128);
3620     block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
3621     check(cudaGetLastError(), "blockreduce launch");
3622     check(cudaDeviceSynchronize(), "blockreduce sync");
3623
3624     float result;
3625     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
3626
3627     float err = fabsf(result - (float)ref);
3628     printf("| %-35s | %.6e | %-4s |\n", "BlockReduce", err,
3629         err < 1e-2f ? "PASS" : "FAIL");
3630
3631     free(h_a);
3632     cudaFree(d_a); cudaFree(d_y);
3633 }
3634
3635
3636 static void test_dot(int N) {
3637
3638     srand(42);
3639     float *h_a = (float *)malloc((size_t)N * sizeof(float));
3640     float *h_b = (float *)malloc((size_t)N * sizeof(float));
3641     for (int i = 0; i < N; i++) {
3642         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3643         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3644     }
3645
3646     // CPU reference
3647     double ref = 0.0;
3648     for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
3649
3650     float *d_a, *d_b, *d_y;
3651     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
3652     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
3653     check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");

```

```

3654
3655 check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
3656 check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
3657 check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
3658
3659 dim3 block(128);
3660 dim3 grid((N + 127) / 128);
3661 dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
3662 check(cudaGetLastError(), "dot launch");
3663 check(cudaDeviceSynchronize(), "dot sync");
3664
3665 float result;
3666 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
3667
3668 float err = fabsf(result - (float)ref);
3669 printf("| %-35s | %.6e | %-4s |\n", "Dot", err,
3670     err < 1e-2f ? "PASS" : "FAIL");
3671
3672 // ---- Dot Vec4 ----
3673 check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
3674 dim3 block_v4(32);
3675 dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
3676 check(cudaGetLastError(), "dot_vec4 launch");
3677 check(cudaDeviceSynchronize(), "dot_vec4 sync");
3678
3679 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
3680 float err_v4 = fabsf(result - (float)ref);
3681 printf("| %-35s | %.6e | %-4s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL");
3682
3683 free(h_a); free(h_b);
3684 cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
3685 }
3686
3687
3688 static void test_relu(int N) {
3689     srand(42);
3690     float *h_x = (float *)malloc((size_t)N * sizeof(float));
3691     float *h_y = (float *)malloc((size_t)N * sizeof(float));
3692     for (int i = 0; i < N; i++)
3693         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3694
3695     float *d_x, *d_y;
3696     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
3697     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
3698     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
3699
3700     for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
3701     dim3 block256(256);
3702     dim3 grid256((N + 255) / 256);
3703     relu<<<grid256, block256>>>(d_x, d_y, N);
3704     check(cudaGetLastError(), "relu launch");
3705     check(cudaDeviceSynchronize(), "relu sync");
3706     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
3707     float max_err = 0.0f;
3708     for (int i = 0; i < N; i++) {
3709         float expected = fmaxf(0.0f, h_x[i]);
3710         float err = fabsf(h_y[i] - expected);
3711         if (err > max_err) max_err = err;
3712     }
3713     printf("| %-35s | %.6e | %-4s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3714
3715     dim3 block64(64);
3716     relu_vec4<<<grid256, block64>>>(d_x, d_y, N);

```

```

3717 check(cudaGetLastError(), "relu_vec4 launch");
3718 check(cudaDeviceSynchronize(), "relu_vec4 sync");
3719 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
3720 max_err = 0.0f;
3721 for (int i = 0; i < N; i++) {
3722     float expected = fmaxf(0.0f, h_x[i]);
3723     float err = fabsf(h_y[i] - expected);
3724     if (err > max_err) max_err = err;
3725 }
3726 printf("| %-35s | %.6e | %-4s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3727
3728 free(h_x); free(h_y);
3729 cudaFree(d_x); cudaFree(d_y);
3730 }
3731
3732
3733 static void test_elementwise(int N) {
3734     srand(42);
3735     float *h_a = (float *)malloc((size_t)N * sizeof(float));
3736     float *h_b = (float *)malloc((size_t)N * sizeof(float));
3737     for (int i = 0; i < N; i++) {
3738         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3739         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3740     }
3741
3742     float *d_a, *d_b, *d_c;
3743     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
3744     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
3745     check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
3746     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
3747     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
3748
3749     dim3 block256(256);
3750     dim3 grid256((N + 255) / 256);
3751     elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
3752     check(cudaGetLastError(), "eadd launch");
3753     check(cudaDeviceSynchronize(), "eadd sync");
3754     float *h_c = (float *)malloc((size_t)N * sizeof(float));
3755     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
3756     float max_err = 0.0f;
3757     for (int i = 0; i < N; i++) {
3758         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
3759         if (err > max_err) max_err = err;
3760     }
3761     printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3762
3763     dim3 block64(64);
3764     check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
3765     elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
3766     check(cudaGetLastError(), "eadd_vec4 launch");
3767     check(cudaDeviceSynchronize(), "eadd_vec4 sync");
3768     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
3769     max_err = 0.0f;
3770     for (int i = 0; i < N; i++) {
3771         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
3772         if (err > max_err) max_err = err;
3773     }
3774     printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3775
3776     free(h_a); free(h_b); free(h_c);
3777     cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
3778 }
3779

```

```

3780
3781 static void test_histogram(int N) {
3782     srand(42);
3783     int BINS = 16;
3784     int *h_hist = (int *)calloc(BINS, sizeof(int));
3785     int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
3786     int *h_idx = (int *)malloc((size_t)N * sizeof(int));
3787     for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
3788     for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
3789
3790     int *d_idx, *d_hist;
3791     check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
3792     check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
3793     check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
3794     check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
3795
3796     dim3 block256(256);
3797     dim3 grid256((N + 255) / 256);
3798     histogram<<<grid256, block256>>>(d_idx, d_hist, N);
3799     check(cudaGetLastError(), "histogram launch");
3800     check(cudaDeviceSynchronize(), "histogram sync");
3801     check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
3802
3803     float max_err = 0.0f;
3804     for (int i = 0; i < BINS; i++) {
3805         float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
3806         if (err > max_err) max_err = err;
3807     }
3808     printf("| %-35s | %.6e | %-4s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3809
3810     free(h_hist); free(h_hist_ref); free(h_idx);
3811     cudaFree(d_idx); cudaFree(d_hist);
3812 }
3813
3814
3815 static void test_merge_attn_states(int num_tokens, int num_heads,
3816                                   int head_size) {
3817
3818     size_t out_size = (size_t)num_tokens * num_heads * head_size * sizeof(float);
3819     size_t lse_size = (size_t)num_heads * num_tokens * sizeof(float);
3820
3821     float *h_prefix_out = (float *)malloc(out_size);
3822     float *h_suffix_out = (float *)malloc(out_size);
3823     float *h_prefix_lse = (float *)malloc(lse_size);
3824     float *h_suffix_lse = (float *)malloc(lse_size);
3825     float *h_output_ref = (float *)malloc(out_size);
3826
3827     srand(42);
3828     for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
3829         h_prefix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3830         h_suffix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3831     }
3832     for (int i = 0; i < num_heads * num_tokens; i++) {
3833         h_prefix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
3834         h_suffix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
3835     }
3836
3837     // CPU reference: 与 kernel 完全相同的浮点计算
3838     for (int t = 0; t < num_tokens; t++) {
3839         for (int h = 0; h < num_heads; h++) {
3840             float p_lse = h_prefix_lse[h * num_tokens + t];
3841             float s_lse = h_suffix_lse[h * num_tokens + t];
3842             p_lse = isinf(p_lse) ? -INFINITY : p_lse;

```

```

3843     s_lse = isinf(s_lse) ? -INFINITY : s_lse;
3844
3845     float max_lse = fmaxf(p_lse, s_lse);
3846     p_lse -= max_lse;
3847     s_lse -= max_lse;
3848     float p_se = expf(p_lse);
3849     float s_se = expf(s_lse);
3850     float p_scale = p_se / (p_se + s_se);
3851     float s_scale = s_se / (p_se + s_se);
3852
3853     int head_off = t * num_heads * head_size + h * head_size;
3854     for (int d = 0; d < head_size; d++) {
3855         h_output_ref[head_off + d] =
3856             h_prefix_out[head_off + d] * p_scale +
3857             h_suffix_out[head_off + d] * s_scale;
3858     }
3859 }
3860 }
3861
3862 float *d_prefix_out, *d_suffix_out, *d_prefix_lse, *d_suffix_lse, *d_output;
3863 check(cudaMalloc(&d_prefix_out, out_size), "merge_attn alloc p_out");
3864 check(cudaMalloc(&d_suffix_out, out_size), "merge_attn alloc s_out");
3865 check(cudaMalloc(&d_prefix_lse, lse_size), "merge_attn alloc p_lse");
3866 check(cudaMalloc(&d_suffix_lse, lse_size), "merge_attn alloc s_lse");
3867 check(cudaMalloc(&d_output, out_size), "merge_attn alloc output");
3868
3869 check(cudaMemcpy(d_prefix_out, h_prefix_out, out_size,
3870                 cudaMemcpyHostToDevice),
3871        "merge_attn H2D p_out");
3872 check(cudaMemcpy(d_suffix_out, h_suffix_out, out_size,
3873                 cudaMemcpyHostToDevice),
3874        "merge_attn H2D s_out");
3875 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
3876                 cudaMemcpyHostToDevice),
3877        "merge_attn H2D p_lse");
3878 check(cudaMemcpy(d_suffix_lse, h_suffix_lse, lse_size,
3879                 cudaMemcpyHostToDevice),
3880        "merge_attn H2D s_lse");
3881
3882 int threads_per_head = head_size / 4;
3883 int total_threads = num_tokens * num_heads * threads_per_head;
3884 dim3 block(128);
3885 dim3 grid((total_threads + 127) / 128);
3886 merge_attn_states<<<grid, block>>>(
3887     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
3888     num_tokens, num_heads, head_size);
3889 check(cudaGetLastError(), "merge_attn launch");
3890 check(cudaDeviceSynchronize(), "merge_attn sync");
3891
3892 float *h_output = (float *)malloc(out_size);
3893 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
3894        "merge_attn D2H");
3895
3896 float max_err = 0.0f;
3897 for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
3898     float err = fabsf(h_output[i] - h_output_ref[i]);
3899     if (err > max_err) max_err = err;
3900 }
3901 printf("| %-35s | %.6e | %-4s |\n", "MergeAttnStates", max_err,
3902        max_err < 1e-4f ? "PASS" : "FAIL");
3903
3904 // 边界测试: +inf LSE → 权重退化为 0 (空 attention 段)
3905 h_prefix_lse[0] = INFINITY; // head 0, token 0 的 LSE = +inf

```

```

3906 h_suffix_lse[0] = 0.0f;
3907 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
3908                 cudaMemcpyHostToDevice),
3909        "merge_attn H2D inf lse");
3910 merge_attn_states<<<grid, block>>>{
3911     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
3912     num_tokens, num_heads, head_size);
3913 check(cudaGetLastError(), "merge_attn inf launch");
3914 check(cudaDeviceSynchronize(), "merge_attn inf sync");
3915 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
3916        "merge_attn inf D2H");
3917
3918 // token 0, head 0 的所有元素应等于 suffix_output (prefix 权重  $\alpha=0$ )
3919 float inf_err = 0.0f;
3920 for (int d = 0; d < head_size; d++) {
3921     float err = fabsf(h_output[d] - h_suffix_out[d]);
3922     if (err > inf_err) inf_err = err;
3923 }
3924 printf("| %-35s | %.6e | %-4s |\n", "MergeAttnStates-inf", inf_err,
3925        inf_err < 1e-4f ? "PASS" : "FAIL");
3926
3927 free(h_prefix_out);
3928 free(h_suffix_out);
3929 free(h_prefix_lse);
3930 free(h_suffix_lse);
3931 free(h_output_ref);
3932 free(h_output);
3933 cudaFree(d_prefix_out);
3934 cudaFree(d_suffix_out);
3935 cudaFree(d_prefix_lse);
3936 cudaFree(d_suffix_lse);
3937 cudaFree(d_output);
3938 }
3939
3940
3941 static void test_softmax(int N) {
3942     // Use N = blockDim.x so one block processes all elements as one token.
3943     constexpr int NUM_THREADS = 256;
3944     if (N != NUM_THREADS) {
3945         printf("  OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", NUM_THREADS, N);
3946         return;
3947     }
3948
3949     srand(42);
3950     float *h_x = (float *)malloc((size_t)N * sizeof(float));
3951     float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
3952     for (int i = 0; i < N; i++)
3953         h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]
3954
3955     // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
3956     float max_val = -FLT_MAX;
3957     for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
3958     double sum_exp = 0.0;
3959     for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
3960     for (int i = 0; i < N; i++)
3961         h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
3962
3963     float *d_x, *d_y;
3964     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
3965     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
3966
3967     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
3968

```

```

3969 dim3 block(256);
3970 dim3 grid(1); // one token covering all N elements
3971 online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
3972 check(cudaGetLastError(), "softmax launch");
3973 check(cudaDeviceSynchronize(), "softmax sync");
3974
3975 float *h_y = (float *)malloc((size_t)N * sizeof(float));
3976 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
3977
3978 float max_err = 0.0f;
3979 for (int i = 0; i < N; i++) {
3980     float err = fabsf(h_y[i] - h_y_ref[i]);
3981     if (err > max_err) max_err = err;
3982 }
3983 printf("| %-35s | %.6e | %-4s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3984
3985 // ---- Safe Softmax ----
3986 safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
3987 check(cudaGetLastError(), "safe_softmax launch");
3988 check(cudaDeviceSynchronize(), "safe_softmax sync");
3989 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
3990 max_err = 0.0f;
3991 for (int i = 0; i < N; i++) {
3992     float err = fabsf(h_y[i] - h_y_ref[i]);
3993     if (err > max_err) max_err = err;
3994 }
3995 printf("| %-35s | %.6e | %-4s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3996
3997 // ---- Naive Softmax ----
3998 softmax_per_token<<<grid, block>>>(d_x, d_y, N);
3999 check(cudaGetLastError(), "naive_softmax launch");
4000 check(cudaDeviceSynchronize(), "naive_softmax sync");
4001 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
4002 max_err = 0.0f;
4003 for (int i = 0; i < N; i++) {
4004     float err = fabsf(h_y[i] - h_y_ref[i]);
4005     if (err > max_err) max_err = err;
4006 }
4007 printf("| %-35s | %.6e | %-4s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4008
4009 free(h_x); free(h_y); free(h_y_ref);
4010 cudaFree(d_x); cudaFree(d_y);
4011 }
4012
4013
4014 static void test_rms_norm(int N, int K) {
4015
4016     srand(42);
4017     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4018     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4019     for (int i = 0; i < N * K; i++)
4020         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4021     float g = 1.5f; // gain
4022
4023     // CPU reference: y = (x / rms(x)) * g
4024     float epsilon = 1e-5f;
4025     for (int n = 0; n < N; n++) {
4026         double sum_sq = 0.0;
4027         for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
4028         float rms = sqrtf((float)sum_sq / (float)K + epsilon);
4029         for (int k = 0; k < K; k++)
4030             h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
4031     }

```



```

4032
4033 float *d_x, *d_y;
4034 check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
4035 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
4036 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
4037
4038 dim3 block(128);
4039 dim3 grid(N);
4040 rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
4041 check(cudaGetLastError(), "rmsnorm launch");
4042 check(cudaDeviceSynchronize(), "rmsnorm sync");
4043
4044 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
4045 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");
4046
4047 float max_err = 0.0f;
4048 for (int i = 0; i < N * K; i++) {
4049     float err = fabsf(h_y[i] - h_y_ref[i]);
4050     if (err > max_err) max_err = err;
4051 }
4052 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4053
4054 // ---- RMS Norm Vec4 ----
4055 dim3 block_rv4(32);
4056 rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
4057 check(cudaGetLastError(), "rmsnorm_vec4 launch");
4058 check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
4059 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
4060 max_err = 0.0f;
4061 for (int i = 0; i < N * K; i++) {
4062     float err = fabsf(h_y[i] - h_y_ref[i]);
4063     if (err > max_err) max_err = err;
4064 }
4065 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4066
4067 free(h_x); free(h_y); free(h_y_ref);
4068 cudaFree(d_x); cudaFree(d_y);
4069 }
4070
4071
4072 static void test_layer_norm(int N, int K) {
4073
4074     srand(42);
4075     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4076     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4077     for (int i = 0; i < N * K; i++)
4078         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4079     float g = 1.5f, b = 0.3f; // gain and bias
4080
4081     // CPU reference: y = ((x - mean) / std) * g + b
4082     float epsilon = 1e-5f;
4083     for (int n = 0; n < N; n++) {
4084         double sum = 0.0;
4085         for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
4086         float mean = (float)sum / (float)K;
4087         double sum_sq = 0.0;
4088         for (int k = 0; k < K; k++) {
4089             float diff = h_x[n * K + k] - mean;
4090             sum_sq += (double)diff * (double)diff;
4091         }
4092         float std = sqrtf((float)sum_sq / (float)K + epsilon);
4093         for (int k = 0; k < K; k++)
4094             h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;

```

```

4095 }
4096
4097 float *d_x, *d_y;
4098 check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
4099 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
4100 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");
4101
4102 dim3 block(128);
4103 dim3 grid(N);
4104 layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
4105 check(cudaGetLastError(), "layernorm launch");
4106 check(cudaDeviceSynchronize(), "layernorm sync");
4107
4108 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
4109 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
4110
4111 float max_err = 0.0f;
4112 for (int i = 0; i < N * K; i++) {
4113     float err = fabsf(h_y[i] - h_y_ref[i]);
4114     if (err > max_err) max_err = err;
4115 }
4116 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4117
4118 // ---- Layer Norm Vec4 ----
4119 dim3 block_lv4(32);
4120 layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
4121 check(cudaGetLastError(), "layernorm_vec4 launch");
4122 check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
4123 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
4124 max_err = 0.0f;
4125 for (int i = 0; i < N * K; i++) {
4126     float err = fabsf(h_y[i] - h_y_ref[i]);
4127     if (err > max_err) max_err = err;
4128 }
4129 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4130
4131 free(h_x); free(h_y); free(h_y_ref);
4132 cudaFree(d_x); cudaFree(d_y);
4133 }
4134
4135
4136 static void test_rope(int seq_len, int N) {
4137
4138     int total_pairs = seq_len * N;
4139     int total_elems = total_pairs * 2;
4140     size_t size = (size_t)total_elems * sizeof(float);
4141
4142     float *h_x = (float *)malloc(size);
4143     float *h_y_ref = (float *)malloc(size);
4144
4145     srand(42);
4146     for (int i = 0; i < total_elems; i++)
4147         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4148
4149     // CPU reference: 2D rotation for each pair
4150     for (int idx = 0; idx < total_pairs; idx++) {
4151         int token_pos = idx / N;
4152         int token_idx = idx % N;
4153         float x1 = h_x[idx * 2];
4154         float x2 = h_x[idx * 2 + 1];
4155         float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
4156         float angle = (float)token_pos * theta;
4157         float cos_v = cosf(angle);

```

```

4158     float sin_v = sinf(angle);
4159     h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
4160     h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
4161 }
4162
4163 float *d_x, *d_y;
4164 check(cudaMalloc(&d_x, size), "rope alloc X");
4165 check(cudaMalloc(&d_y, size), "rope alloc Y");
4166 check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
4167
4168 dim3 block(256);
4169 dim3 grid((total_pairs + 255) / 256);
4170 rope<<<grid, block>>>(d_x, d_y, seq_len, N);
4171 check(cudaGetLastError(), "rope launch");
4172 check(cudaDeviceSynchronize(), "rope sync");
4173
4174 float *h_y = (float *)malloc(size);
4175 check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
4176
4177 float max_err = 0.0f;
4178 for (int i = 0; i < total_elems; i++) {
4179     float err = fabsf(h_y[i] - h_y_ref[i]);
4180     if (err > max_err) max_err = err;
4181 }
4182 printf("| %-35s | %.6e | %-4s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4183
4184 free(h_x);
4185 free(h_y);
4186 free(h_y_ref);
4187 cudaFree(d_x);
4188 cudaFree(d_y);
4189 }
4190
4191 static void test_mat_transpose(int row, int col) {
4192
4193     size_t size_in = (size_t)row * col * sizeof(float);
4194     size_t size_out = (size_t)col * row * sizeof(float);
4195
4196     float *h_x = (float *)malloc(size_in);
4197     float *h_y_ref = (float *)malloc(size_out);
4198
4199     srand(42);
4200     for (int i = 0; i < row * col; i++)
4201         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4202
4203     // CPU reference: y[j][i] = x[i][j] (row-major)
4204     for (int i = 0; i < row; i++)
4205         for (int j = 0; j < col; j++)
4206             h_y_ref[j * row + i] = h_x[i * col + j];
4207
4208     float *d_x, *d_y;
4209     check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
4210     check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
4211     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
4212
4213     dim3 block(16, 16);
4214     dim3 grid((col + 15) / 16, (row + 15) / 16);
4215     mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
4216     check(cudaGetLastError(), "mattrans launch");
4217     check(cudaDeviceSynchronize(), "mattrans sync");
4218
4219     float *h_y = (float *)malloc(size_out);
4220

```

```

4221 check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");
4222
4223 float max_err = 0.0f;
4224 for (int i = 0; i < col * row; i++) {
4225     float err = fabsf(h_y[i] - h_y_ref[i]);
4226     if (err > max_err) max_err = err;
4227 }
4228 printf("| %-35s | %.6e | %-4s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
4229
4230 free(h_x);
4231 free(h_y);
4232 free(h_y_ref);
4233 cudaFree(d_x);
4234 cudaFree(d_y);
4235 }
4236
4237
4238 static void test_mat_transpose_padded(int row, int col) {
4239
4240     size_t size_in = (size_t)row * col * sizeof(float);
4241     size_t size_out = (size_t)col * row * sizeof(float);
4242
4243     float *h_x = (float *)malloc(size_in);
4244     float *h_y_ref = (float *)malloc(size_out);
4245
4246     srand(42);
4247     for (int i = 0; i < row * col; i++)
4248         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4249
4250     // CPU reference: y[j][i] = x[i][j] (row-major)
4251     for (int i = 0; i < row; i++)
4252         for (int j = 0; j < col; j++)
4253             h_y_ref[j * row + i] = h_x[i * col + j];
4254
4255     float *d_x, *d_y;
4256     check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
4257     check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
4258     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
4259
4260     dim3 block(16, 16);
4261     dim3 grid((col + 15) / 16, (row + 63) / 64);
4262     mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
4263     check(cudaGetLastError(), "mattrans_padded launch");
4264     check(cudaDeviceSynchronize(), "mattrans_padded sync");
4265
4266     float *h_y = (float *)malloc(size_out);
4267     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
4268
4269     float max_err = 0.0f;
4270     for (int i = 0; i < col * row; i++) {
4271         float err = fabsf(h_y[i] - h_y_ref[i]);
4272         if (err > max_err) max_err = err;
4273     }
4274     printf("| %-35s | %.6e | %-4s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
4275
4276     free(h_x);
4277     free(h_y);
4278     free(h_y_ref);
4279     cudaFree(d_x);
4280     cudaFree(d_y);
4281 }
4282
4283

```

```

4284 static void test_sgemv(int M, int K) {
4285
4286     srand(42);
4287     float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
4288     float *h_x = (float *)malloc((size_t)K * sizeof(float));
4289     float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
4290     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4291     for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4292
4293     // CPU reference: y[m] = sum_k A[m,k] * x[k]
4294     for (int m = 0; m < M; m++) {
4295         double sum = 0.0;
4296         for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
4297         h_y_ref[m] = (float)sum;
4298     }
4299
4300     float *d_a, *d_x, *d_y;
4301     check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
4302     check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
4303     check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
4304
4305     check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
4306     check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
4307
4308     dim3 block(32, 4);
4309     dim3 grid((M + 3) / 4);
4310     sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
4311     check(cudaGetLastError(), "sgemv launch");
4312     check(cudaDeviceSynchronize(), "sgemv sync");
4313
4314     float *h_y = (float *)malloc((size_t)M * sizeof(float));
4315     check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
4316
4317     float max_err = 0.0f;
4318     for (int m = 0; m < M; m++) {
4319         float err = fabsf(h_y[m] - h_y_ref[m]);
4320         if (err > max_err) max_err = err;
4321     }
4322     printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4323
4324     // ---- SGEMV K32 ----
4325     check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
4326     sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
4327     check(cudaGetLastError(), "sgemv_k32 launch");
4328     check(cudaDeviceSynchronize(), "sgemv_k32 sync");
4329
4330     check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");
4331
4332     max_err = 0.0f;
4333     for (int m = 0; m < M; m++) {
4334         float err = fabsf(h_y[m] - h_y_ref[m]);
4335         if (err > max_err) max_err = err;
4336     }
4337     printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4338
4339     // ---- SGEMV K16 ----
4340     free(h_a); free(h_x); free(h_y); free(h_y_ref);
4341     cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
4342
4343     int K16 = 16;
4344     h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
4345     h_x = (float *)malloc((size_t)K16 * sizeof(float));
4346     h_y_ref = (float *)malloc((size_t)M * sizeof(float));

```

```

4347 for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4348 for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4349
4350 for (int m = 0; m < M; m++) {
4351     double sum = 0.0;
4352     for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
4353     h_y_ref[m] = (float)sum;
4354 }
4355
4356 check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
4357 check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
4358 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
4359
4360 check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
4361 check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
4362
4363 dim3 grid_k16((M + 7) / 8);
4364 sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
4365 check(cudaGetLastError(), "sgemv_k16 launch");
4366 check(cudaDeviceSynchronize(), "sgemv_k16 sync");
4367
4368 h_y = (float *)malloc((size_t)M * sizeof(float));
4369 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
4370
4371 max_err = 0.0f;
4372 for (int m = 0; m < M; m++) {
4373     float err = fabsf(h_y[m] - h_y_ref[m]);
4374     if (err > max_err) max_err = err;
4375 }
4376 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4377
4378 free(h_a); free(h_x); free(h_y); free(h_y_ref);
4379 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
4380 }
4381
4382
4383 static void test_sgemm(int M, int N, int K) {
4384
4385     size_t size_a = (size_t)M * K * sizeof(float);
4386     size_t size_b = (size_t)K * N * sizeof(float);
4387     size_t size_c = (size_t)M * N * sizeof(float);
4388
4389     float *h_a = (float *)malloc(size_a);
4390     float *h_b = (float *)malloc(size_b);
4391     float *h_c_ref = (float *)malloc(size_c);
4392
4393     srand(42);
4394     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4395     for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4396
4397     float *d_a, *d_b, *d_c;
4398     check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
4399     check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
4400     check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
4401
4402     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
4403     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
4404
4405     // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
4406     cublasHandle_t handle;
4407     cublasCreate(&handle);
4408     float alpha = 1.0f, beta = 0.0f;
4409     cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,

```

```

4410         &alpha, d_b, N, d_a, K, &beta, d_c, N);
4411     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");
4412
4413     // Kernel
4414     cudaEvent_t start, stop;
4415     cudaEventCreate(&start);
4416     cudaEventCreate(&stop);
4417
4418     dim3 block(32, 32);
4419     dim3 grid((N + 31) / 32, (M + 31) / 32);
4420     sgemm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
4421     check(cudaGetLastError(), "sgemm launch");
4422     check(cudaDeviceSynchronize(), "sgemm sync");
4423
4424     float *h_c = (float *)malloc(size_c);
4425     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
4426
4427     // Verify
4428     float max_err = 0.0f;
4429     for (int i = 0; i < M * N; i++) {
4430         float err = fabsf(h_c[i] - h_c_ref[i]);
4431         if (err > max_err) max_err = err;
4432     }
4433     printf("| %-35s | %.6e | %-4s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4434
4435     // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
4436
4437     dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
4438     check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
4439     sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
4440     check(cudaGetLastError(), "sgemm_vec4 launch");
4441     check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
4442
4443     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");
4444
4445     max_err = 0.0f;
4446     for (int i = 0; i < M * N; i++) {
4447         float err = fabsf(h_c[i] - h_c_ref[i]);
4448         if (err > max_err) max_err = err;
4449     }
4450     printf("| %-35s | %.6e | %-4s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4451
4452     free(h_a); free(h_b); free(h_c); free(h_c_ref);
4453     cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
4454     cublasDestroy(handle);
4455     cudaEventDestroy(start);
4456     cudaEventDestroy(stop);
4457 }
4458
4459
4460 static void test_hgemm_mma(int M, int N, int K) {
4461
4462     size_t size_a = (size_t)M * K * sizeof(half);
4463     size_t size_b = (size_t)K * N * sizeof(half);
4464     size_t size_c = (size_t)M * N * sizeof(half);
4465
4466     half *h_a = (half *)malloc(size_a);
4467     half *h_b = (half *)malloc(size_b);
4468     half *h_c_ref = (half *)malloc(size_c);
4469
4470     srand(42);
4471     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4472     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);

```



```

4473 // Kernel expects B^T [N×K] row-major layout (TN layout convention).
4474 // We store h_b as B [K×N] row-major for cuBLAS, then create B^T for kernel.
4475 size_t size_b_t = (size_t)N * K * sizeof(half);
4476 half *h_b_t = (half *)malloc(size_b_t);
4477 for (int n = 0; n < N; n++)
4478     for (int k = 0; k < K; k++)
4479         h_b_t[n * K + k] = h_b[k * N + n];
4480
4481 half *d_a, *d_b, *d_b_t, *d_c;
4482 check(cudaMalloc(&d_a, size_a), "hgemm alloc A");
4483 check(cudaMalloc(&d_b, size_b), "hgemm alloc B (cuBLAS)");
4484 check(cudaMalloc(&d_b_t, size_b_t), "hgemm alloc B_t (kernel)");
4485 check(cudaMalloc(&d_c, size_c), "hgemm alloc C");
4486
4487 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm H2D A");
4488 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm H2D B (cuBLAS)");
4489 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");
4490
4491 // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
4492 // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
4493 cublasHandle_t handle;
4494 cublasCreate(&handle);
4495 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4496 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4497             &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
4498             &beta_h, d_c, CUDA_R_16F, N,
4499             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4500 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");
4501
4502 // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
4503 cudaEvent_t start, stop;
4504 cudaEventCreate(&start);
4505 cudaEventCreate(&stop);
4506
4507 constexpr int BM = 128, BN = 128, BK = 16, K_STAGE = 3;
4508 size_t smem_bytes = K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 24576
4509 dim3 block(256);
4510 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
4511 hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
4512 check(cudaGetLastError(), "hgemm launch");
4513 check(cudaDeviceSynchronize(), "hgemm sync");
4514
4515 half *h_c = (half *)malloc(size_c);
4516 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");
4517
4518 // Verify
4519 float max_err = 0.0f;
4520 for (int i = 0; i < M * N; i++) {
4521     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4522     if (err > max_err) max_err = err;
4523 }
4524 printf("| %-35s | %.6e | %-4s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL");
4525
4526 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
4527 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
4528 cublasDestroy(handle);
4529 cudaEventDestroy(start);
4530 cudaEventDestroy(stop);
4531 }
4532
4533 static void test_hgemm_swizzle(int M, int N, int K) {

```

```

4536 // HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
4537 // TN layout:  $C[M \times N] = A[M \times K] \times B^T[N \times K]$ 
4538 // Kernel: hgemm_mma_stages_tn_swizzle with default template params
4539
4540 size_t size_a = (size_t)M * K * sizeof(half);
4541 size_t size_b = (size_t)K * N * sizeof(half);
4542 size_t size_c = (size_t)M * N * sizeof(half);
4543
4544 half *h_a = (half *)malloc(size_a);
4545 half *h_b = (half *)malloc(size_b);
4546 half *h_c_ref = (half *)malloc(size_c);
4547
4548 srand(42);
4549 for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4550 for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4551
4552 // Kernel expects  $B^T [N \times K]$  row-major layout (TN layout convention).
4553 size_t size_b_t = (size_t)N * K * sizeof(half);
4554 half *h_b_t = (half *)malloc(size_b_t);
4555 for (int n = 0; n < N; n++)
4556     for (int k = 0; k < K; k++)
4557         h_b_t[n * K + k] = h_b[k * N + n];
4558
4559 half *d_a, *d_b, *d_b_t, *d_c;
4560 check(cudaMalloc(&d_a, size_a), "hgemm_swizzle alloc A");
4561 check(cudaMalloc(&d_b, size_b), "hgemm_swizzle alloc B (cuBLAS)");
4562 check(cudaMalloc(&d_b_t, size_b_t), "hgemm_swizzle alloc B_t (kernel)");
4563 check(cudaMalloc(&d_c, size_c), "hgemm_swizzle alloc C");
4564
4565 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_swizzle H2D A");
4566 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B (cuBLAS)");
4567 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B_t (kernel)");
4568
4569 // cuBLAS FP16 reference
4570 cublasHandle_t handle;
4571 cublasCreate(&handle);
4572 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4573 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4574             &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
4575             &beta_h, d_c, CUDA_R_16F, N,
4576             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4577 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H ref");
4578
4579 // MMA swizzle kernel (default params: K_STAGE=3)
4580 constexpr int BM = 128, BN = 128, BK = 16, K_STAGE_S = 3;
4581 size_t smem_bytes = K_STAGE_S * (BM * BK + BN * BK) * sizeof(half);
4582 dim3 block(256);
4583 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
4584 hgemm_mma_stages_tn_swizzle<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
4585 check(cudaGetLastError(), "hgemm_swizzle launch");
4586 check(cudaDeviceSynchronize(), "hgemm_swizzle sync");
4587
4588 half *h_c = (half *)malloc(size_c);
4589 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H");
4590
4591 // Verify
4592 float max_err = 0.0f;
4593 for (int i = 0; i < M * N; i++) {
4594     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4595     if (err > max_err) max_err = err;
4596 }
4597 printf("| %-35s | %.6e | %-4s |\n", "HGEMM MMA Swizzle", max_err, max_err < 1.0f ? "PASS" : "FAIL");
4598

```

```

4599 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
4600 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
4601 cublasDestroy(handle);
4602 }
4603
4604
4605 #if defined(NOTES_V2_ENABLE_CUTE)
4606 static void test_hgemm_cute(int M, int N, int K) {
4607     // HGEMM CuTe — SM80_16x8x16_F16F16F16F16_TN + Swizzle<3,3,3> + kStage=2
4608     // TN layout: C[M×N] = A[M×K] × BT[N×K]
4609     // Kernel: hgemm_mma_stages_tn_cute via launch_hgemm_mma_stages_tn_cute
4610     // Tile: BM=128, BN=256, BK=32, 128 threads/block
4611
4612     size_t size_a = (size_t)M * K * sizeof(half);
4613     size_t size_b = (size_t)K * N * sizeof(half);
4614     size_t size_c = (size_t)M * N * sizeof(half);
4615
4616     half *h_a = (half *)malloc(size_a);
4617     half *h_b = (half *)malloc(size_b);
4618     half *h_c_ref = (half *)malloc(size_c);
4619
4620     srand(42);
4621     for (int i = 0; i < M * K; i++)
4622         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4623     for (int i = 0; i < K * N; i++)
4624         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4625
4626     // CuTe kernel expects BT [N×K] row-major (TN layout).
4627     size_t size_b_t = (size_t)N * K * sizeof(half);
4628     half *h_b_t = (half *)malloc(size_b_t);
4629     for (int n = 0; n < N; n++)
4630         for (int k = 0; k < K; k++)
4631             h_b_t[n * K + k] = h_b[k * N + n];
4632
4633     half *d_a, *d_b, *d_b_t, *d_c;
4634     check(cudaMalloc(&d_a, size_a), "hgemm_cute alloc A");
4635     check(cudaMalloc(&d_b, size_b), "hgemm_cute alloc B (cuBLAS)");
4636     check(cudaMalloc(&d_b_t, size_b_t), "hgemm_cute alloc B_t (kernel)");
4637     check(cudaMalloc(&d_c, size_c), "hgemm_cute alloc C");
4638
4639     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_cute H2D A");
4640     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_cute H2D B (cuBLAS)");
4641     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_cute H2D B_t (kernel)");
4642
4643     // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
4644     cublasHandle_t handle;
4645     cublasCreate(&handle);
4646     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4647     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
4648                 CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
4649                 CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4650     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H ref");
4651
4652     // CuTe kernel (Stages=2, TN layout)
4653     launch_hgemm_mma_stages_tn_cute<half, 2>(d_a, d_b_t, d_c, M, N, K);
4654     check(cudaGetLastError(), "hgemm_cute launch");
4655     check(cudaDeviceSynchronize(), "hgemm_cute sync");
4656
4657     half *h_c = (half *)malloc(size_c);
4658     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H");
4659
4660     // Verify
4661     float max_err = 0.0f;

```

```

4662     for (int i = 0; i < M * N; i++) {
4663         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4664         if (err > max_err) max_err = err;
4665     }
4666     printf("| %-35s | %.6e | %-4s |\n", "HGEMM CuTe", max_err, max_err < 1.0f ? "PASS" : "FAIL");
4667
4668     free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
4669     cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
4670     cublasDestroy(handle);
4671 }
4672 #endif /* NOTES_V2_ENABLE_CUTE */
4673
4674
4675 #if defined(NOTES_V2_ENABLE_WGMMMA)
4676 static void test_hgemm_wgmma(int M, int N, int K) {
4677     // HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
4678     // TN layout: C[M×N] = A[M×K] × BT[N×K]
4679     // Kernel: hgemm_wgmma_stages_tn with default template params
4680
4681     constexpr int BM = 128, BN = 128, BK = 64, K_STAGE = 3, NUM_THREADS = 256;
4682
4683     // M, K must be divisible by tile dims
4684     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
4685         printf("| %-35s | %-12s | %-4s |\n",
4686             "HGEMM WGMMMA", "SKIP", "SKIP");
4687         return;
4688     }
4689
4690     size_t size_a = (size_t)M * K * sizeof(half);
4691     size_t size_b = (size_t)K * N * sizeof(half);
4692     size_t size_c = (size_t)M * N * sizeof(half);
4693
4694     half *h_a = (half *)malloc(size_a);
4695     half *h_b = (half *)malloc(size_b);
4696     half *h_c_ref = (half *)malloc(size_c);
4697
4698     srand(42);
4699     for (int i = 0; i < M * K; i++)
4700         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4701     for (int i = 0; i < K * N; i++)
4702         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4703
4704     // BT [N×K] row-major for TN layout (same as hgemm_mma kernel)
4705     size_t size_b_t = (size_t)N * K * sizeof(half);
4706     half *h_b_t = (half *)malloc(size_b_t);
4707     for (int n = 0; n < N; n++)
4708         for (int k = 0; k < K; k++)
4709             h_b_t[n * K + k] = h_b[k * N + n];
4710
4711     half *d_a, *d_b, *d_b_t, *d_c;
4712     check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
4713     check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
4714     check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
4715     check(cudaMalloc(&d_c, size_c), "wgmma alloc C");
4716
4717     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
4718     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
4719         "wgmma H2D B (cuBLAS)");
4720     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
4721         "wgmma H2D B_t (kernel)");
4722
4723     // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
4724     cublasHandle_t handle;

```

```

4725 cublasCreate(&handle);
4726 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4727 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
4728             CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
4729             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4730 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
4731       "wgmma D2H ref");
4732
4733 // Create TMA tensor maps for A and B^T
4734 // A[MxK] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
4735 // B^T[NxK] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
4736 CUTensorMap *tma_a =
4737     allocate_and_create_tensor_map(d_a, M / BM, K / BK);
4738 CUTensorMap *tma_b =
4739     allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
4740
4741 // Launch WGMMMA kernel
4742 // BLOCK_SWIZZLE=false → 2D grid, no swizzle
4743 size_t smem_bytes =
4744     K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
4745 cudaFuncSetAttribute(
4746     hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE,
4747     false>,
4748     cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
4749
4750 dim3 block(NUM_THREADS);
4751 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
4752 hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE, false>
4753     <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
4754 check(cudaGetLastError(), "wgmma launch");
4755 check(cudaDeviceSynchronize(), "wgmma sync");
4756
4757 half *h_c = (half *)malloc(size_c);
4758 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");
4759
4760 // Verify
4761 float max_err = 0.0f;
4762 for (int i = 0; i < M * N; i++) {
4763     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4764     if (err > max_err)
4765         max_err = err;
4766 }
4767 printf("| %-35s | %.6e | %-4s |\n", "HGEMM WGMMMA", max_err,
4768       max_err < 1.0f ? "PASS" : "FAIL");
4769
4770 free(h_a);
4771 free(h_b);
4772 free(h_b_t);
4773 free(h_c);
4774 free(h_c_ref);
4775 cudaFree(d_a);
4776 cudaFree(d_b);
4777 cudaFree(d_b_t);
4778 cudaFree(d_c);
4779 cudaFree(tma_a);
4780 cudaFree(tma_b);
4781 cublasDestroy(handle);
4782 }
4783 #endif /* NOTES_V2_ENABLE_WGMMMA */
4784
4785
4786 static void test_flash_attn(int seqlen, int head_dim) {
4787     // FlashAttention-2 with split-Q, MMA m16n8k16

```

```

4788 int B = 1, H = 8;
4789
4790 size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
4791
4792 srand(42);
4793 half *h_q = (half *)malloc(sz);
4794 half *h_k = (half *)malloc(sz);
4795 half *h_v = (half *)malloc(sz);
4796 for (int i = 0; i < B * H * seqlen * head_dim; i++) {
4797     h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4798     h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4799     h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4800 }
4801
4802 // CPU reference in FP32: 0 = softmax(Q @ K^T / sqrt(d)) @ V
4803 float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float
4804 float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
4805 float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
4806 float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
4807 int count = B * H * seqlen * head_dim;
4808 for (int i = 0; i < count; i++) {
4809     ref_q[i] = __half2float(h_q[i]);
4810     ref_k[i] = __half2float(h_k[i]);
4811     ref_v[i] = __half2float(h_v[i]);
4812 }
4813
4814 float scale = 1.0f / sqrtf((float)head_dim);
4815 for (int bi = 0; bi < B * H; bi++) {
4816     for (int qi = 0; qi < seqlen; qi++) {
4817         // S[qi, kj] = Q[qi, :] @ K[kj, :]^T * scale
4818         float smax = -INFINITY;
4819         float *S = (float *)malloc((size_t)seqlen * sizeof(float));
4820         for (int kj = 0; kj < seqlen; kj++) {
4821             float s = 0.0f;
4822             for (int d = 0; d < head_dim; d++)
4823                 s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
4824                     ref_k[bi * seqlen * head_dim + kj * head_dim + d];
4825             S[kj] = s * scale;
4826             if (S[kj] > smax) smax = S[kj];
4827         }
4828         // softmax
4829         double sum_exp = 0.0;
4830         for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
4831         float inv_sum = 1.0f / (float)sum_exp;
4832         // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
4833         for (int d = 0; d < head_dim; d++) {
4834             double o_acc = 0.0;
4835             for (int kj = 0; kj < seqlen; kj++)
4836                 o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
4837                     ref_v[bi * seqlen * head_dim + kj * head_dim + d];
4838             ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
4839         }
4840         free(S);
4841     }
4842 }
4843
4844 half *d_q, *d_k, *d_v, *d_o;
4845 check(cudaMalloc(&d_q, sz), "fa alloc Q");
4846 check(cudaMalloc(&d_k, sz), "fa alloc K");
4847 check(cudaMalloc(&d_v, sz), "fa alloc V");
4848 check(cudaMalloc(&d_o, sz), "fa alloc O");
4849 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
4850 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");

```

```

4851 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
4852
4853 // Template params for kHeadDim=64, kStage=2
4854 constexpr int kHeadDim = 64, kStageV = 2, kPadV = 8;
4855 constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
4856 constexpr int kMmaTileSeqLenQ = 4;
4857 constexpr int kMmaTileSeqLenK = 1;
4858 constexpr int kMmaTileSeqLenP = 4;
4859 constexpr int kMmaTileHeadDimV = 1;
4860 constexpr int kValTileSeqLenQ = 1;
4861 constexpr int kValTileSeqLenK = 8;
4862 constexpr int kValTileSeqLenP = 1;
4863 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
4864
4865 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
4866 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
4867 size_t smem_bytes = (Br * (kHeadDim + kPadV) +
4868                     kStageV * Bc * (kHeadDim + kPadV) +
4869                     Bc * (kHeadDim + kPadV)) * sizeof(half);
4870
4871 dim3 block(128);
4872 dim3 grid((seqlen + Br - 1) / Br, B * H);
4873
4874 flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK,
4875                               kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV,
4876                               kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV,
4877                               kStageV, kPadV>
4878     <<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
4879 check(cudaGetLastError(), "fa launch");
4880 check(cudaDeviceSynchronize(), "fa sync");
4881
4882 half *h_o = (half *)malloc(sz);
4883 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
4884
4885 float max_err = 0.0f;
4886 for (int i = 0; i < count; i++) {
4887     float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
4888     if (err > max_err) max_err = err;
4889 }
4890 printf("| %-35s | %.6e | %-4s |\n", "FlashAttn-SplitQ", max_err, max_err < 1e-1f ? "PASS" : "FAIL");
4891
4892 free(h_q); free(h_k); free(h_v); free(h_o);
4893 free(ref_q); free(ref_k); free(ref_v); free(ref_o);
4894 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
4895 }
4896
4897
4898 int main(int argc, char *argv[]) {
4899 #if defined(NOTES_V2_ENABLE_WGMMA)
4900     cuInit(0); // Driver API init required for cuTensorMapEncodeTiled (TMA, sm_90a+)
4901 #endif
4902     int M = 1024, N = 1024, K = 1024;
4903     if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
4904
4905     printf("=== notes-v2.cu verification harness ===\n");
4906     printf("| %-35s | %-12s | %-4s |\n", "Kernel", "Max Err", "Pass");
4907     printf("| -----|-----|-----|\n");
4908
4909     test_block_reduce(N);
4910     test_dot(N);
4911     test_relu(1024);
4912     test_elementwise(1024);
4913     test_histogram(1024);

```



```

4914     test_merge_attn_states(512, 16, 128);
4915     test_softmax(256);
4916     test_rms_norm(8, 128);
4917     test_layer_norm(8, 128);
4918     test_rope(8, 128);
4919     test_mat_transpose(256, 256);
4920     test_mat_transpose_padded(256, 256);
4921     test_sgemv(256, 128);
4922     test_sgemm(M, N, K);
4923     test_hgemm_mma(M, N, K);
4924     test_hgemm_swizzle(M, N, K);
4925     #if (defined(NOTES_V2_ENABLE_CUTE))
4926         test_hgemm_cute(M, N, K);
4927     #endif
4928     #if (defined(NOTES_V2_ENABLE_WGMMA))
4929         test_hgemm_wgmma(M, N, K);
4930     #endif
4931     test_flash_attn(1024, 64);
4932
4933     printf("=== All tests done ===\n");
4934     return 0;
4935 }
4936
4937 // =====
4938 // Quick build & run reference
4939 // =====
4940 // # sm_89 单独编译 + 运行:
4941 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
4942 //
4943 // # sm_89 + CuTe (需要 CUTLASS include 路径, 编译 CUDA_HOME 指向的 CUTLASS 或
4944 //   项目内置 third-party/cutlass) :
4945 // nvcc -std=c++20 -O2 -arch=sm_89 -DNOTES_V2_ENABLE_CUTE \
4946 //   -I ../../third-party/cutlass/include \
4947 //   -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm89.bin
4948 //
4949 // # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可) :
4950 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
4951 //   -DNOTES_V2_ENABLE_WGMMA -lcublas -lcuda notes-v2.cu -o notes_v2_sm90.bin
4952 //
4953 // # sm_90a + CuTe + WGMMA:
4954 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
4955 //   -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_WGMMA \
4956 //   -I ../../third-party/cutlass/include \
4957 //   -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm90.bin

```